From numbers to AI

# PROGRAMMING

파이썬

# 질문있나요?

https://discord.gg/mQa86T9NU2

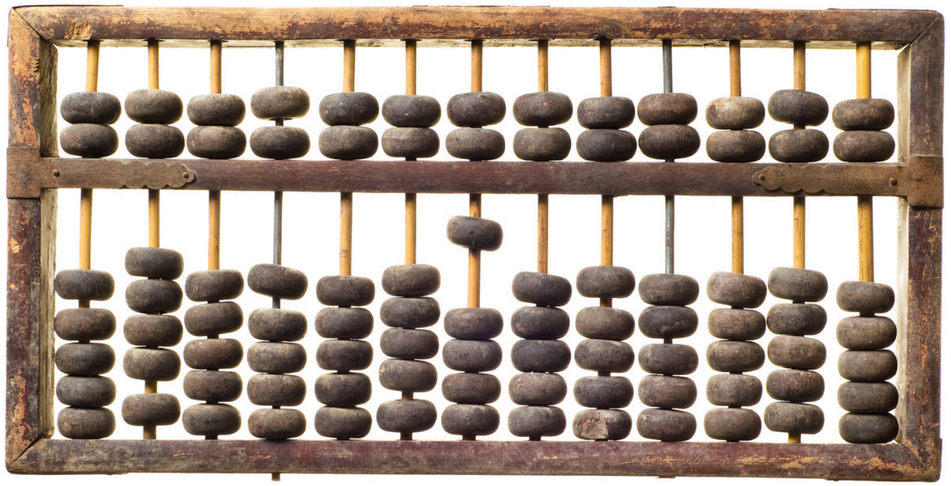
cm@playidealab.com 또는 lcmin@yonsei.ac.kr

| | |
|---|---|
| 모르는건 당연한거지만, 모르는채로 있는건 선택이지요 | 질문하는 과정에서 깨닫기도 합니다 |

# 컴퓨터? 사람?

## 생각에 대하여

25.03.12 이  창 민



?

10:22 AM

https://wikidocs.net/196326

# 컴퓨터의 역사

**1642** 파스칼 계산기: 최초의 기계식 계산기

**1834** 해석기관: 현대 컴퓨터의 개념

**1937** 튜링 머신: 계산의 논리적 모델

**1946** ENIAC: 최초의 범용 전자식 컴퓨터

**1951** UNIVAC-I: 최초의 상업용 전자계산기

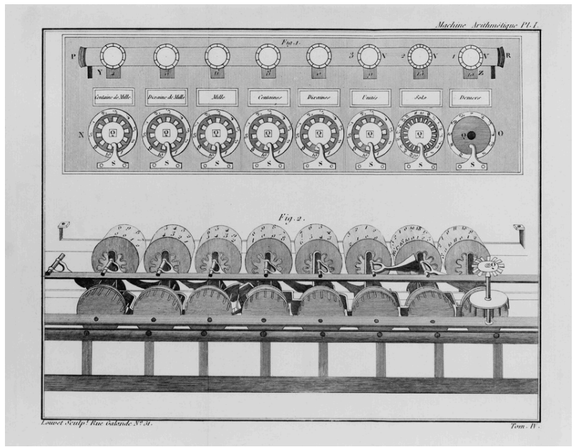
**1971** Intel 4004: 최초의 상용 마이크로프로세서

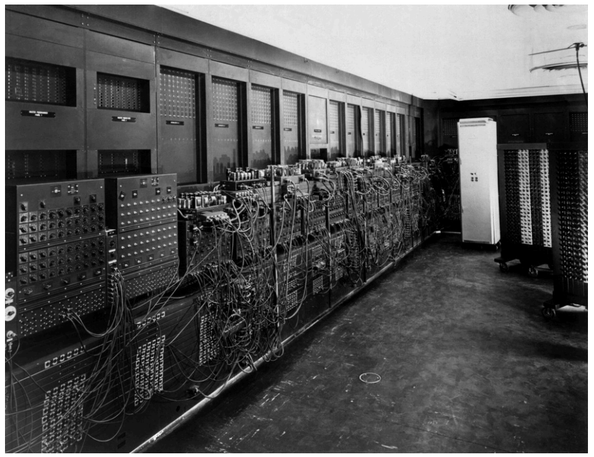
**1981** IBM PC: 개인용 컴퓨터의 표준

**1991** 월드 와이드 웹: 웹의 탄생

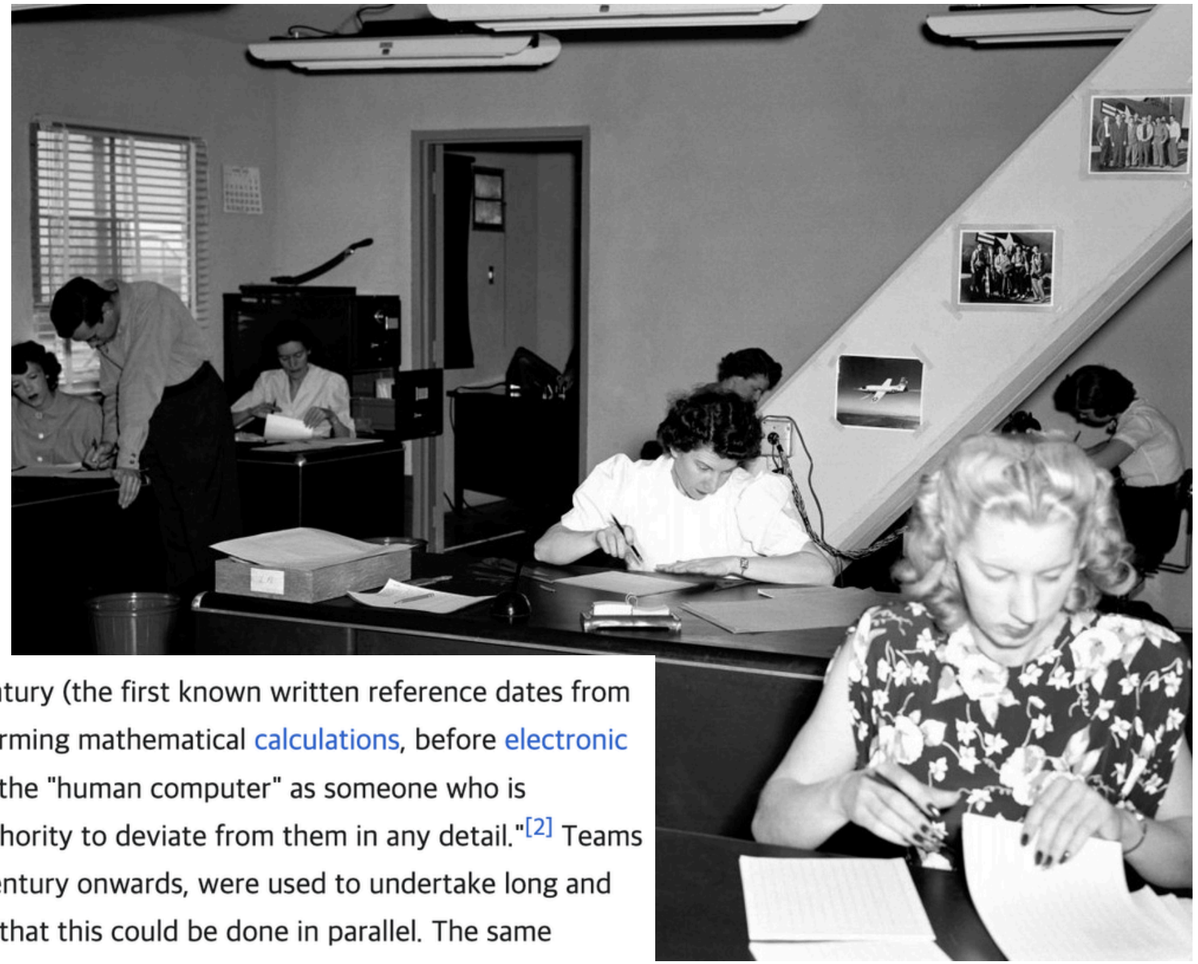
**2011** IBM Watson: 자연어 처리가 가능한 AI

**2020** GPT-3: 대규모 언어 모델





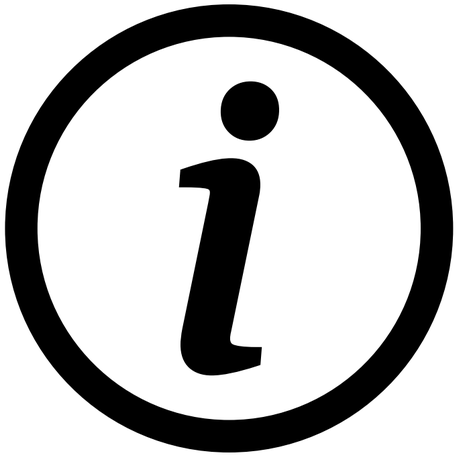
# Compute + - er



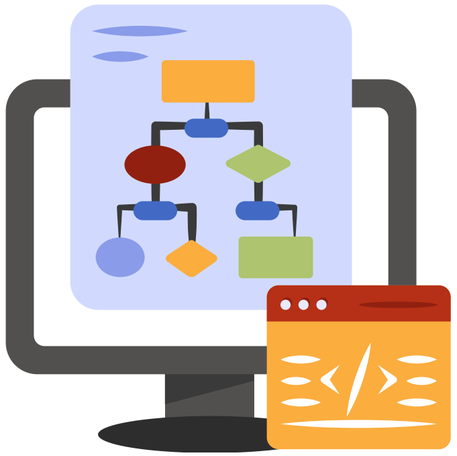
The term "**computer**", in use from the early 17th century (the first known written reference dates from 1613),[1] meant "one who computes": a person performing mathematical calculations, before electronic calculators became available. Alan Turing described the "human computer" as someone who is "supposed to be following fixed rules; he has no authority to deviate from them in any detail."[2] Teams of people, often women from the late nineteenth century onwards, were used to undertake long and often tedious calculations; the work was divided so that this could be done in parallel. The same calculations were frequently performed independently by separate teams to check the correctness of the results.

## computation

문제에 주어진 정보를, 어떠한 알고리즘을 통하여, 조합해 문제의 해답을 구하는 것.



정보



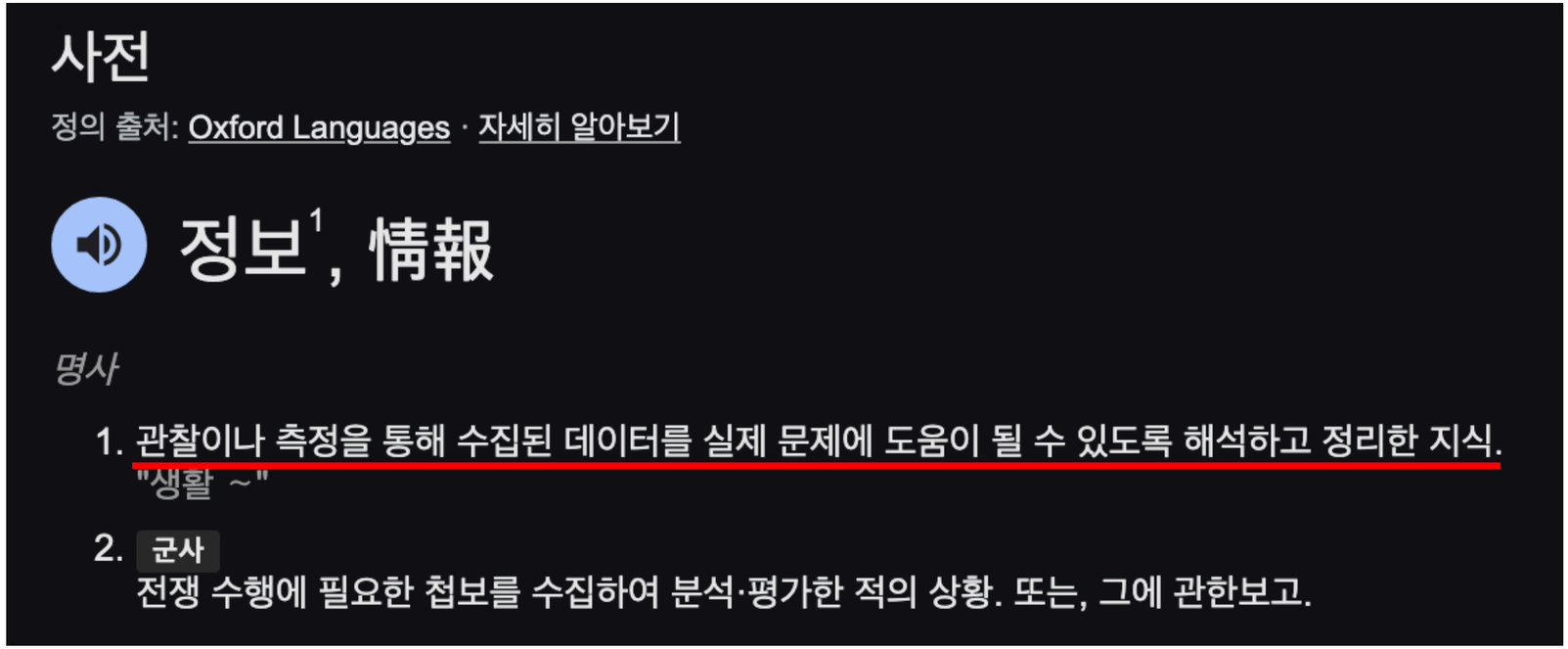
알고리즘

해답

# 정보란?

## 情報

## 느낌(상황)-
## 보고하는 것(방법)



사전

정의 출처: Oxford Languages · 자세히 알아보기

정보[1], 情報

*명사*

1. 관찰이나 측정을 통해 수집된 데이터를 실제 문제에 도움이 될 수 있도록 해석하고 정리한 지식.
"생활 ~"

2. 군사
전쟁 수행에 필요한 첩보를 수집하여 분석·평가한 적의 상황. 또는, 그에 관한보고.

**정보**(情報, 영어: information, info, info.)는 컴퓨터과학에서 특정 목적을 위하여 광(光) 또는 전자적 방식으로 처리되어 부호, 문자, 음성, 음향 및 영상 등을 표현하는 모든 종류의 자료 또는 지식을 말한다.[1]

# 정보

**정보 = 지금의 상황(?)을 표현!**

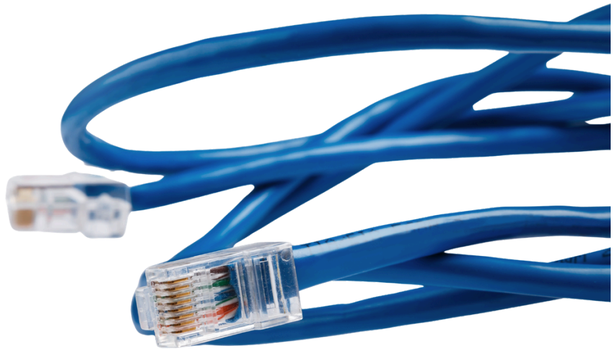
# 정보의 전달

Input

“정보”를
어떻게
전달하나?

Output

8GB



# 신호!

# 信號
# 믿다 - 부호

[정보통신기술용어해설]

---

## Signal 신호

시그널, 신호 세기, 신호 강도, 시스템 신호

▷ Top ▷ 전기전자공학 ▷ 신호 및 시스템 ▷ 신호 표현/성질

---

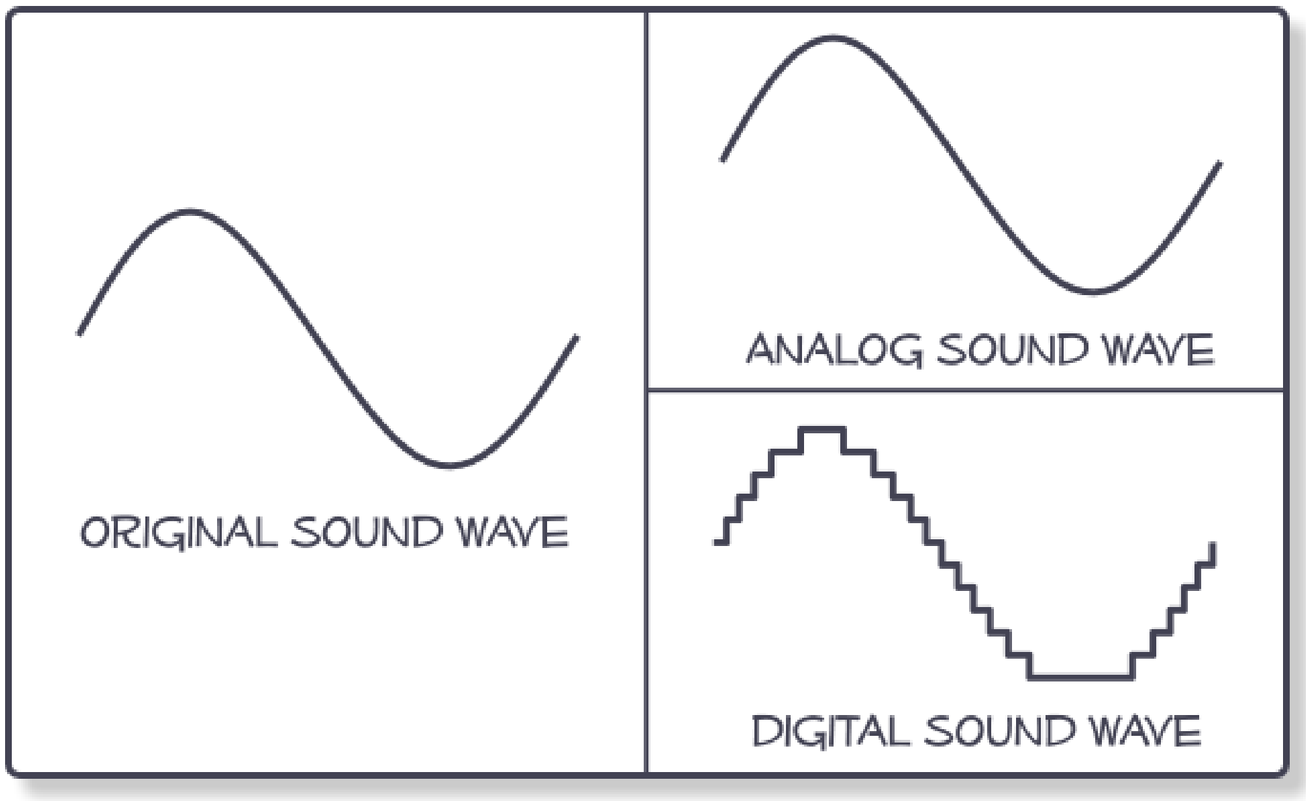
**1. 신호 (Signal)**

- ㅇ 정보를 갖는 것은 모두 신호로 볼 수 있음
- ㅇ 주로, `시간`에 따라 변화하는, 물리량의 수학적 표현을 일컬음
  - 수학적으로, 어떤 현상의 시간,공간에 따른 `변화`를, `함수` 형태로 표현한 것

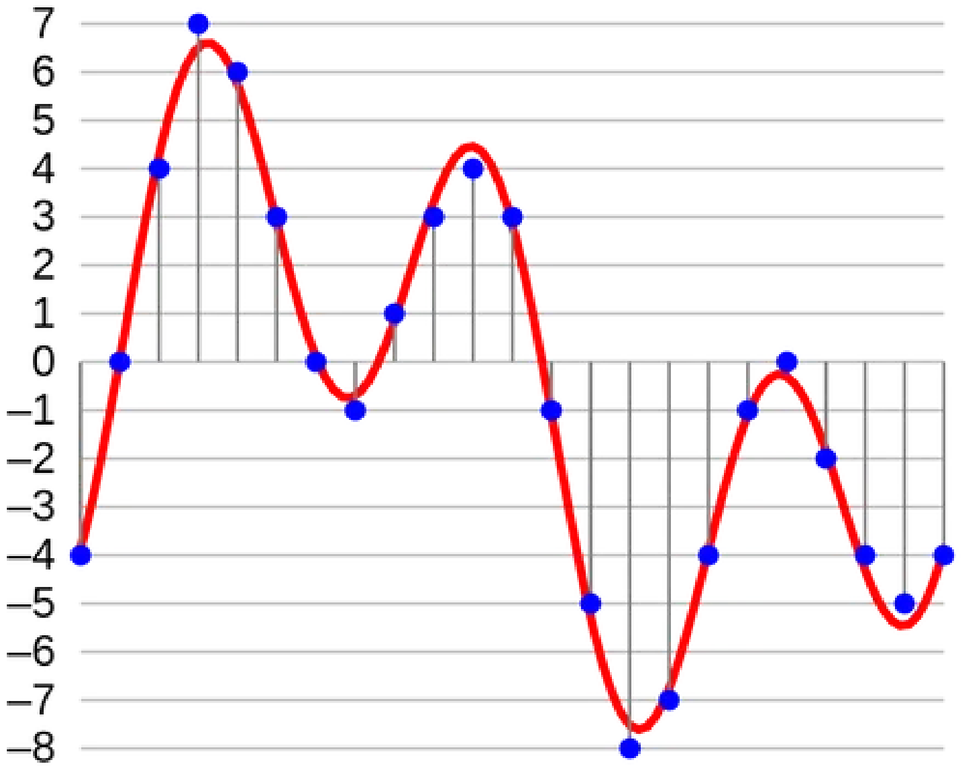
**2. 신호의 의미**

- ㅇ (수학) 1 이상의 독립변수의 함수로 표현 가능
- ㅇ (통신) 정보의 운반체 (시간 또는 주파수의 함수)
  - 정보를 전기적인 파형 형태의 신호로써 전달함
- ㅇ (전산) 프로세스 간 메세지 전달 방법 ☞ 시그널 참조

# 아날로그 신호? 디지털 신호?



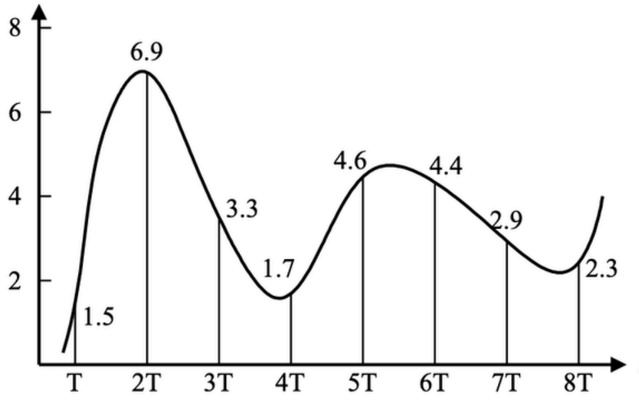
# 양자화:Quantization




양자화(quantization)는 연속적인 양을 기본 단위(양자)의 정수배로 측정하는 것을 뜻합니다. 물리학, 인공지능, 컴퓨터 등 다양한 분야에서 사용됩니다.

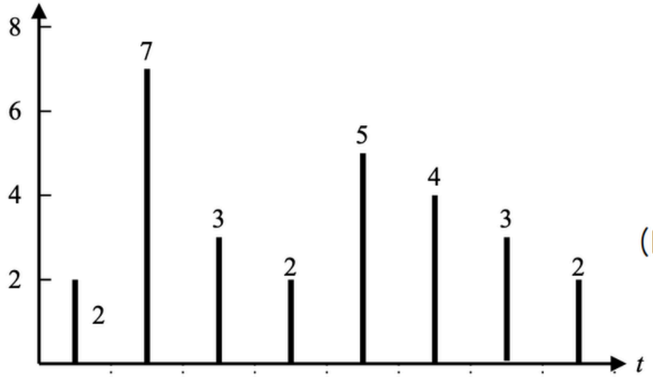
1948 <Information Theory>

| | |
|---|---|
| 표본화 (sampling) | • 샤논(Shannon)의 표본화 정리(sampling theorem) : 신호의 최고 주파수의 2배 이상의 빈도로 샘플링하면 샘플링된 데이터로부터 본래 데이터를 재현 가능<br>• 사람의 음성인 경우 1초 동안에 8000번 샘플링 필요.(2x4KHz=8KHz) |
| 양자화 (quantization) | • 펄스의 진폭의 크기를 디지털 양으로 변환<br>• 이 과정에서 불가피한 양자화 잡음이 발생<br>• 양자화 잡음은 미리 정한 신호레벨의 수를 늘리거나 줄일 수 있으나, 데이터의 양이 많아지는 단점이 있다. |
| 부호화 (coding) | • 부호화는 양자화한 값을 2진 디지털 부호로 변환<br>• 일반적으로 전화 음성에서는 8비트로 부호화 |



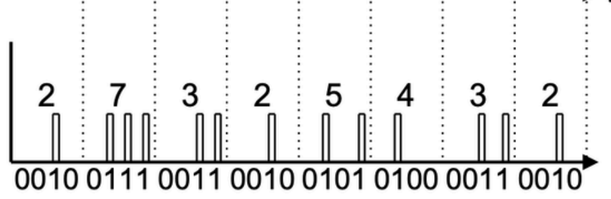

가로로 자르고

(a) 표본화




세로로 반올림

(b) 양자화




펴자!

(c)부호화

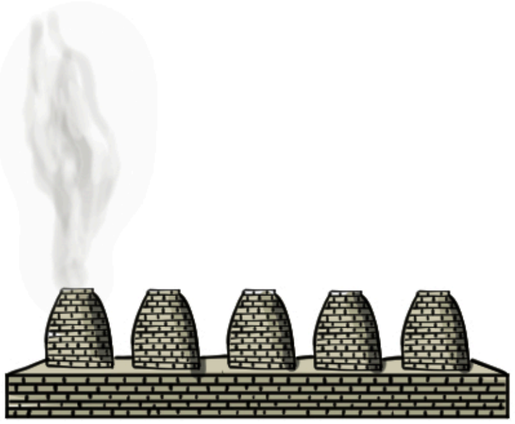
아날로그가 좋은건가요?

디지털이 좋은건가요?

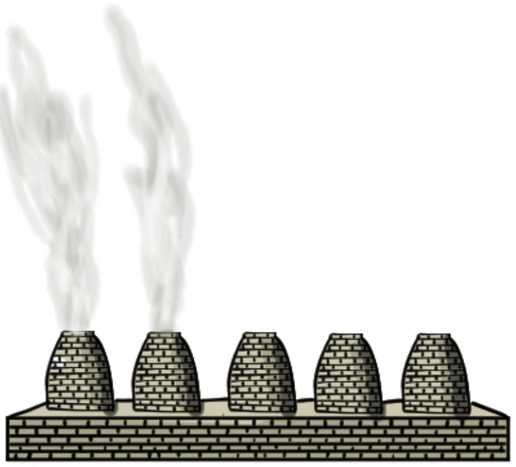
왜 이런건가요?

정보란 무엇인가?

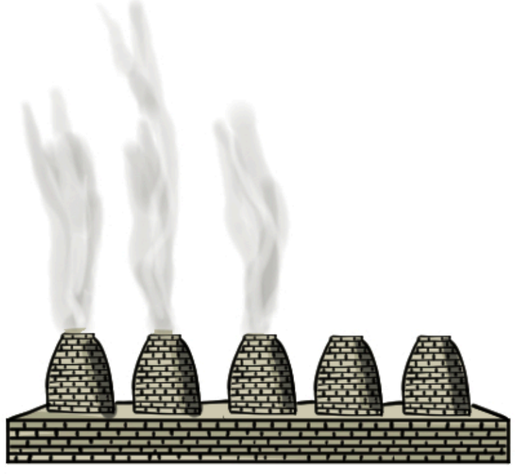
# 정보의 표현



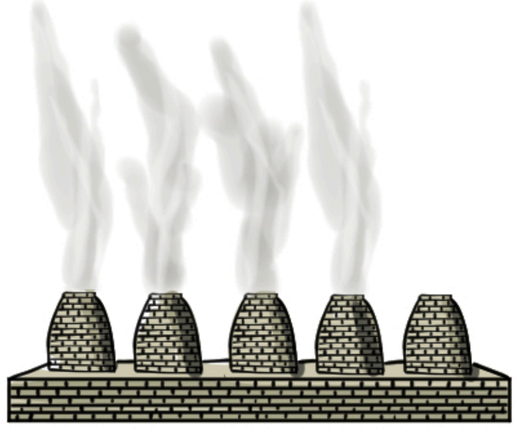
평화로울 때



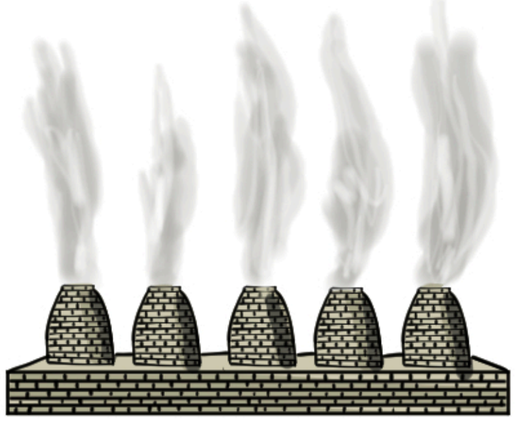
적이 국경 지대에 나타났을 때



적이 국경 가까이 왔을 때



적이 국경을 넘었을 때



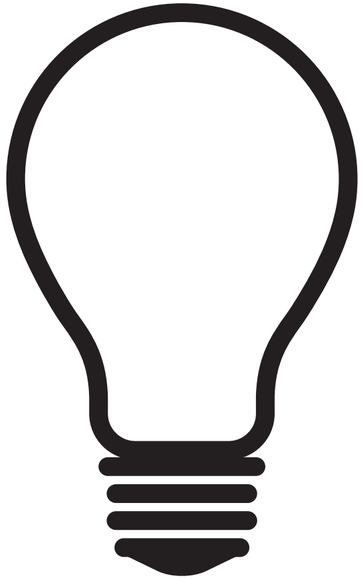
국경을 넘어온 적군과 전투가 벌어졌을 때

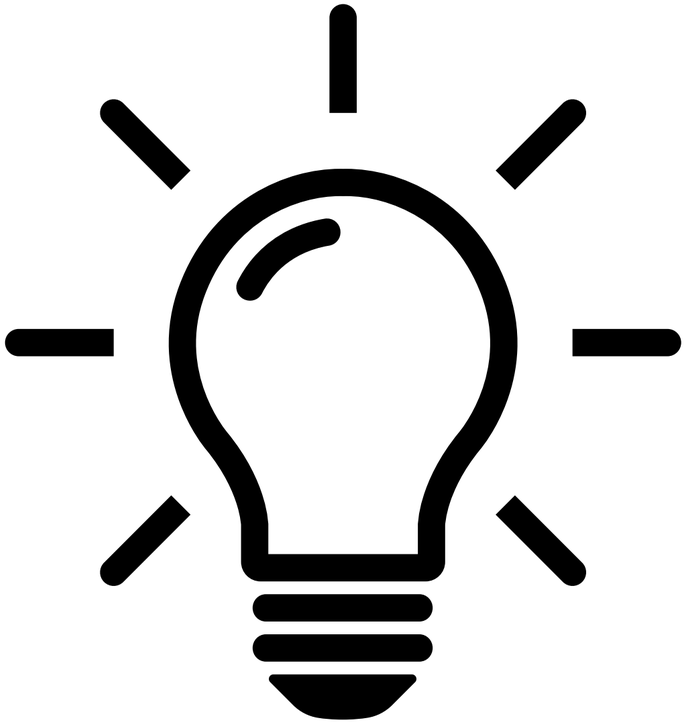
**의미 전달을 위한 약속이 필요함**

**Computation!!!!**

**복잡한 정보를 알고리즘으로 가공해서 해답을 구하고 싶어!!!!**

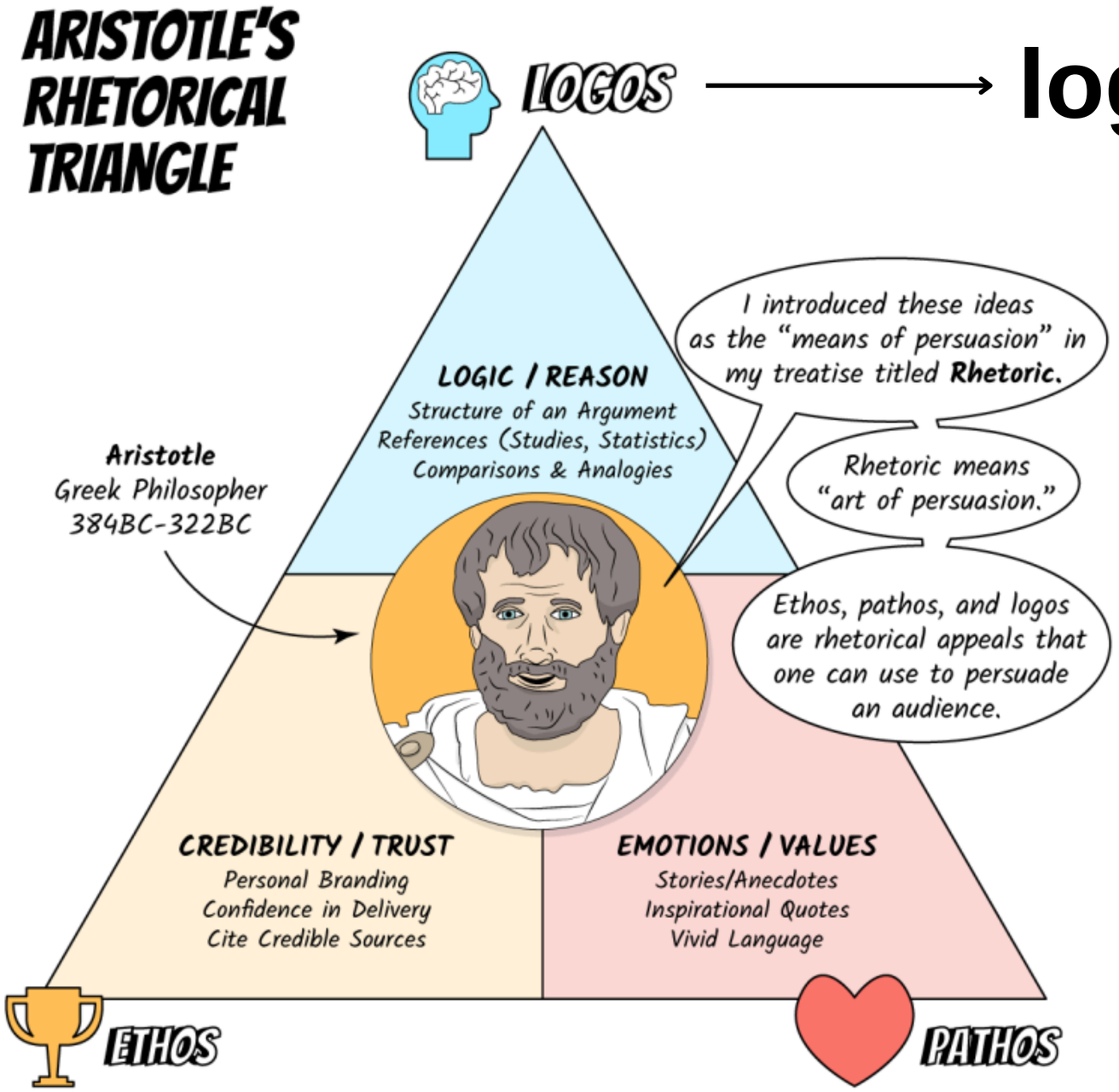
# 알고리즘





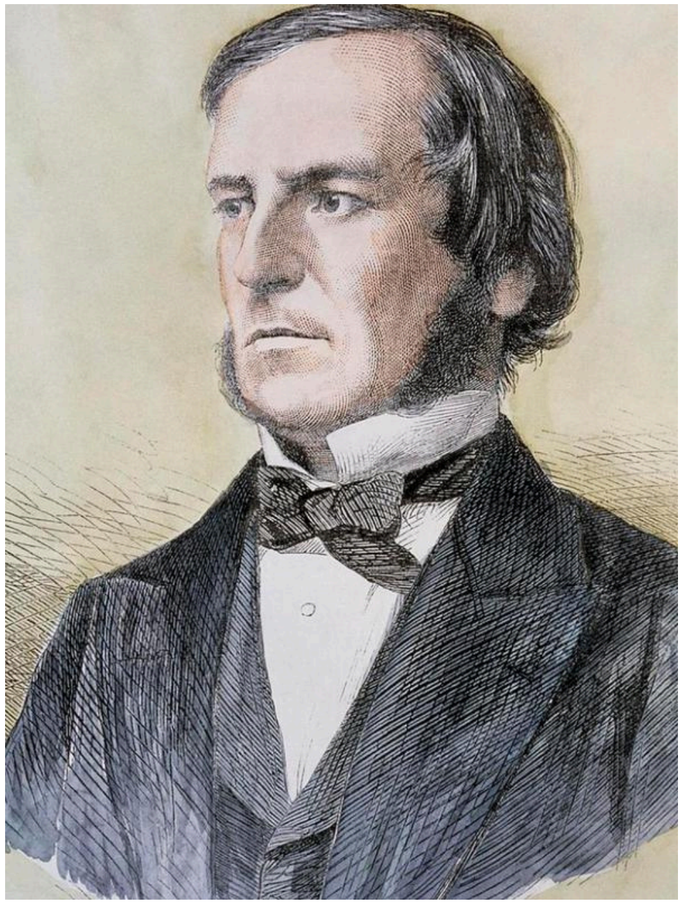
**Logic!**

# 사람을
# 설득하려면?




→ logic! 논리!

# 조지 불



《The Laws of Thought》(1854)

An Investigation of the Laws of Thought
on Which are Founded the Mathematical Theories
of Logic and Probabilities

# Bool Algebra

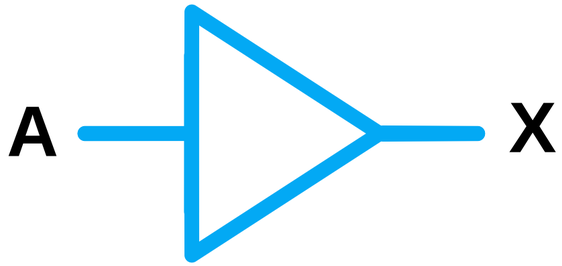
논리학(아리스토텔레스 논리)
논리식을 수학적 연산으로 표현할 수 있다고 주장

1은 참(True)  0은 거짓(False)

AND(∧) → 곱셈: $A \cdot B$
OR(∨) → 덧셈: A+B
NOT(¬) → 보수 연산: 1−A1

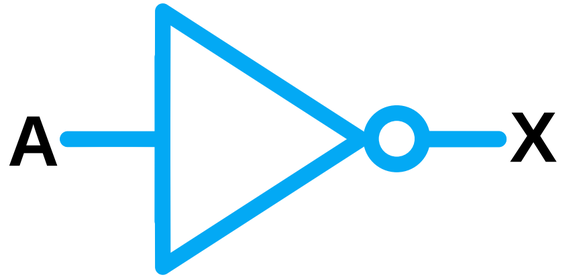
A와 B가 동시에 참이어야 한다 = $A \cdot B$

X=A

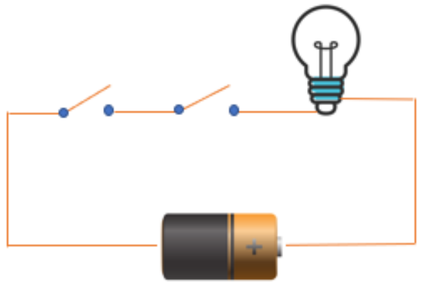


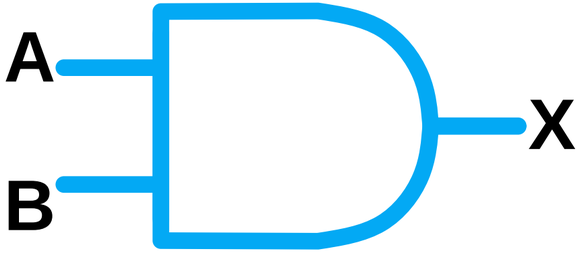

| A | X |
|---|---|
| 0 | 0 |
| 1 | 1 |

X=A’




| A | X |
|---|---|
| 0 | 1 |
| 1 | 0 |

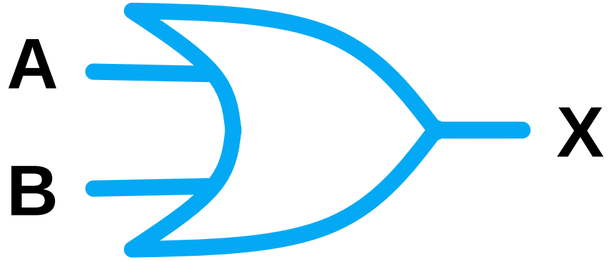





| A | B | X |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

**X=AB**

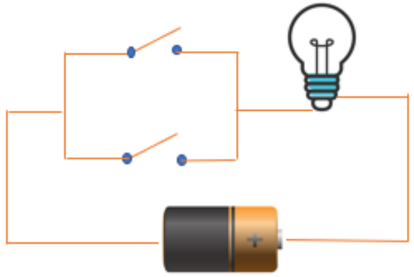
**= A and B**

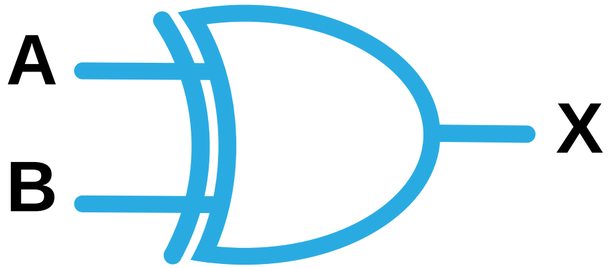



| A | B | X |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

**X=A+B**

**= A or B**






| A | B | A' | B' | AB' | A'B | AB'+A'B | A XOR B |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |

**X = AB’ + A’B**

**= A  xor B**

# Dual Form

| Name | AND form | OR form |
|---|---|---|
| Identity law | $1A = A$ | $0 + A = A$ |
| Null law | $0A = 0$ | $1 + A = 1$ |
| Idempotent law | $AA = A$ | $A + A = A$ |
| Inverse law | $A\bar{A} = 0$ | $A + \bar{A} = 1$ |
| Commutative law | $AB = BA$ | $A + B = B + A$ |
| Associative law | $(AB)C = A(BC)$ | $(A + B) + C = A + (B + C)$ |
| Distributive law | $A + BC = (A + B)(A + C)$ | $A(B + C) = AB + AC$ |
| Absorption law | $A(A + B) = A$ | $A + AB = A$ |
| De Morgan's law | $\overline{AB} = \bar{A} + \bar{B}$ | $\overline{A + B} = \bar{A}\bar{B}$ |

**1(True) and A = A**

**1(True) and True = True**

**1(True) and False = False**

# Dual Form

| Name | AND form | OR form |
|---|---|---|
| Identity law | $1A = A$ | $0 + A = A$ |
| Null law | $0A = 0$ | $1 + A = 1$ |
| Idempotent law | $AA = A$ | $A + A = A$ |
| Inverse law | $A\bar{A} = 0$ | $A + \bar{A} = 1$ |
| Commutative law | $AB = BA$ | $A + B = B + A$ |
| Associative law | $(AB)C = A(BC)$ | $(A + B) + C = A + (B + C)$ |
| Distributive law | $A + BC = (A + B)(A + C)$ | $A(B + C) = AB + AC$ |
| Absorption law | $A(A + B) = A$ | $A + AB = A$ |
| De Morgan's law | $\overline{AB} = \bar{A} + \bar{B}$ | $\overline{A + B} = \bar{A}\bar{B}$ |

· 0 → 1, 1 → 0

· AND → OR, OR → AND

0(False) or A = A

0(False) or True = True

0(False) or False = False

같다!!!

# 클로드 섀넌

**A Symbolic Analysis of Relay and Switching Cir cuits**

**역사상 가장 위대한 석사논문..**
**당시 21살....**

**1937 MIT 대학원생 석사과정**
**아날로그 컴퓨터의 전기회로를 만지던중...**

**깨닳음. “전기스위치 == 불 대수”**

왜??

**그러면? “전기회로 == 논리식”**

디지털 회로 탄생!

**이후 컴퓨터 탄생!**

# A Symbolic Analysis of Relay and Switching Circuits*

*Claude E. Shannon***

## I. Introduction

In the control and protective circuits of complex electrical systems it is frequently necessary to make intricate interconnections of relay contacts and switches. Examples of these circuits occur in automatic telephone exchanges, industrial motor-control equipment, and in almost any circuits designed to perform complex operations automatically. In this paper a mathematical analysis of certain of the properties of such networks will be made. Particular attention will be given to the problem of network synthesis. Given certain characteristics, it is required to find a circuit incorporating these characteristics. The solution of this type of problem is not unique and methods of finding those particular circuits requiring the least number of relay contacts and switch blades will be studied. Methods will also be described for finding any number of circuits equivalent to a given circuit in all operating characteristics. It will be shown that several of the well-known theorems on impedance networks have roughly analogous theorems in relay circuits. Notable among these are the delta-wye and star-mesh transformations, and the duality theorem.

The method of attack on these problems may be described briefly as follows: any circuit is represented by a set of equations, the terms of the equations corresponding to the various relays and switches in the circuit. A calculus is developed for manipulating these equations by simple mathematical processes, most of which are similar to ordinary algebraic algorisms. This calculus is shown to be exactly analogous to the calculus of propositions used in the symbolic study of logic. For the synthesis problem the desired characteristics are first written as a system of equations, and the equations are then manipulated into the form representing the simplest circuit. The circuit may then be immediately drawn from the equations. By this method it is always possible to find the simplest circuit containing only series and parallel connections, and in some cases the simplest circuit containing any type of connection.

Our notation is taken chiefly from symbolic logic. Of the many systems in common use we have chosen the one which seems simplest and most suggestive for our interpretation. Some of our phraseology, such as node, mesh, delta, wye, etc., is borrowed from ordinary network theory for simple concepts in switching circuits.

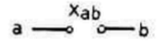



Figure 1 (left). Symbol for hindrance function

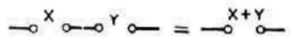



Figure 2 (right). Interpretation of addition

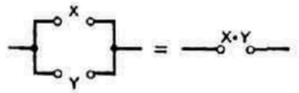



Figure 3 (middle). Interpretation of multiplication

It is evident that with the above definitions the following postulates will hold:

$$X + Y = Y + X\,, \tag{1a}$$

$$XY = YX\,, \tag{1b}$$

$$X + (Y + Z) = (X + Y) + Z\,, \tag{2a}$$

$$X(YZ) = (XY)Z\,, \tag{2b}$$

$$X(Y + Z) = XY + XZ\,, \tag{3a}$$

$$X + YZ = (X + Y)(X + Z)\,, \tag{3b}$$

$$1 \cdot X = X\,, \tag{4a}$$

$$0 + X = X\,, \tag{4b}$$

$$1 + X = 1\,, \tag{5a}$$

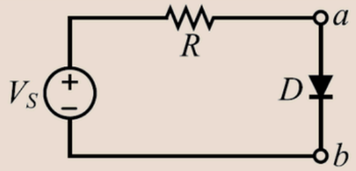
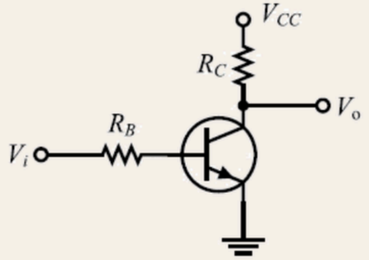
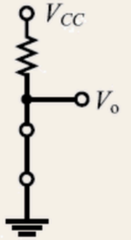
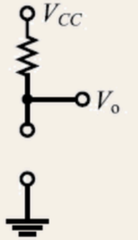
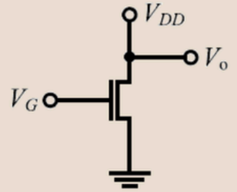
$$0 \cdot X = 0\,. \tag{5b}$$

https://www.cs.virginia.edu/~evans/greatworks/shannon38.pdf

The method of attack on these problems may be described briefly as follows: any circuit is represented by a set of equations, the terms of the equations corresponding to the various relays and switches in the circuit. A calculus is developed for manipulating these equations by simple mathematical processes, most of which are similar to ordinary algebraic algorisms. This calculus is shown to be exactly analogous to the calculus of propositions used in the symbolic study of logic. For the synthesis problem the desired characteristics are first written as a system of equations, and the equations are then manipulated into the form representing the simplest circuit. The circuit may then be immediately drawn from the equations. By this method it is always possible to find the simplest circuit containing only series and parallel connections, and in some cases the simplest circuit containing any type of connection.

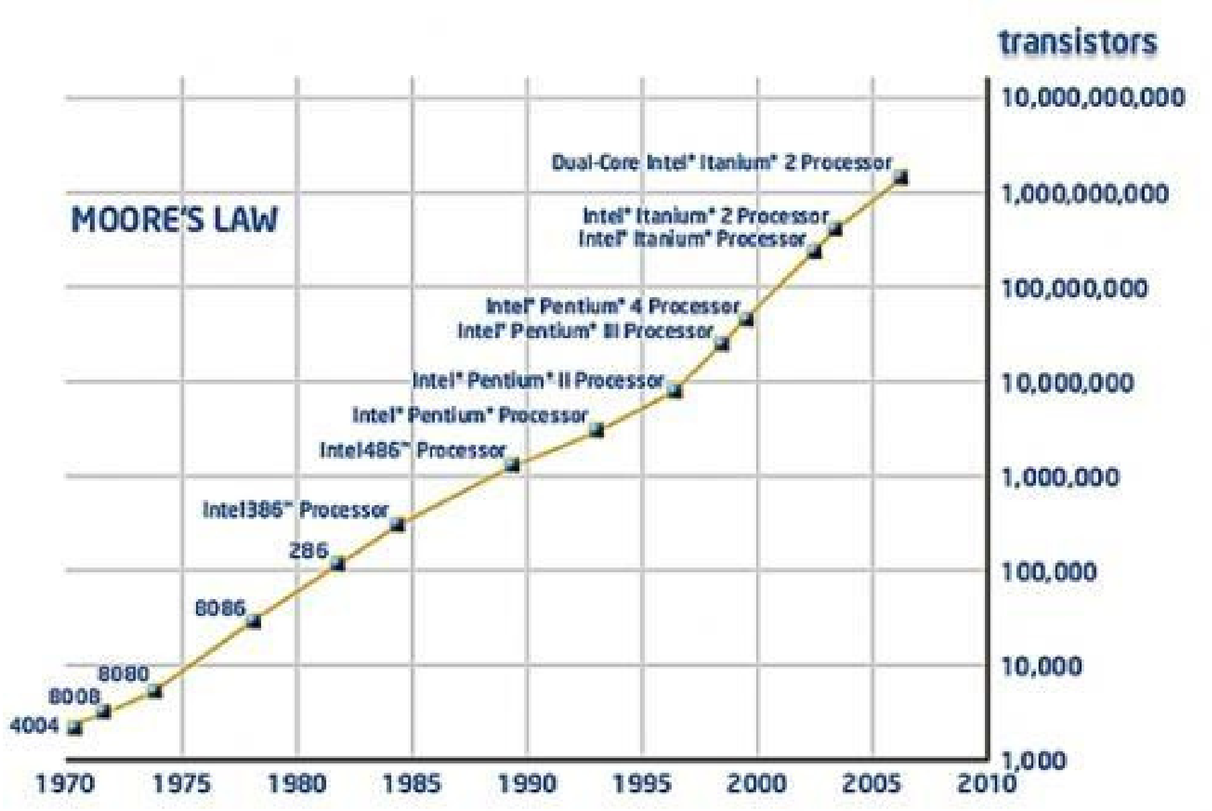
**The circuit may then be immediately drawn from the equations.**

# 더 좋은 전기스위치!!!!

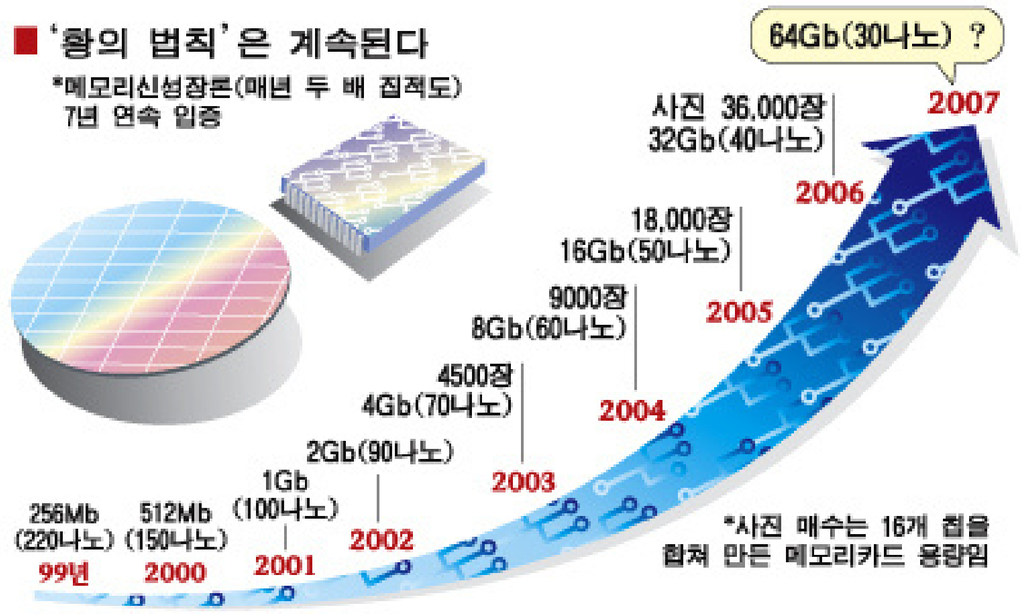
| | | | |
|---|---|---|---|
| 다이오드에 의한 스위칭 |  | $V_S$=5V | $V_S$=0V |
| 쌍극성 트랜지스터에 의한 스위칭 |  |  $V_i$=$V_{cc}$<br>*switch close*<br>$V_o$=0V |  $V_i$=0V<br>*switch open*<br>$V_o$=$V_{cc}$ |
| NMOS 트랜지스터에 의한 스위칭 |  | $V_G$=$V_{DD}$<br>*switch close*<br>$V_o$=0V | $V_G$=0V<br>*switch open*<br>$V_o$=$V_{DD}$ |

**반도체: Semi-conductor 0,1**

# 더 많은 전기 스위치!!!




무어의 법칙!
트린지스터의 집적 숫자가 18~24개월마다 2배!




황의법칙!
반도체 메모리 용량이 1년마다 2배!

'엔비디아 황의 법칙'은 인공지능(AI)을 작동시키는 실리콘칩의 성능이 2년마다 2배 이상 향상한다는 법칙이다. 미국 반도체 기업 엔비디아의 공통 창업자이자 최고경영자(CEO)인 젠슨 황이 주창한 것이다. 2021. 2. 25.

einfomax.co.kr
http://news.einfomax.co.kr › news › articleView

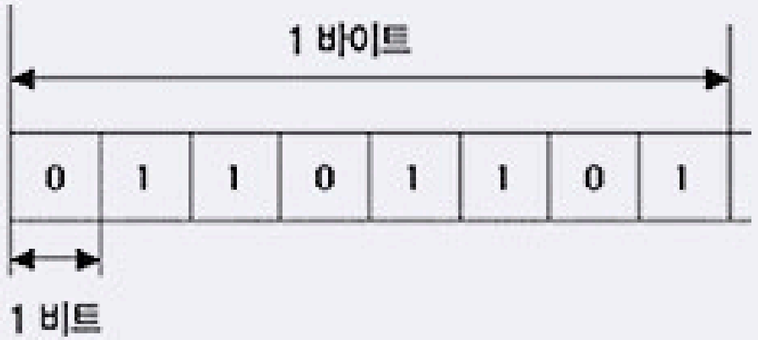
[시사금융용어] 엔비디아 황의 법칙 < 시사용어 < 기사본문

# 1bit = 스위치 하나

나타낼 수 있는 정보는 0 or 1

큰 정보는 더 많은 bit!!!

# = 비트와 바이트 =

| 비트 (binary digit, bit) | 바이트 (Byte) |
| --- | --- |
| 0과 1, 두 가지 값만<br>가질 수 있는 측정 단위 | 여덟 개의 비트로 구성된<br>데이터의 양을 나타내는 단위 |
| 0 OFF FALSE<br> 1 ON TRUE |  |

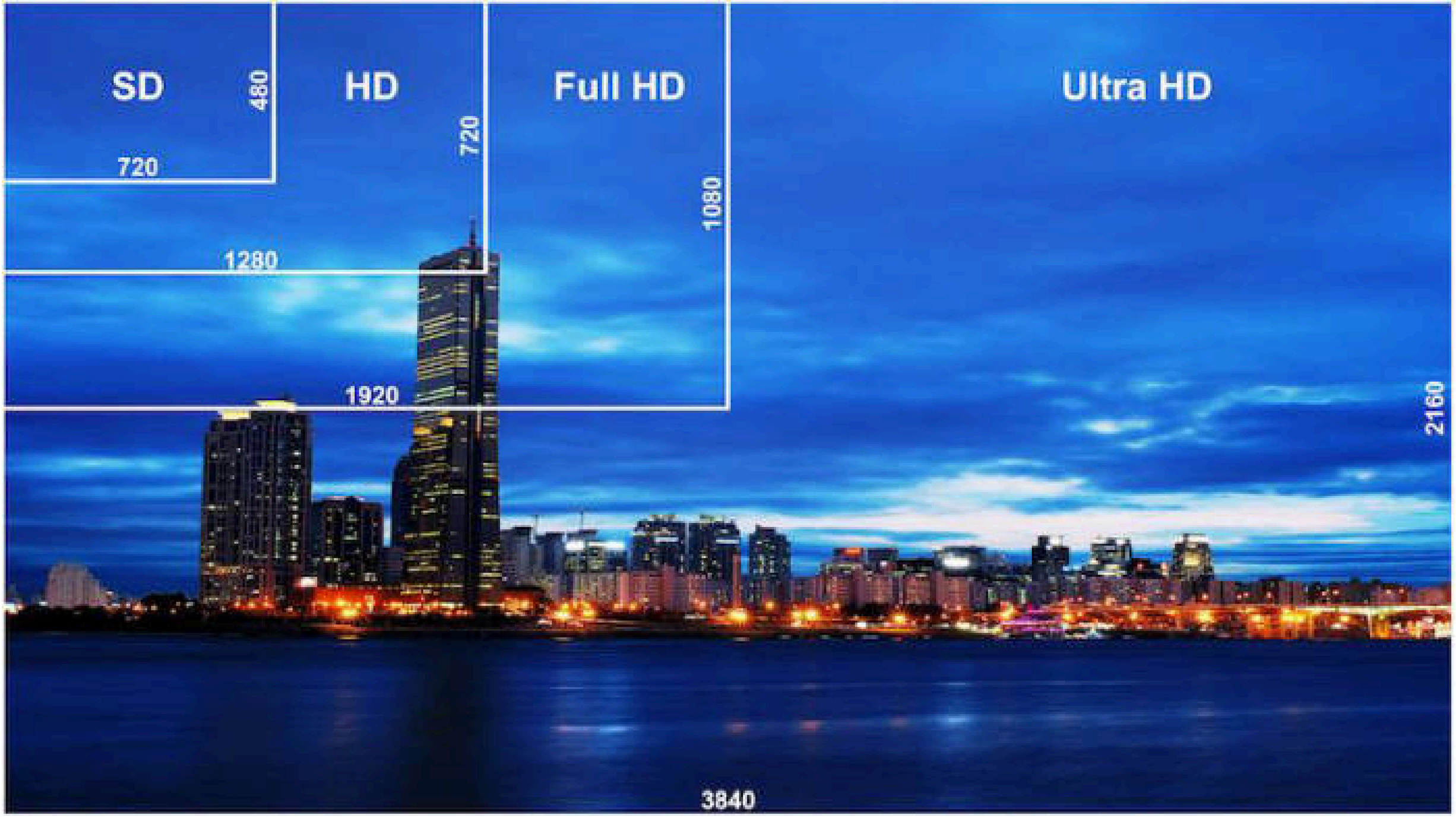
https://news.samsungsemiconductor.com/kr/%EC%95%84%EB%A6%AC%EC%86%A1%ED%95%9C-%EB%8D%B0%EC%9D%B4%ED%84%B0%EC%9D%98-%EB%8B%A8%EC%9C%84-%EB%B0%94%EC%9D%B4%ED%8A%B8byte/

삼성반도체이야기

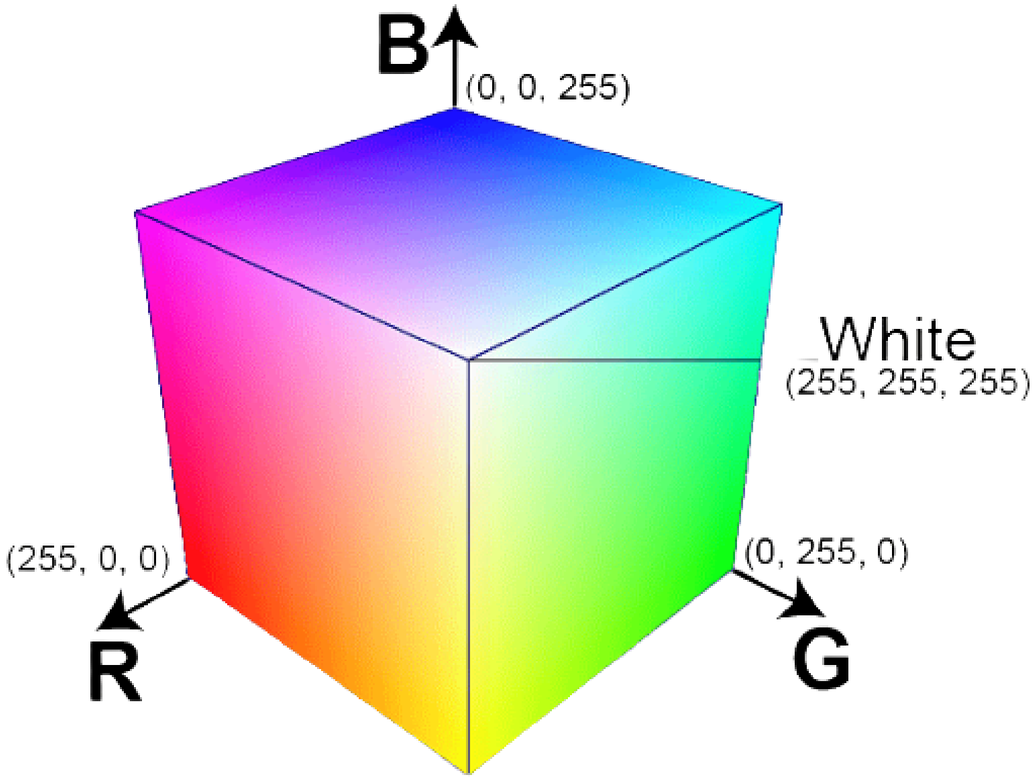
# = 데이터 단위 =

| 단위 | 정의 | 저장공간 크기 |
|---|---|---|
| 비트(bit) | 0 또는 1 | 예 / 아니오 |
| 1바이트(Byte) | 8 bit | 알파벳과 숫자 한 개 |
| 1킬로바이트(KB) | 1,024 Byte | 몇 개의 문단 |
| 1메가바이트(MB) | 1,024 KB | 1분 길이의 MP3 노래 |
| 1기가바이트(GB) | 1,024 MB | 30분 길이의 HD 영화 |
| 1테라바이트(TB) | 1,024 GB | 약 200편의 FHD 영화 |

삼성반도체이야기
samsungsemiconstory.com


SD
480
720
HD
720
1280
Full HD
1080
1920
Ultra HD
2160
3840

# 30분 길이의 영상의 용량은?



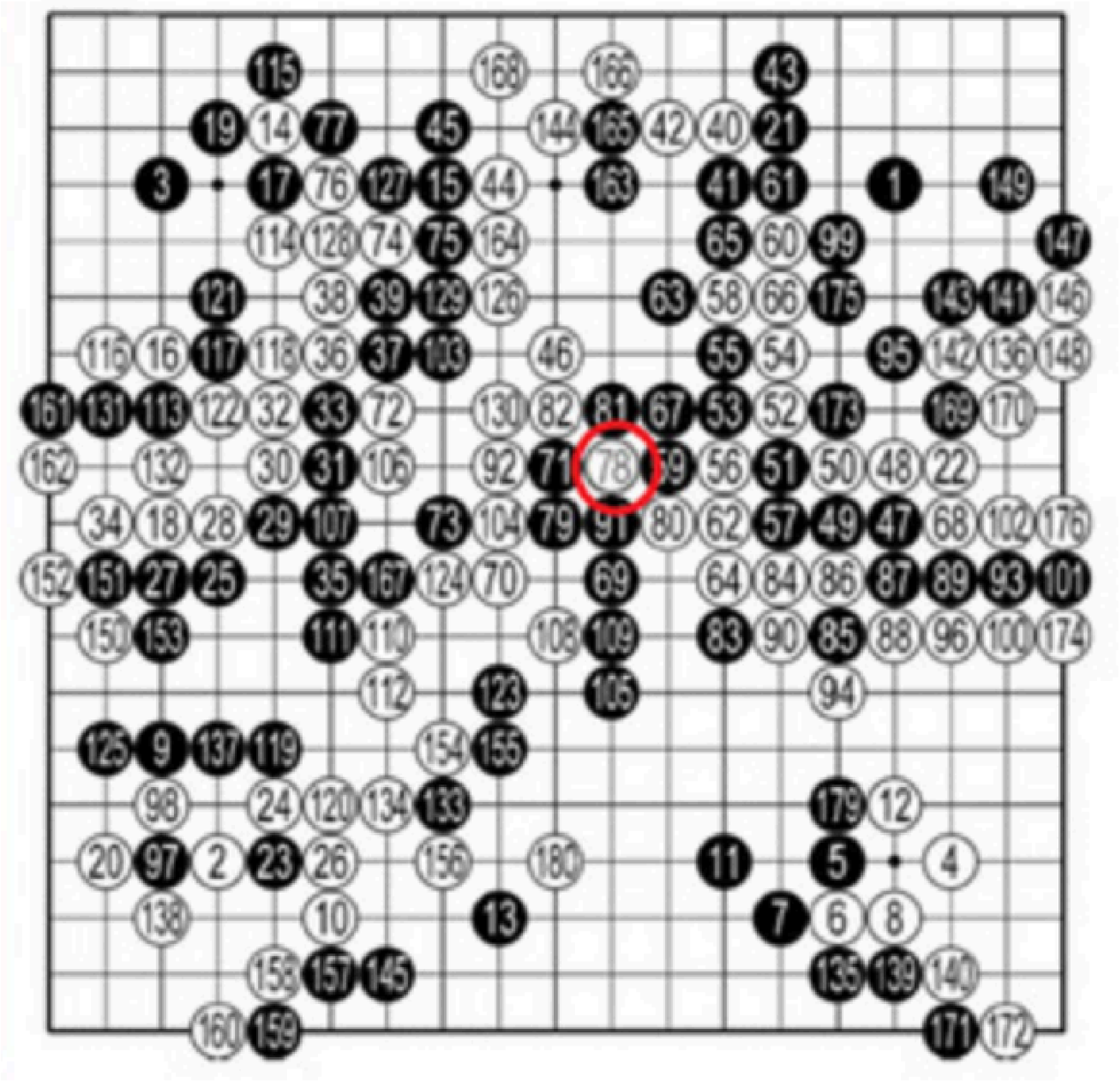

**(0~255,0~255,0~255)**
**3바이트 이상.**

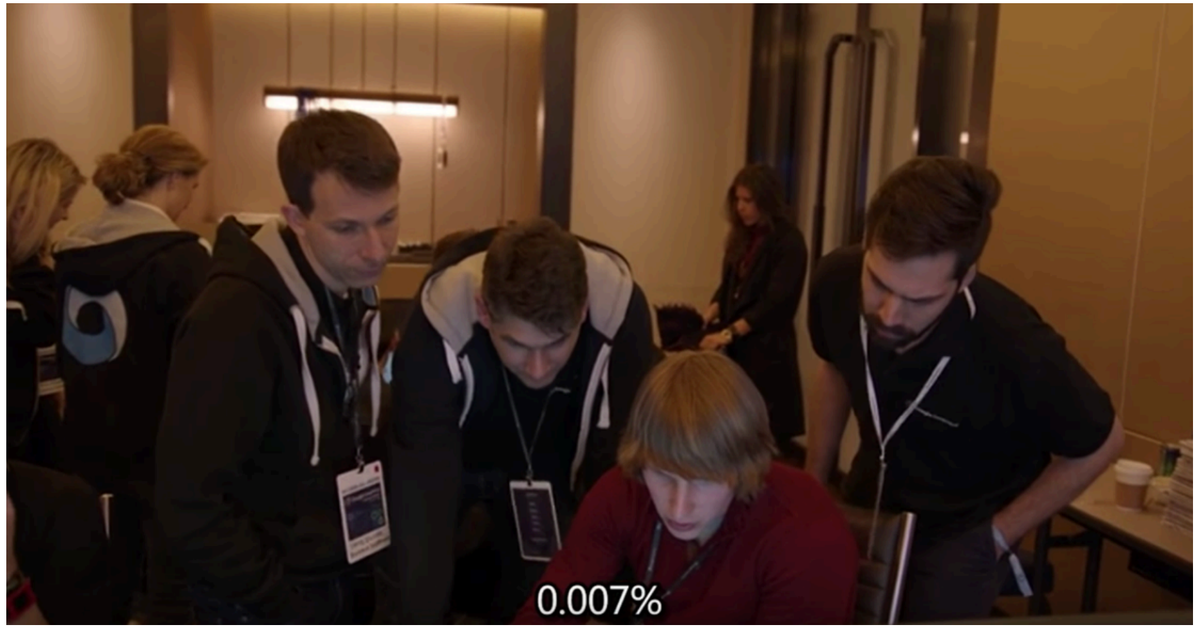
- 픽셀개수 = 영상크기: 1280x720
- 색상 = RGB: 3 byte
- 1초에 60프레임, 30분이면...

W1280*H720*FPS60*Min30*sec60*RGB3 Byte
= 298,598,400,000 Byte
= **298 GB** ??????????

# 정보의 가치는 언제나 같은가?

# 아자 황 ?



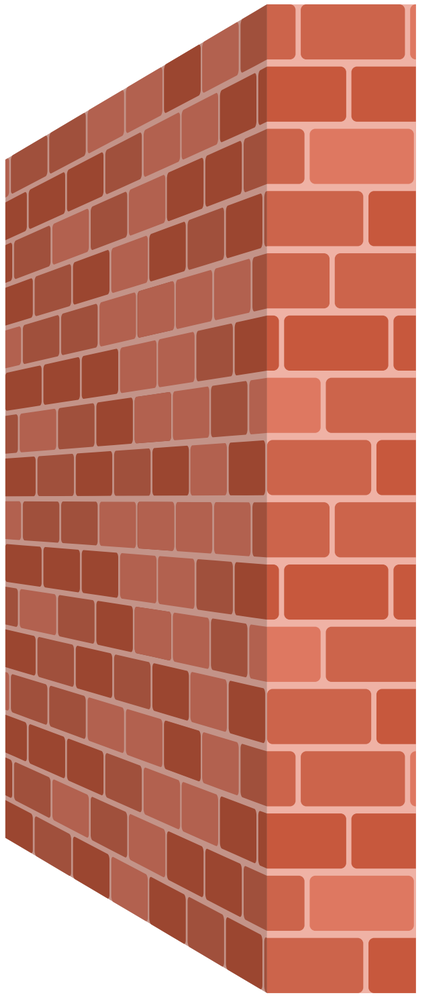


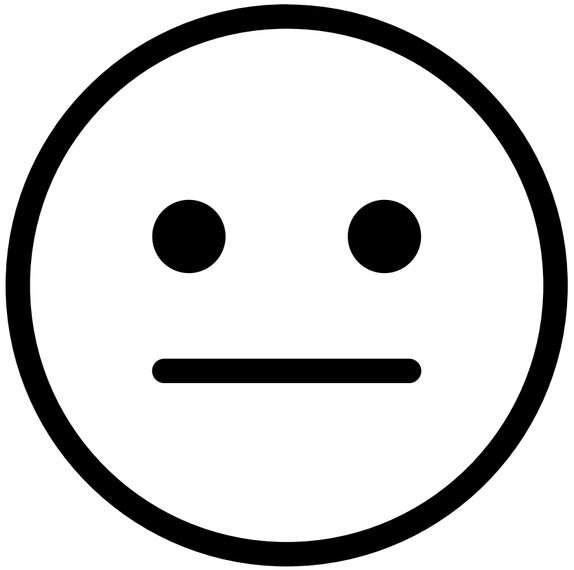

0.07%...

100%

# 정보를 어떻게 표현할 수 있을까?

지구는 둥근가?

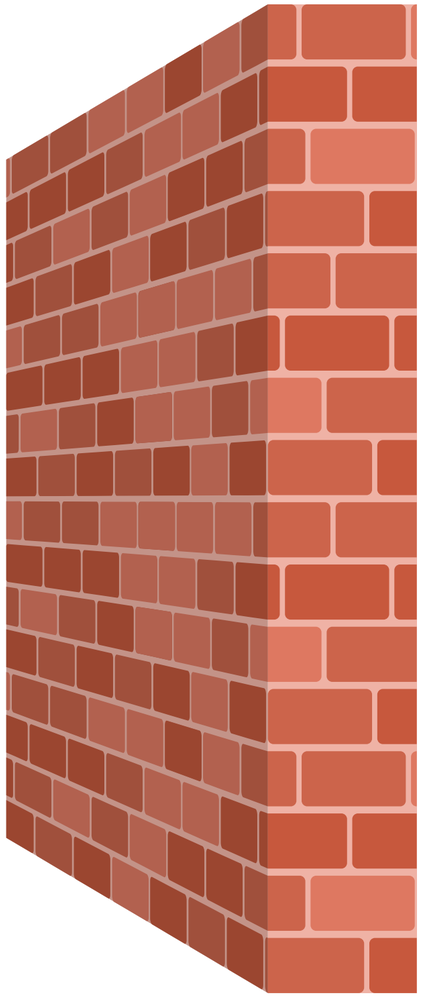


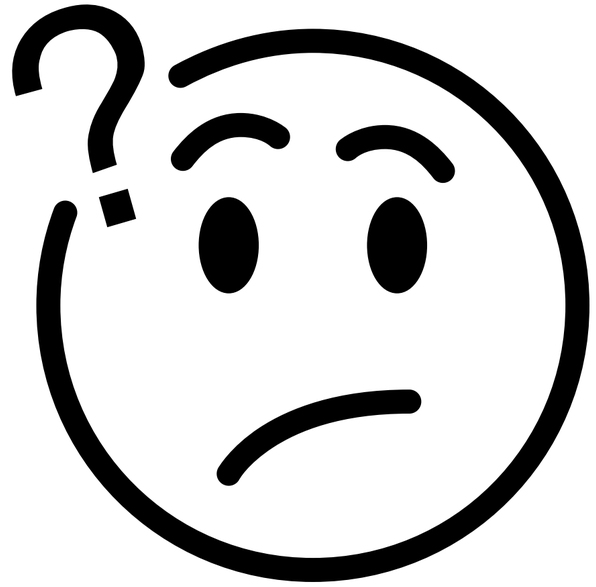


당연하지..

# 정보의 표현

동전은 앞면일까?
뒷면일까?

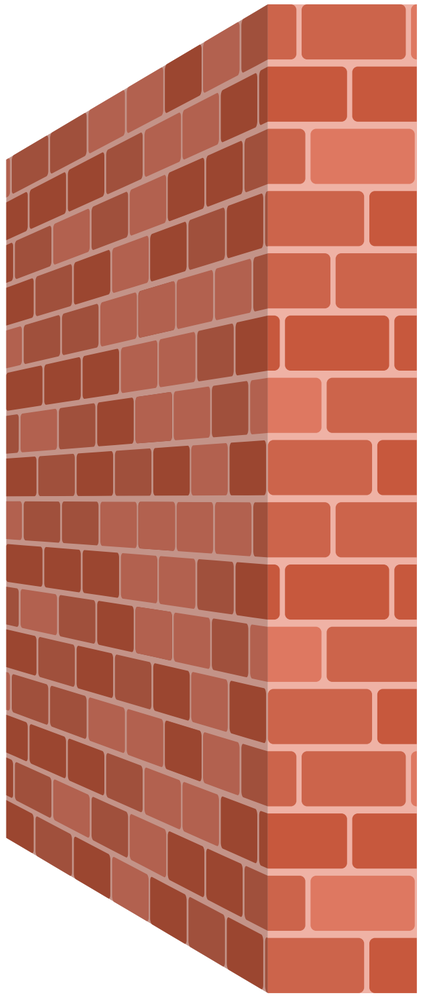


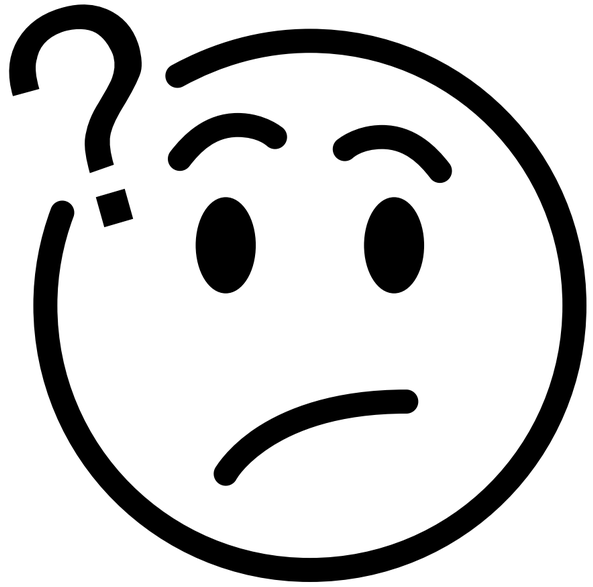


반반..

# 정보의 표현

주사위가 어떤 수가 나왔을까?

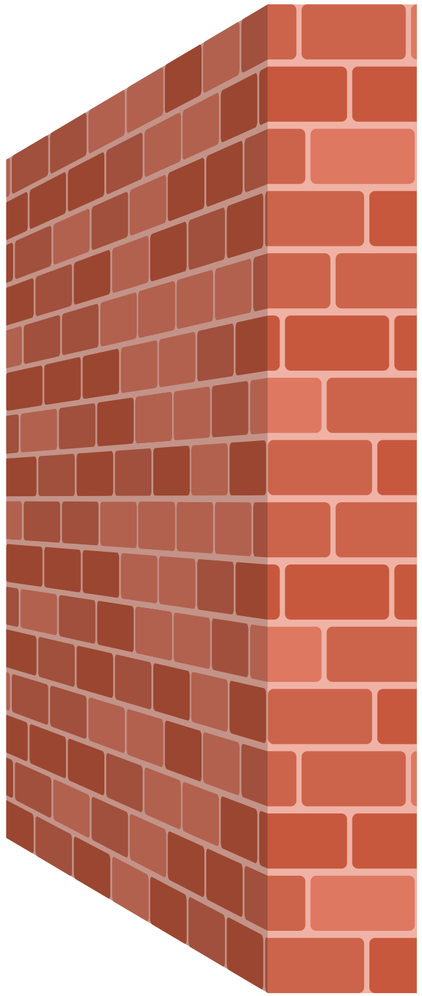


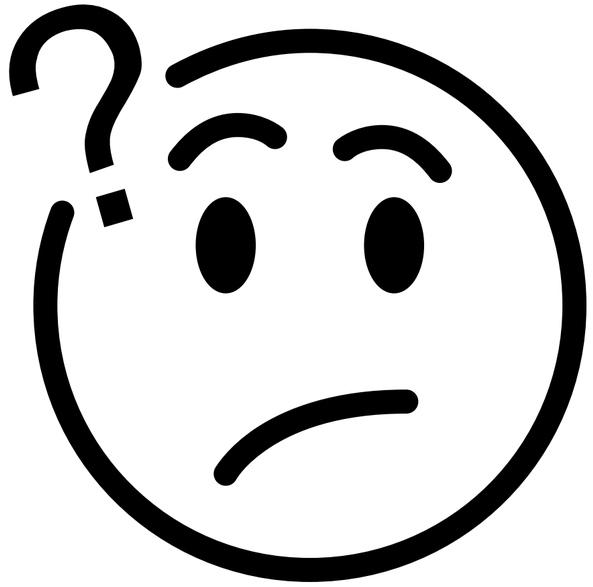


1/6...

# 정보의 표현

주사위가 어떤 수가 나왔을까?

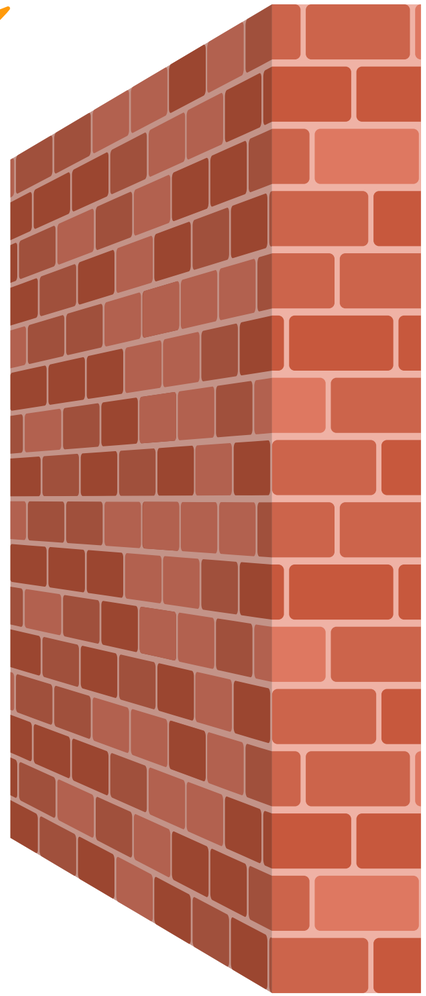


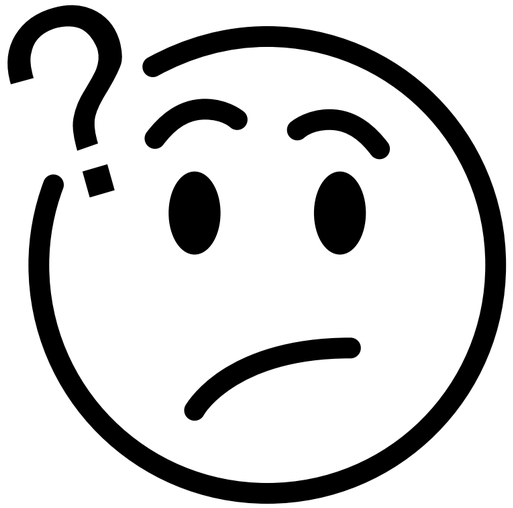


?!

# 정보의 표현

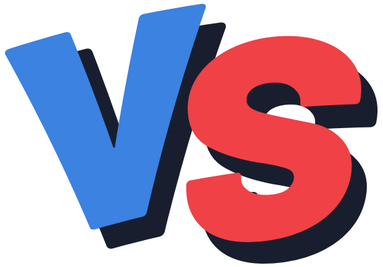
지구는 ...?!





!!!!!!!!

사건이 일어나는지 일어나지 않는지?




얼마나 희귀한 사건이 일어나는지?

# 정보량

$$I(X) = log(\frac{1}{p(x)})$$

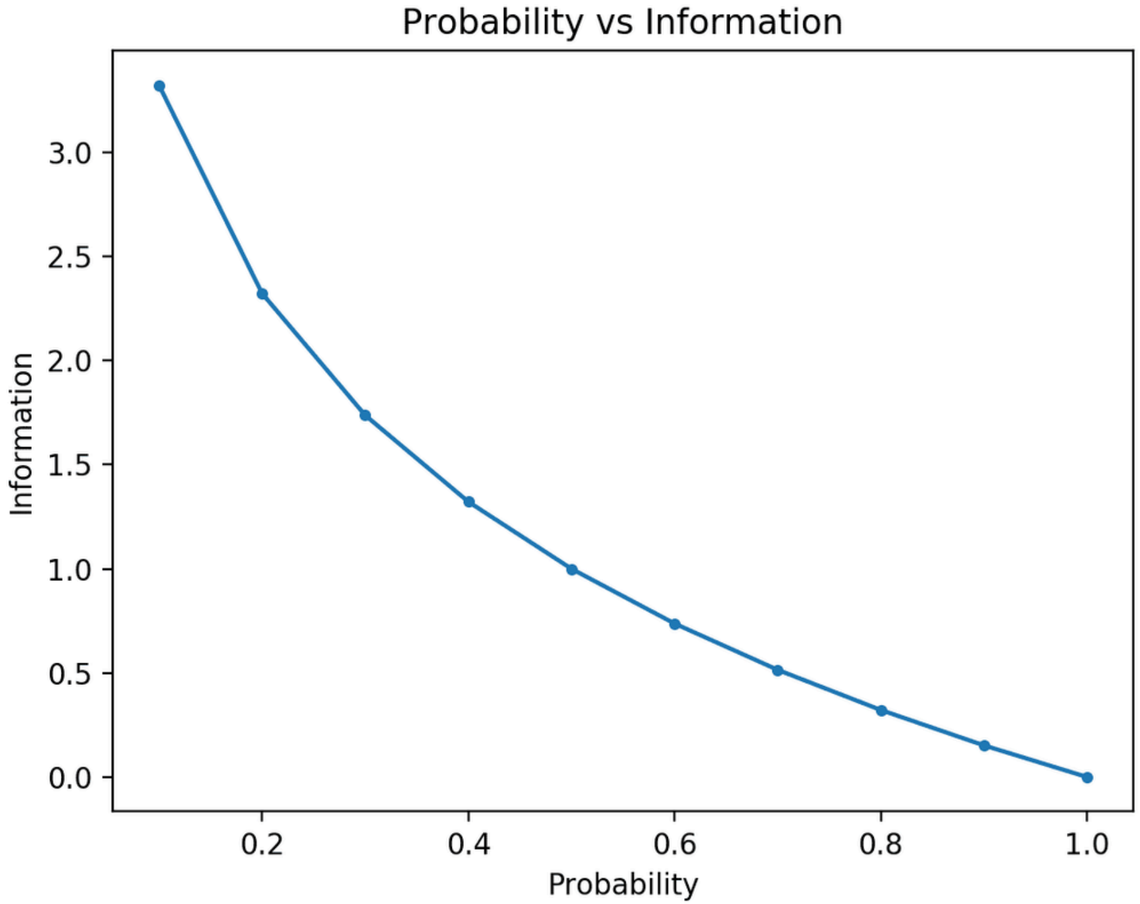
$$= -log(p(x))$$

# 왜 확률의 역수인가?

# 왜 로그인가?

$$I(X) = log(\frac{1}{p(x)})$$

$$= -log(p(x))$$



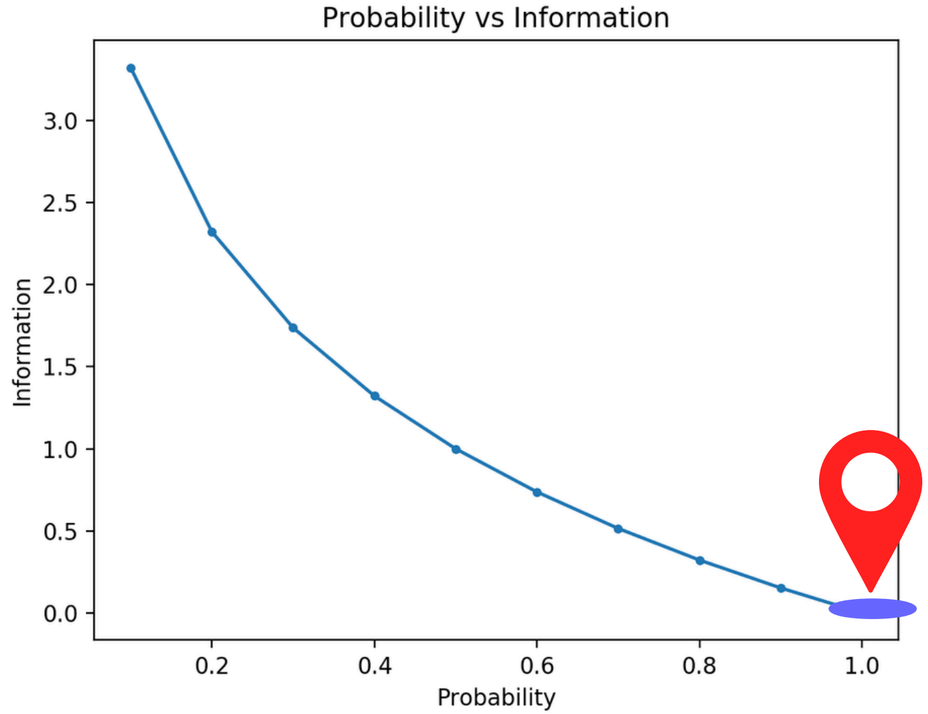
$$\lim_{p \to 0} p \log_2 p = 0$$

$$p(X = 1) = 1$$

$$p(X = 0) = 0$$

$$I(X) = log(\frac{1}{p(x)})$$

$$= -log(p(x))$$




**확률이 1인 정보의 정보량은 0!!!!**

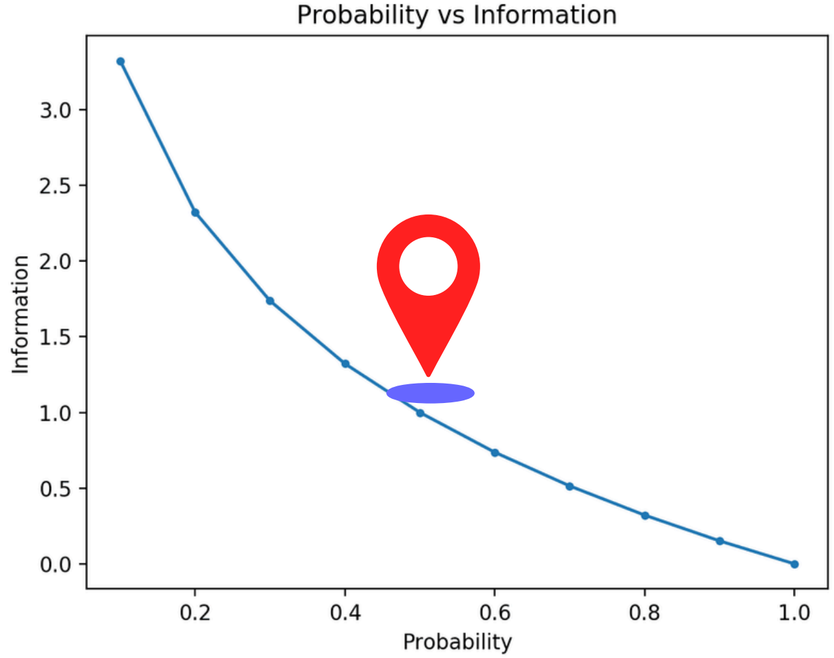
**확률이 0인 정보의 정보량은 무한대!!!!**

$$p(X = 1) = 0.5$$

$$p(X = 0) = 0.5$$

$$I(X) = log(\frac{1}{p(x)})$$

$$= -log(p(x))$$



엔트로피 평균 정보량 정보량 출현 확률

$$H(X) = E\big(I(X)\big) = \sum_{x_i} I(X = x_i)P(X = x_i)$$

$$= -\sum_{x_i} log\big(P(x_i)\big)P(x_i)$$

**동전을 던졌을때 기대 정보량은?**

$$P(H) = 0.5, \quad P(T) = 0.5$$

$$H(X) = -[P(H)\log_2 P(H) + P(T)\log_2 P(T)]$$

$$H(X) = -[0.5\log_2 0.5 + 0.5\log_2 0.5]$$

$$H(X) = -[0.5 \times (-1) + 0.5 \times (-1)]$$

$$H(X) = -(-0.5 - 0.5)$$

$$H(X) = 1 \text{ bit}$$

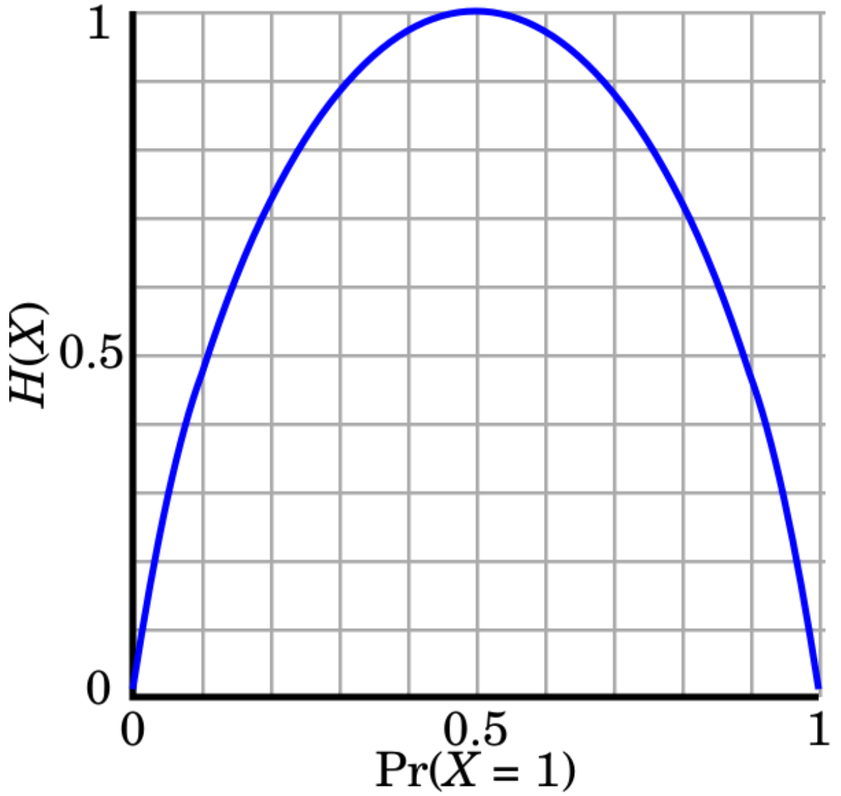
로그의 밑이 2일경우 bit by

동전 하나로 표현되는 정보

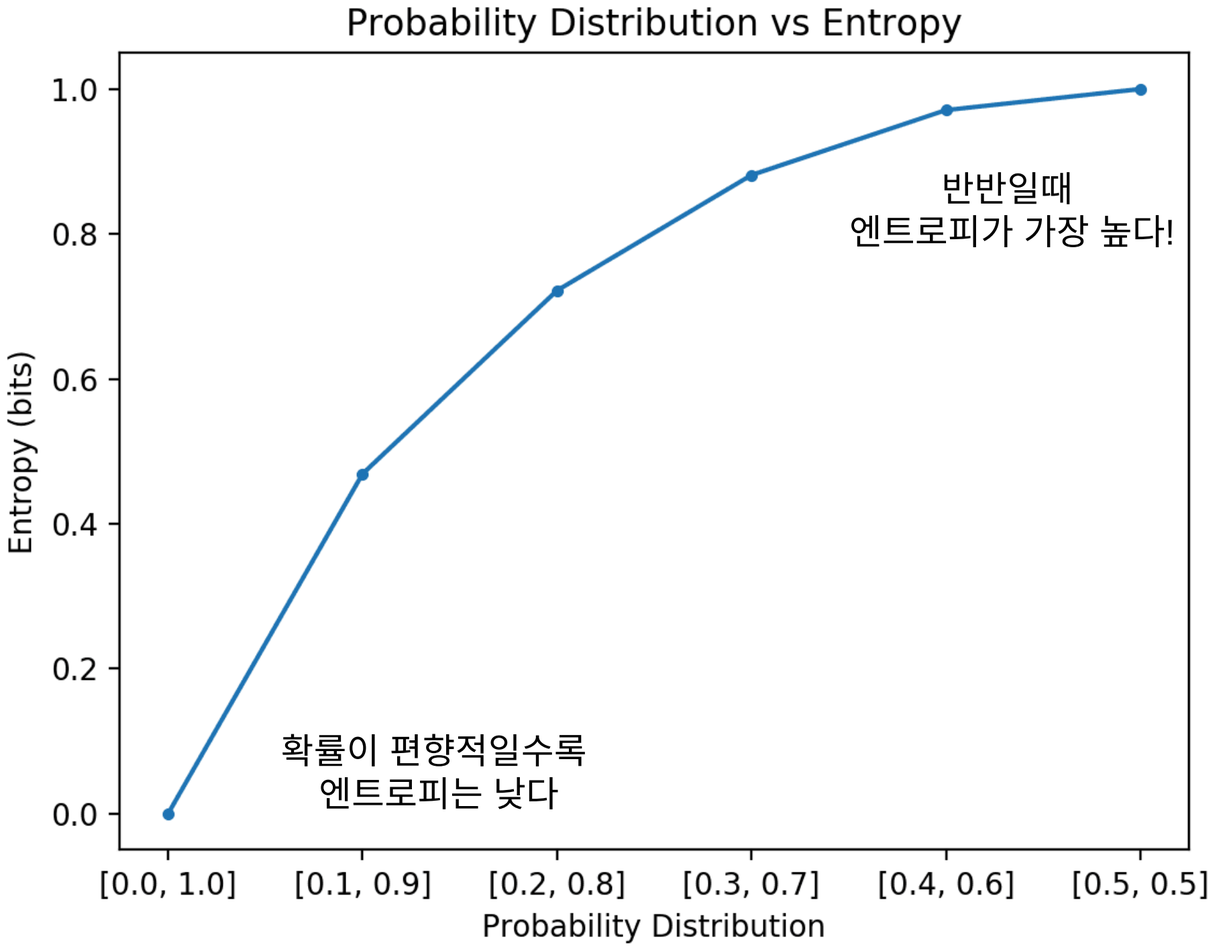
# 1bit = 확률이 1/2일때의 정보량

어떤 사건이 발생할 확률이 50%일 때, 해당 사건을 구별하는 데 필요한 최소한의 정보량

베루누이 분포 (이항분포) 에서 엔트로피




반반 동전이면?! 1bit!



$$P(1) = P(2) = P(3) = P(4) = P(5) = P(6) = \frac{1}{6}$$

$$H(X) = -\sum_{i=1}^{6} \frac{1}{6} \log_2 \frac{1}{6}$$

$$H(X) = -6 \times \frac{1}{6} \log_2 \frac{1}{6}$$

$$H(X) = -\log_2 \frac{1}{6}$$

$$H(X) = \log_2 6$$

$$H(X) \approx 2.585 \text{ bits}$$

편향적인 3가지 사건이 있을때!

$$P(A) = 0.7, \quad P(B) = 0.2, \quad P(C) = 0.1$$

$$H(X) = -(0.7 \log_2 0.7 + 0.2 \log_2 0.2 + 0.1 \log_2 0.1)$$

1.157비트!

$$H(X) \approx 1.157 \text{ bits}$$

---

사건별 확률분포가 고른 경우
vs
확률 분포가 편향적인 경우

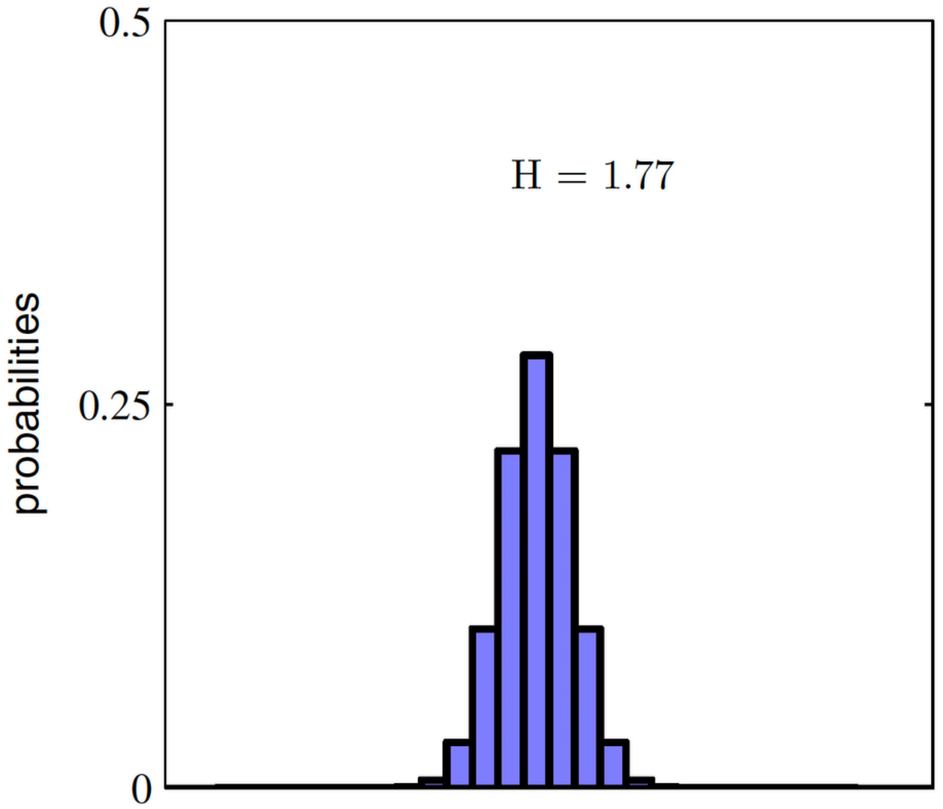
더 편향적일 경우!

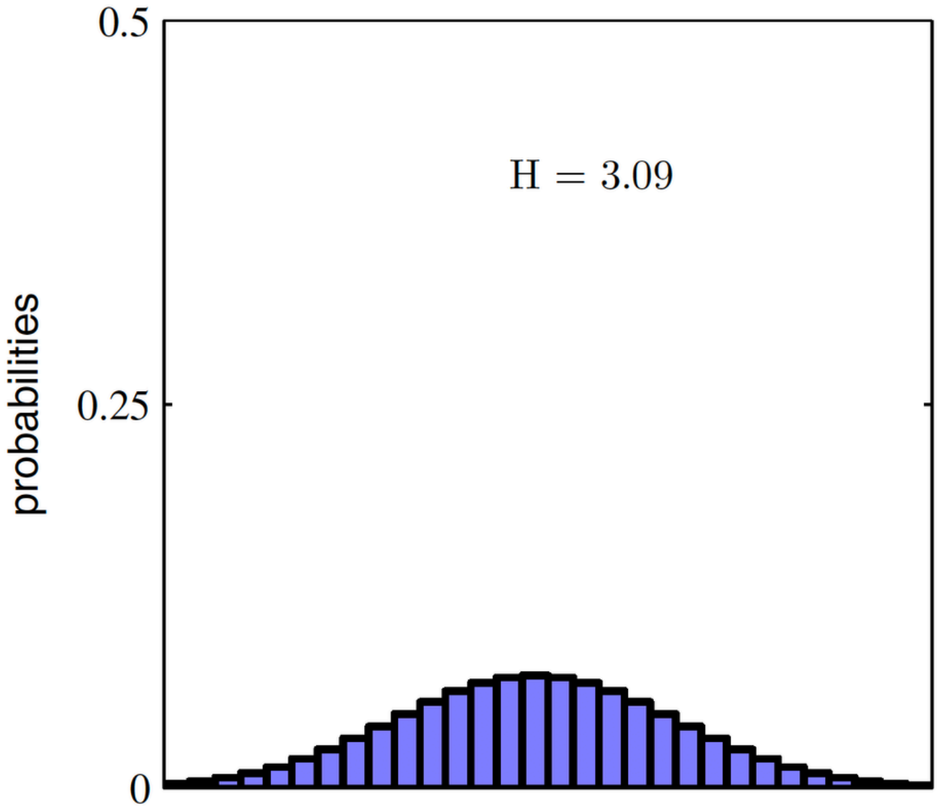
$$P(A) = 0.9, \quad P(B) = 0.05, \quad P(C) = 0.05$$

$$H(X) = -(0.9 \log_2 0.9 + 0.05 \log_2 0.05 + 0.05 \log_2 0.05)$$

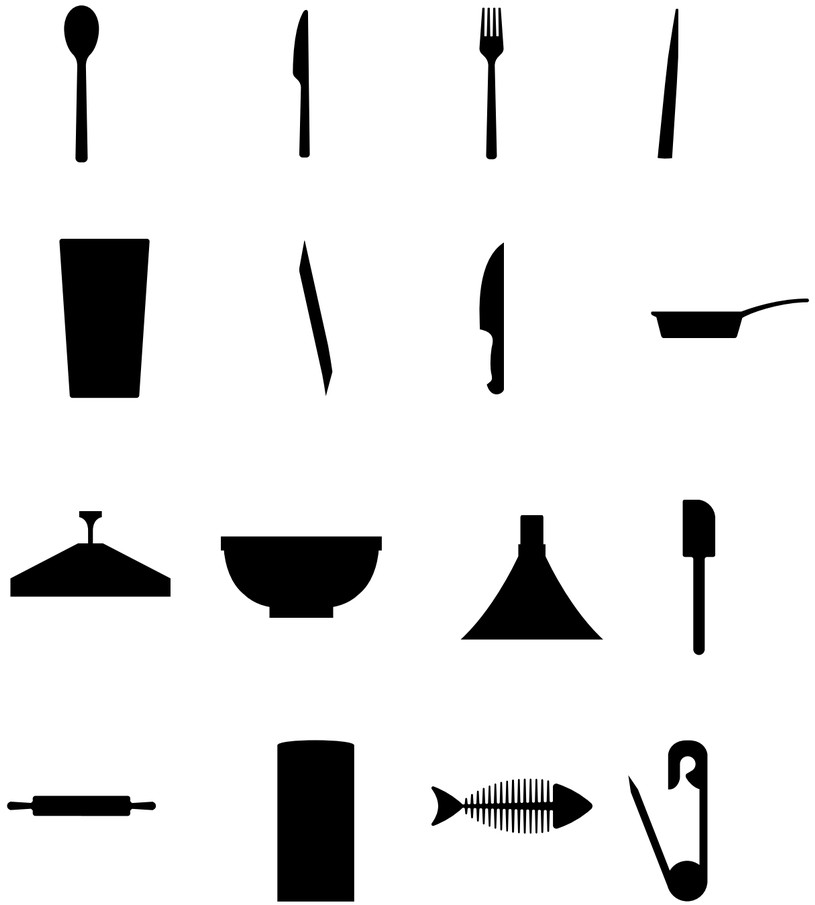
0.569 비트!

$$H(X) \approx 0.569 \text{ bits}$$

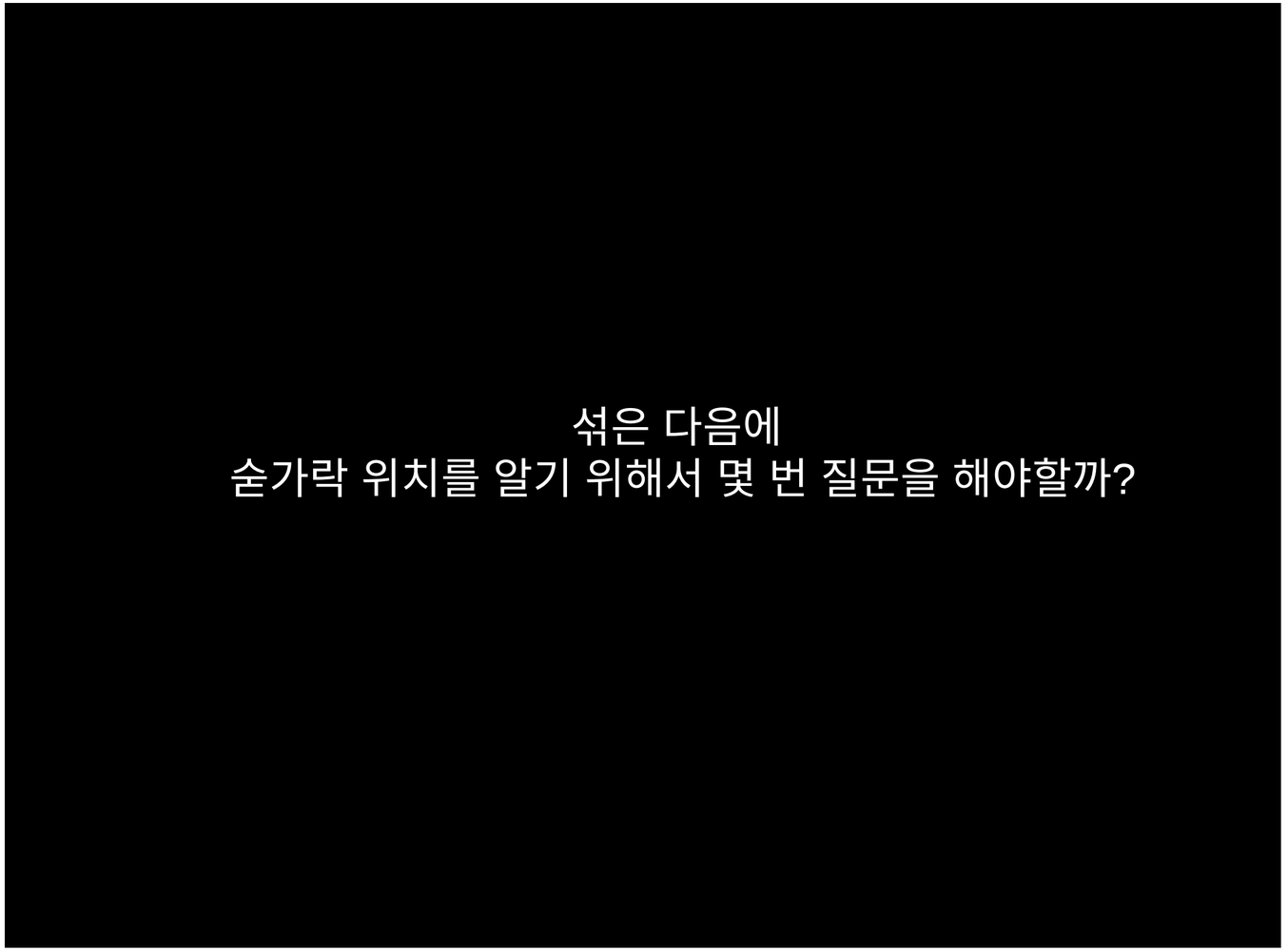

0.5
0.25
0
probabilities
H = 1.77


0.5
0.25
0
probabilities
H = 3.09

# 정보의 표현



# 정보의 표현



(1,1)?
(1,2)?
(1,3)?
...
(16,16)?

# 16번 물어보기?

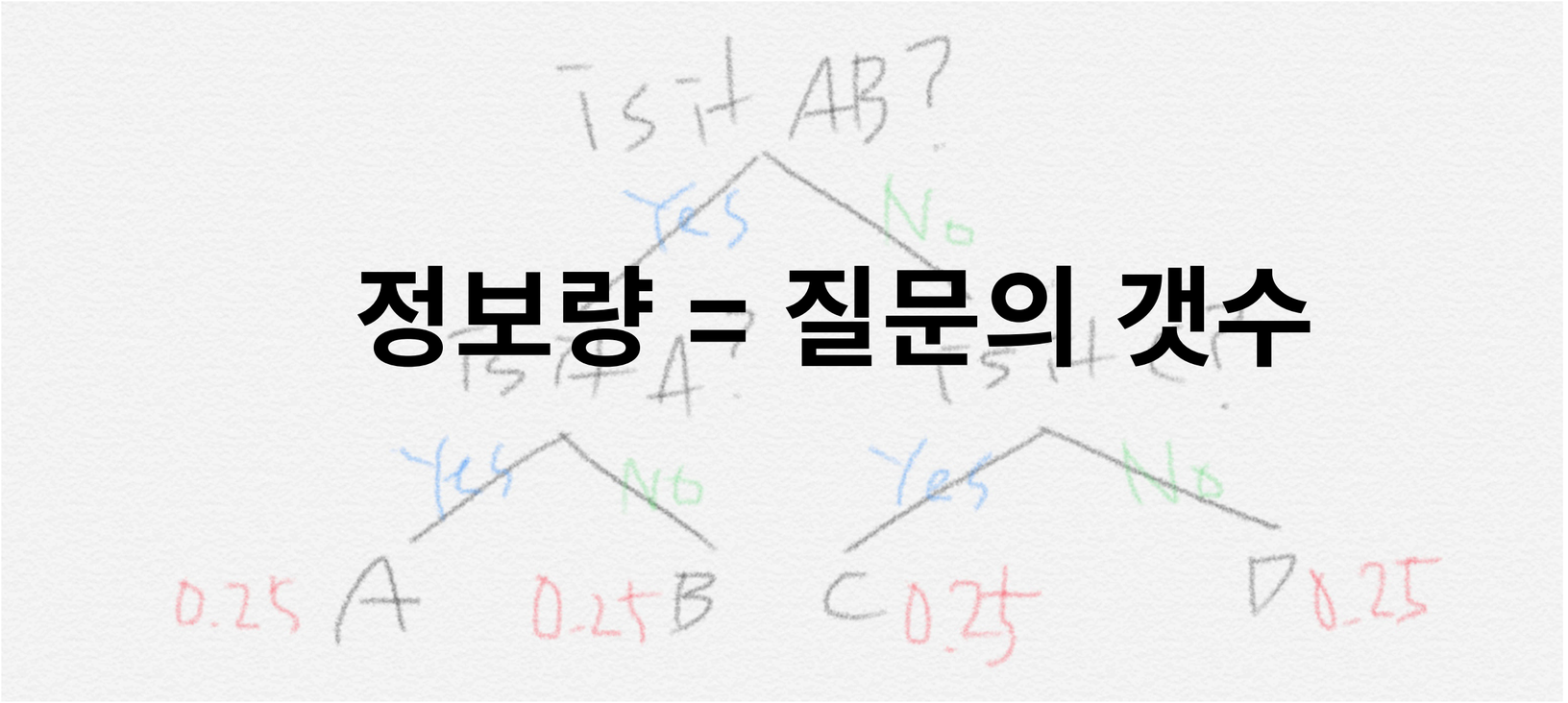
# 더 적게 물어보는 방법은?

$$\text{질문개수} = \log_2(\text{가능한 결과의 수})$$

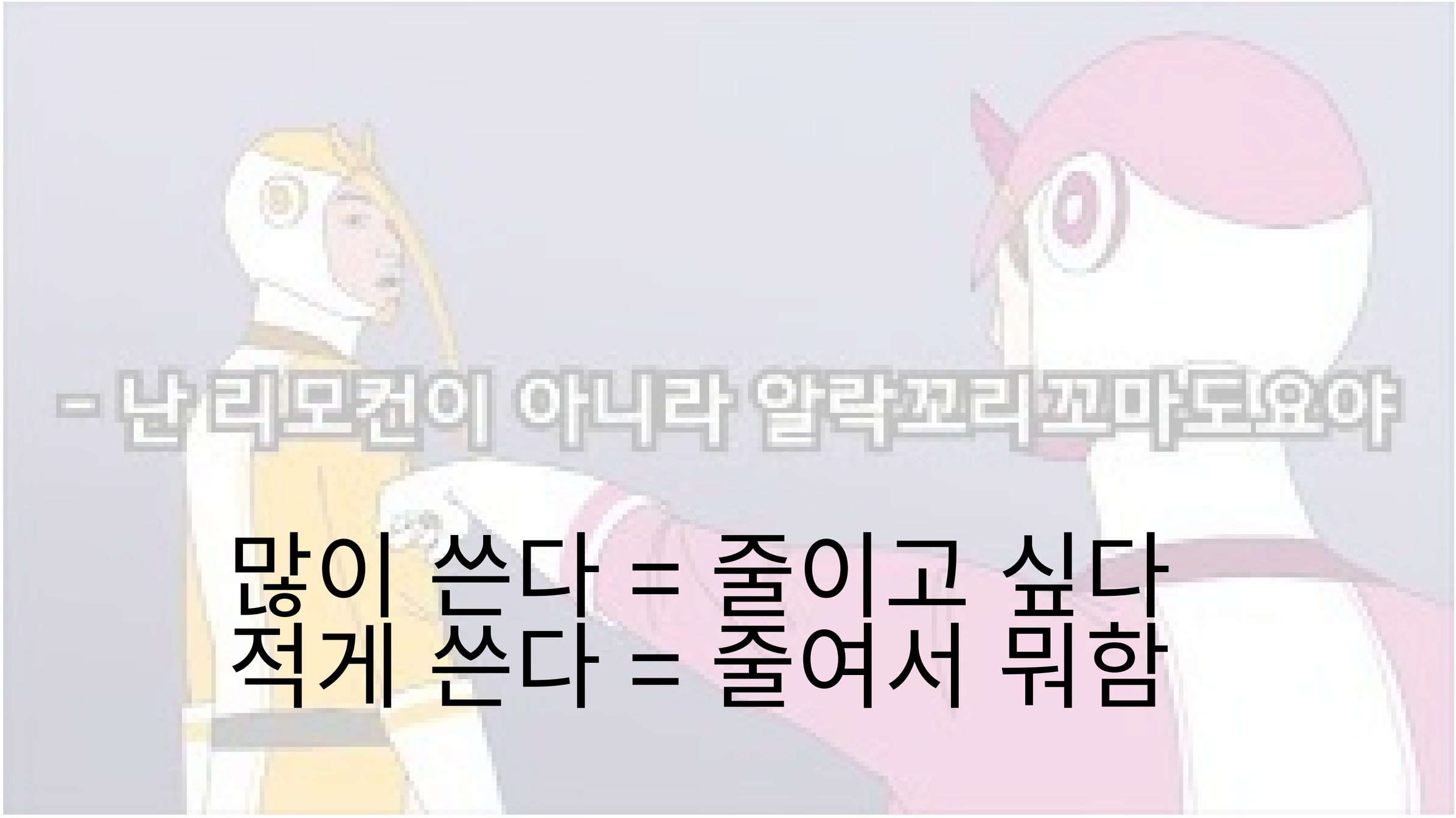
왼쪽? ㅇㅇ
아래쪽? ㄴㄴ
그중에서 왼쪽? ㅇㅇ
그중에서 위쪽? ㅇㅇ

**TFTT**

**1011**




**정보량 = 질문의 갯수**




많이 쓴다 = 줄이고 싶다
적게 쓴다 = 줄여서 뭐함

# ABC가 나오는 상태일때

P(A) = 0.33
P(B) = 0.33
P(C) = 0.33

평균 엔트로피: 1.6 비트

**AABC**

최소 2비트.

**00 00 01 10**

P(A) = 0.9 정보량 0.15
P(B) = 0.05 정보량 4.32
P(C) = 0.05 정보량 4.32

평균 엔트로피: 0.56 비트

**AABC**

허프만 코딩

**0 0 10 11**

# 정보는 압축 될 수 있다

# 클로드 섀넌

저글러,
땜장이, 놀이꾼,
디지털 세상을
설계하다
세상을 바꾼
괴짜 천재의
궁극의 놀이본능
지미 소니 · 로브 굿맨 지음
양병찬 옮김
A Mind at Play
〈금요일 금요일 밤에, 알쓸신잡 3〉 과학박사 김상욱 추천
“무척 재미있는, 정보이론의
역사에 대한 흥미진진한 이야기!”
☆ 미국응용수학회에서 수여하는 ‘폰노이만상’ 수상
☆ 《블룸버그Bloomberg》와 《네이처Nature》 ‘올해의 책’ 선정
☆ 《모닝선The Morning Sun》 ‘2017년 최고의 책’ 선정
곰출판

Ultimate machine

룰렛 도움용 웨어러블 기기

실제로 라스베가스에서 판돈 100원씩- 4천원 벌었음.

리프트

무선 조정차

저글링 로봇

체스 로봇

동전앞뒤 맞추기 로봇

불 나오는 트럼펫

루빅큐브 맞추는 로봇

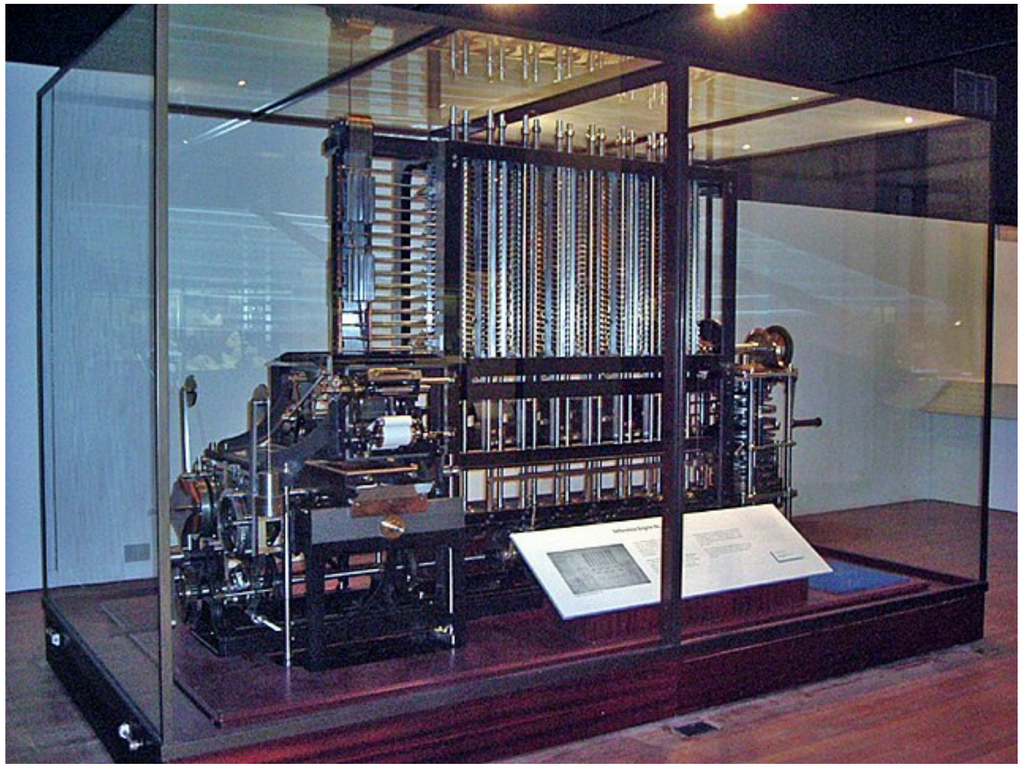
미로 탈출 인공지능 쥐

# 최초의 컴퓨터와 프로그래밍

별자리 관련 검수 하다가… 계산하기 싫어서 제안!
찰스 배비지

이거 있으면 이거 돌릴 수 있음!
에이다 러브레이스

# 찰스 배비지 차분기관



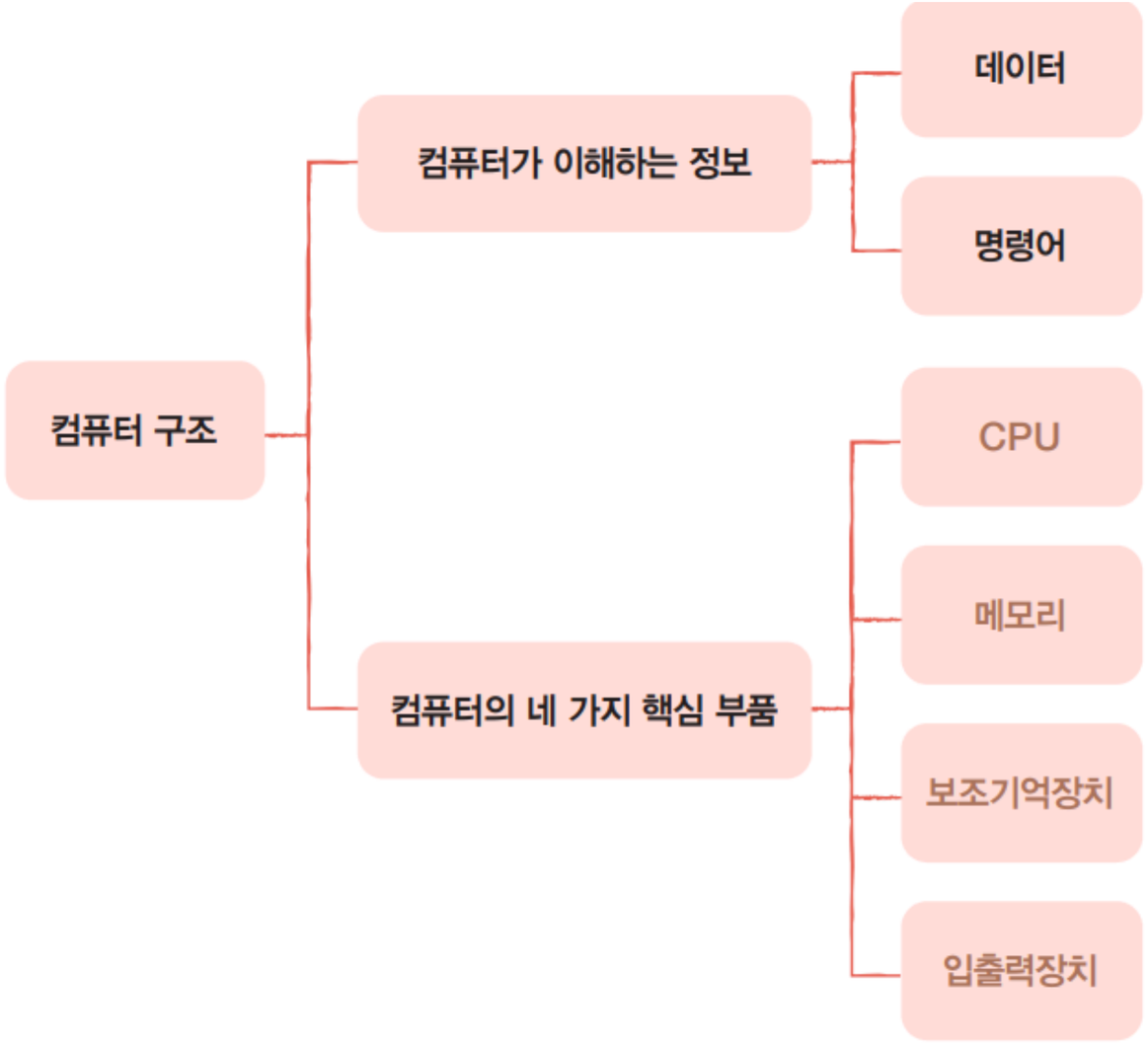
- 천공카드를 이용한 입출력 장치
- 데이터와 명령을 분리
- 스토어(store) : 기억장치
- 밀(mill) : 중앙 처리 장치
- 분기(branch), 서브루틴(subroutine), 루프(loop) 같은 프로그래밍할 수 있는 요소 사용 가능

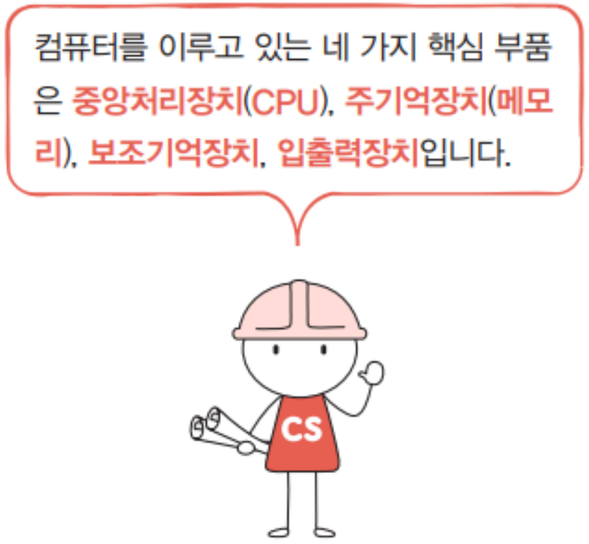
# 컴퓨터 아키텍쳐

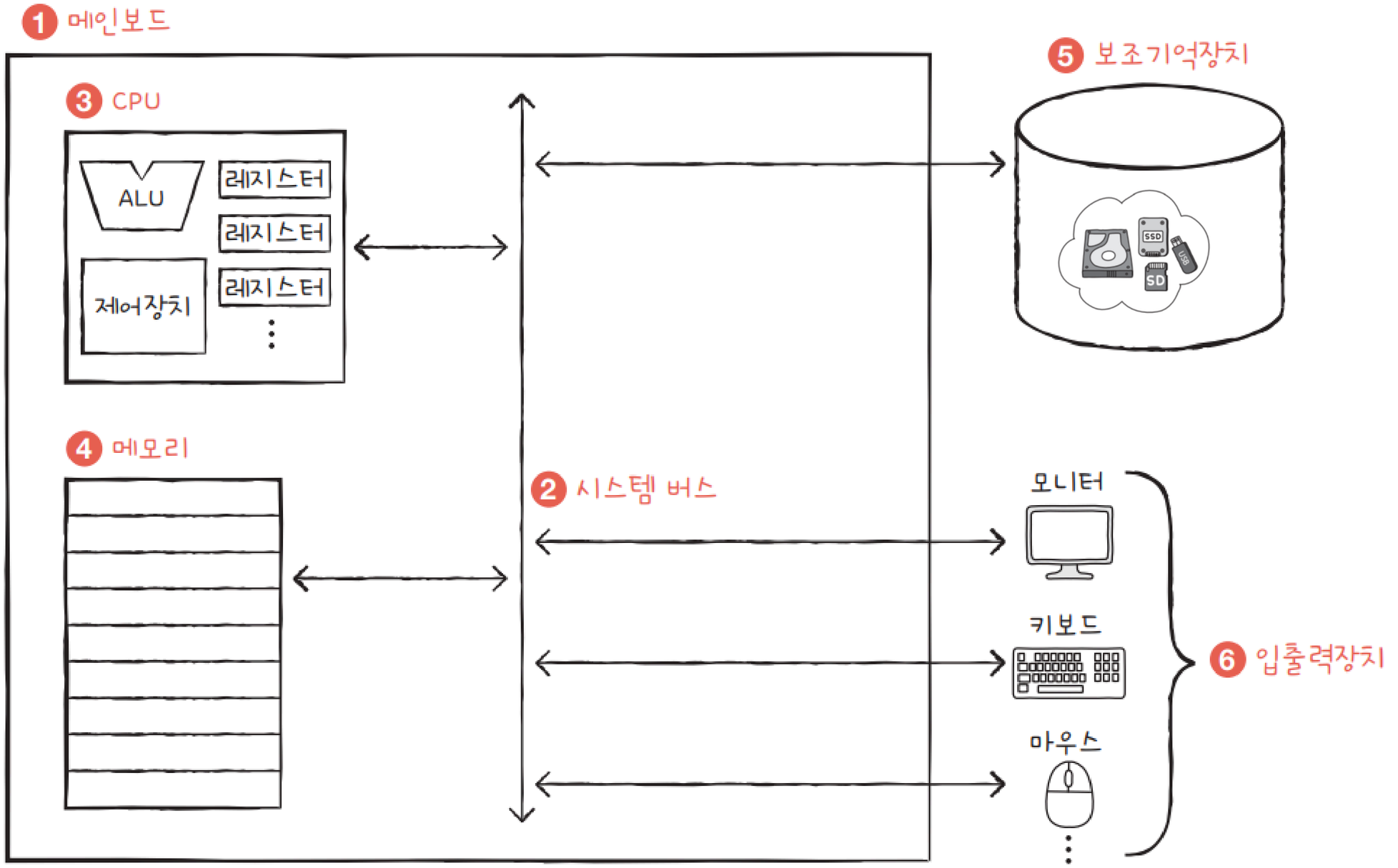
**[컴퓨터 구조] CPU, 메모리, 보조기억장치, 입출력장치를 알아보자**

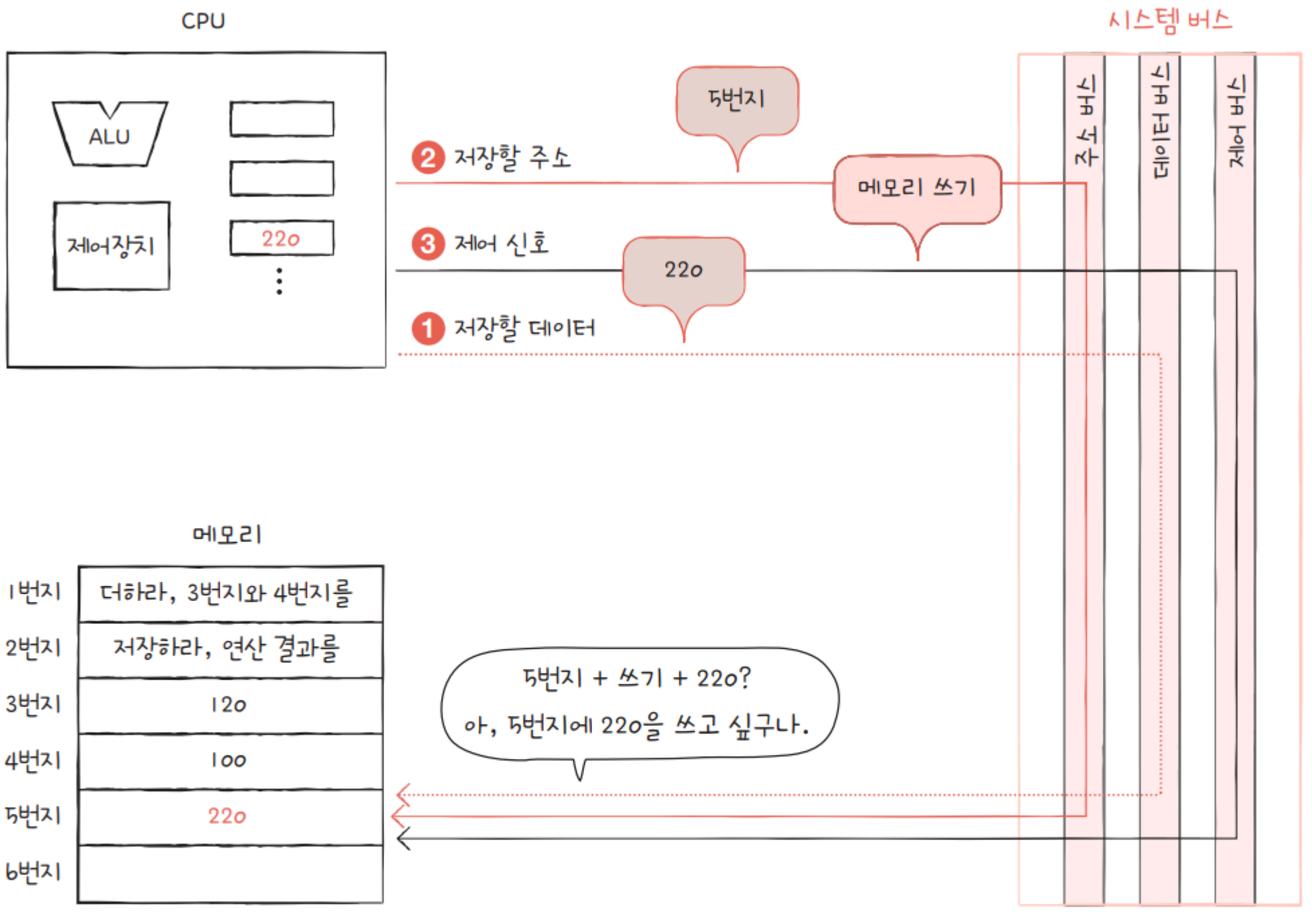
컴퓨터의 핵심 부품은 CPU, 메모리, 보조기억장치, 입출력장치입니다. 이 네 가지 부품의 역할만 이해하고 있어도 컴퓨터의 …

혼자 공부하는 책 / Mar 7, 2024




1 메인보드
3 CPU
ALU
레지스터
레지스터
레지스터
제어장치
5 보조기억장치
SSD
USB
SD
4 메모리
2 시스템 버스
모니터
키보드
마우스
6 입출력장치




컴퓨터의 네 가지 핵심 부품은 **메인보드**에 연결되어 **시스템 버스**를 통해 서로 정보 또는 데이터를 주고받습니다.

# 프로세서란?
## 모형 컴퓨터

- **GET** : 키보드에서 수를 입력 받은 후 누산기에 넣음
- **PRINT** : 누산기의 내용을 출력
- **STORE M** : 누산기 내용의 복사본을 메모리 위치 M에 저장
- **LOAD M** : 메모리 위치 M의 내용을 누산기에 적재
- **ADD M** : 메모리 위치 M의 내용을 누산기 내용에 더함
- **STOP** : 실행 중지

**첫 번째 모형 프로그램**

```
GET
PRINT
STOP
//키보드로 누산기에 입력 > 출력 > 정지
```

# 프로세서란?
## 모형 컴퓨터

- **GET** : 키보드에서 수를 입력 받은 후 누산기에 넣음
- **PRINT** : 누산기의 내용을 출력
- **STORE M** : 누산기 내용의 복사본을 메모리 위치 M에 저장
- **LOAD M** : 메모리 위치 M의 내용을 누산기에 적재
- **ADD M** : 메모리 위치 M의 내용을 누산기 내용에 더함
- **STOP** : 실행 중지

**두 번째 모형 프로그램**

```
GET
STORE FirstNum
GET
ADD FirstNum
PRINT
STOP
FirstNum:
```

# 프로세서란?
## 모형 컴퓨터

- **GET** : 키보드에서 수를 입력 받은 후 누산기에 넣음
- **PRINT** : 누산기의 내용을 출력
- **STORE M** : 누산기 내용의 복사본을 메모리 위치 M에 저장
- **LOAD M** : 메모리 위치 M의 내용을 누산기에 적재
- **ADD M** : 메모리 위치 M의 내용을 누산기 내용에 더함
- **STOP** : 실행 중지

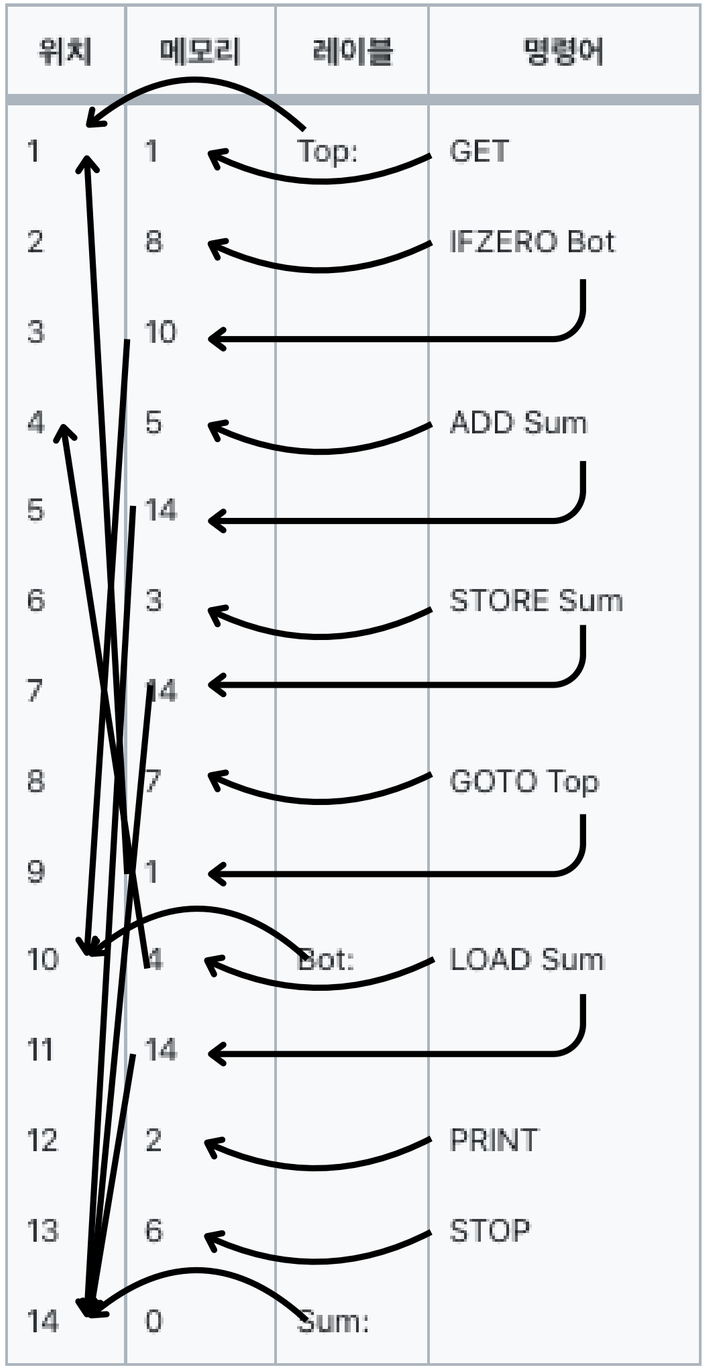
# 프로세서란?
## 모형 컴퓨터

```
Top:  GET           get a number
      IFZERO Bot    if it's zero, go to Bot
      ADD Sum       add running sum to the latest number
      STORE Sum     store result as new running sum
      GOTO Top      go back to Top to get another number
Bot:  LOAD Sum      load running sum into accumulator
      PRINT         and print it
      STOP
Sum:  0             memory location to hold running sum
                      (initialized to 0 when program starts)
```

**Figure 3.8:** Toy machine program to add up a sequence of numbers.

# 프로세서란?
## 모형 컴퓨터

1. 저장하는 명령어는 메모리를 1칸 사용한다.
2. 참조하는 명령어는 메모리를 2칸 사용한다.
3. 데이터는 메모리를 1칸 사용한다.
4. 각각의 명령어는 숫자형 코드를 가진다.
5. 참조하는 명령어의 빈칸은 참조하는 명령어의 위치를 가리킨다.



| 위치 | 메모리 | 레이블 | 명령어 |
|---|---|---|---|
| 1 | 1 | Top: | GET |
| 2 | 8 | | IFZERO Bot |
| 3 | 10 | | |
| 4 | 5 | | ADD Sum |
| 5 | 14 | | |
| 6 | 3 | | STORE Sum |
| 7 | 14 | | |
| 8 | 7 | | GOTO Top |
| 9 | 1 | | |
| 10 | 4 | Bot: | LOAD Sum |
| 11 | 14 | | |
| 12 | 2 | | PRINT |
| 13 | 6 | | STOP |
| 14 | 0 | Sum: | |

GET = 1
PRINT = 2
STORE = 3
LOAD = 4
ADD = 5
STOP = 6
GOTO = 7
IFZERO = 8

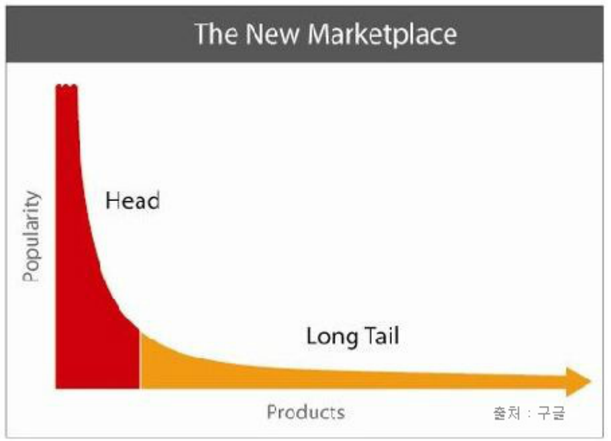
기능. 메모리

Top = 1
Bot = 10
Sum = 14

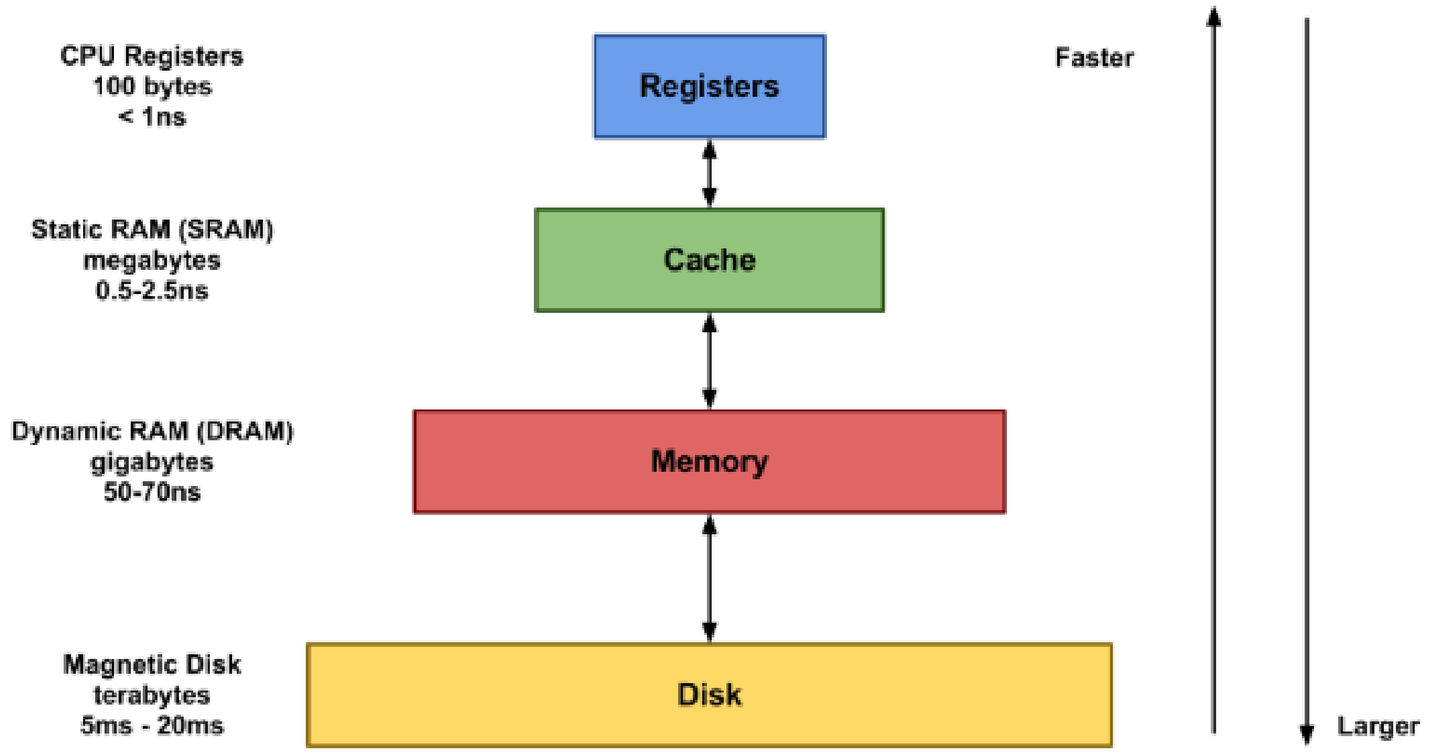
저장. 위치

실제 명령어와 메모리 위치를 참조하여
반복적인 연산을 실시하는 것

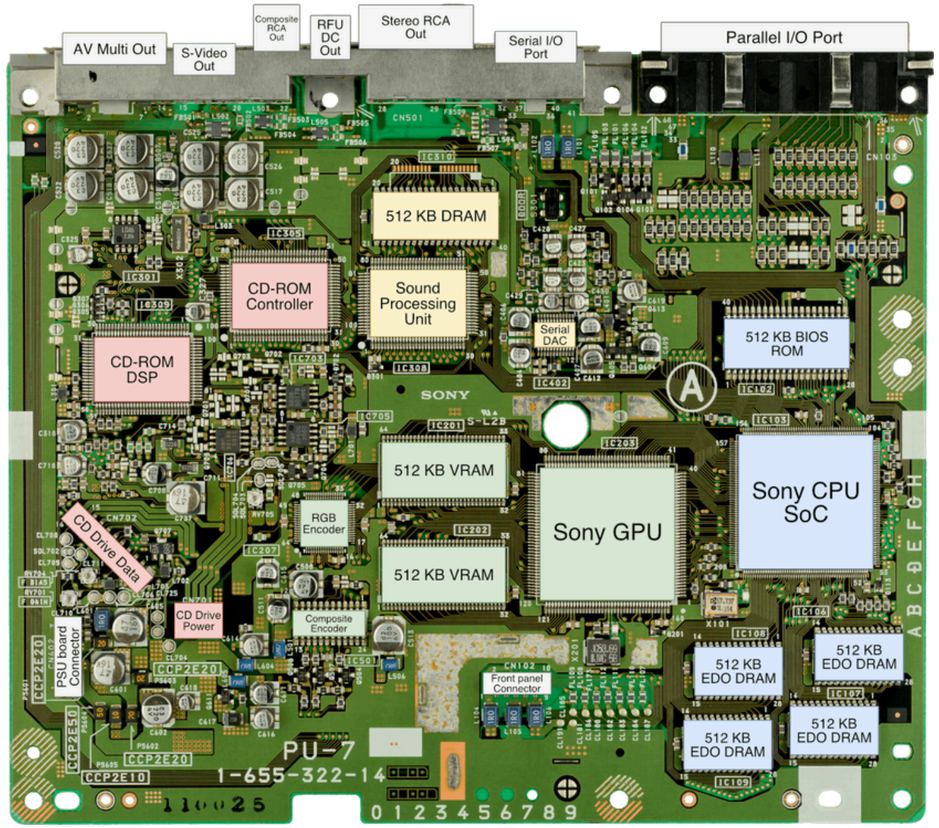
# 캐시란?
## 자주쓰는걸 가까이 두면 안되나?




- Long tail의 법칙:
- 20%의 요구가 시스템 리소스의 대부분을 사용한다는 법칙

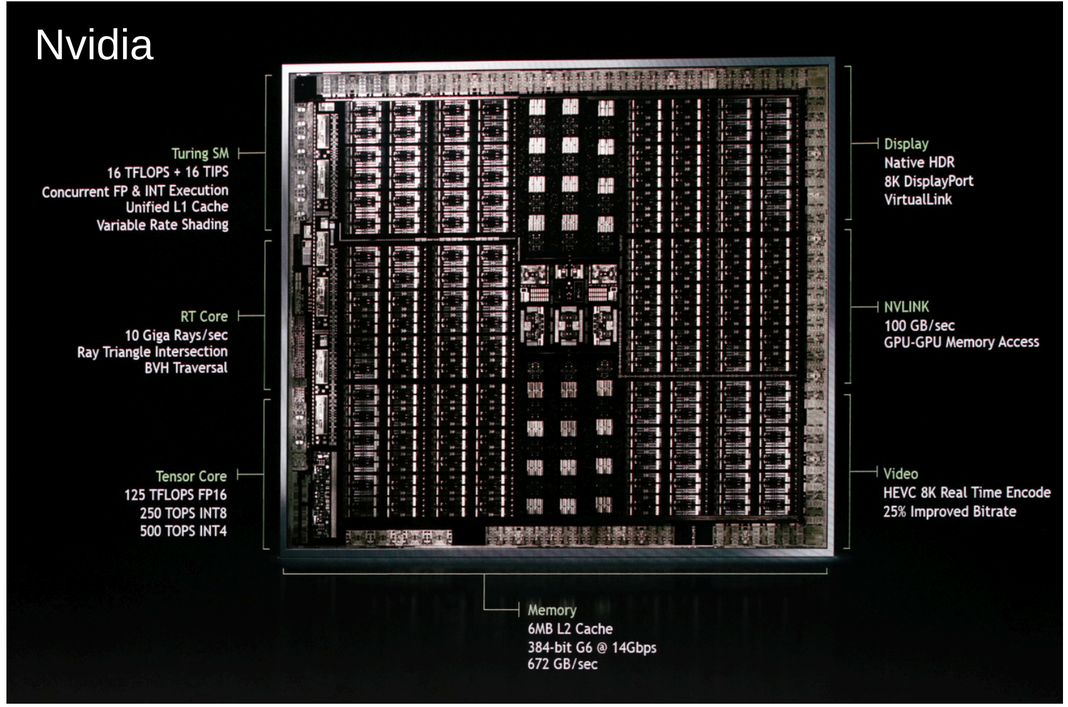


# Architecture

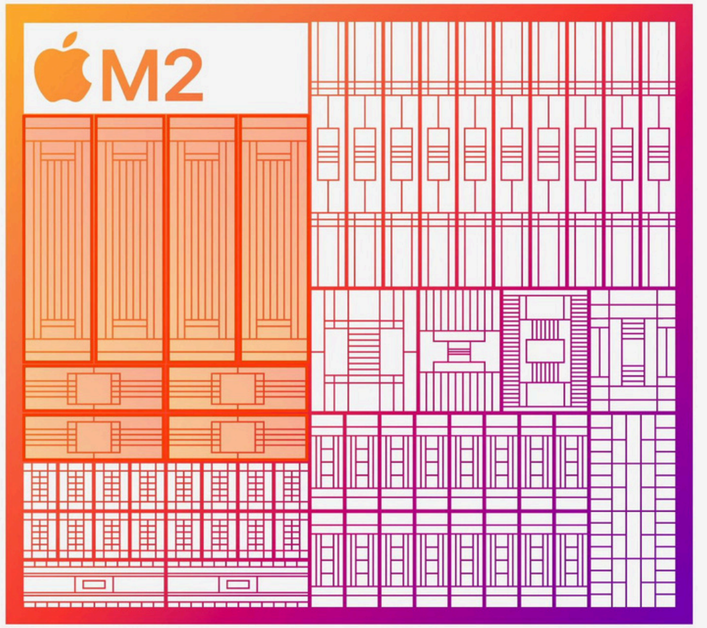



play station

Nvidia




Apple silicon M2




**4 high-performance cores**

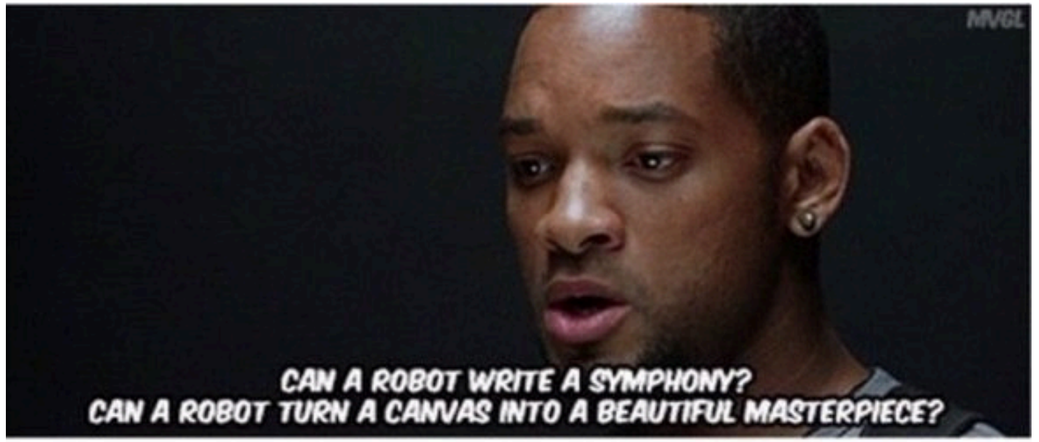
Ultrawide microarchitecture
192KB instruction cache
128KB data cache
Shared 16MB cache

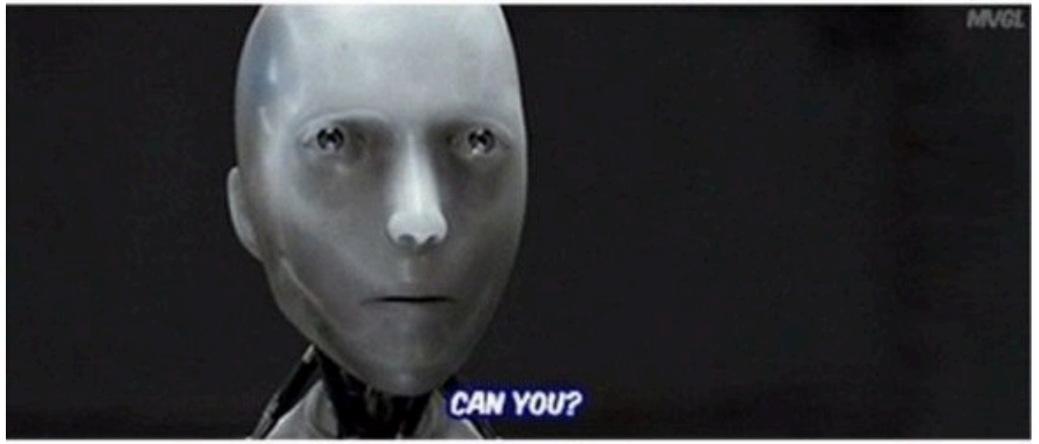
**4 high-efficiency cores**

Wide microarchitecture
128KB instruction cache
64KB data cache
Shared 4MB cache

# 기능과 성능이 뛰어나면?

- 세탁기: 세탁을 잘함
- 차: 빠름
- 냉장고: 차가움
- TV: 능력이 좋음
- 스마트폰: 다재다능함
- 우리는?
- 언젠가는 생각하지 않을까?



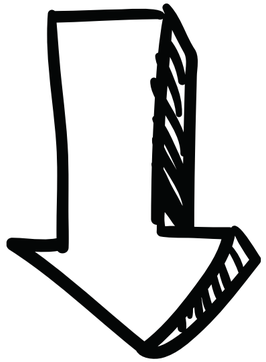



# 1928년

“모든 수학적 문제에 대해서 참인지 거짓인지를 확실하게 결정해줄 수 있는 명확한 방법이 존재하는 가를 증명하시오”

**다비트 힐베르트**

원칙적으로 수학의 모든 문제를 순서대로 해결할 수 있는 일반적인 기계적 절차가 있는가?
풀수 있는 알고리즘을 만드는 알고리즘이 있나?
모든 수학을 풀 수 있는 알고리즘이 존재하는가?



수학의 추론과정에서 기계적 패턴을 찾으면?
자동으로 수학의 모든 사실을 풀 수 있을 거야!

# 1931년

**불완전성 정리**

**“기계적 방식만으론 수학의 모든 사실을 만들어 낼 수 없다”**

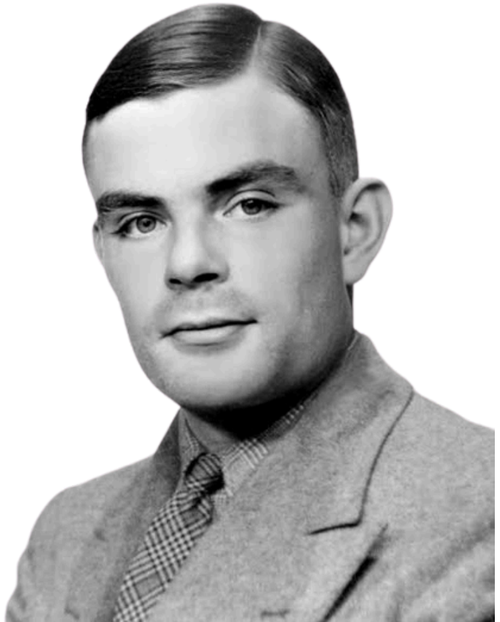
**아무리 추론규칙들을 잘 만들어도, 기계적인 방식만으론 참인지 거짓인지 판단할 수 없는 명제가 항상 존재한다는 증명**

**쿠르트 괴델**

# 1936년

<계산 가능한 수와 결정문제의 응용에 관하여(On Computable Numbers, with an Application to the Entscheidungsproblem)>

튜링기계라는 가상의 기계를 구현하여 기계적인 방식을 적용해 괴델의 정리를 증명
(불완전성 정리의 증명)

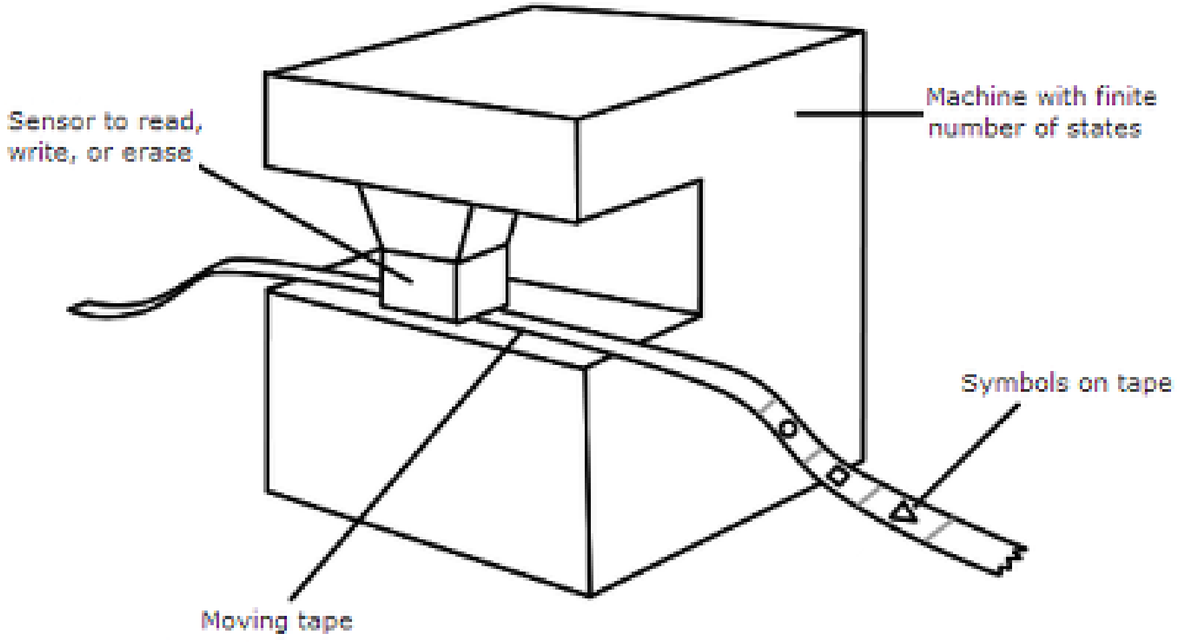


‘보편만능의 기계(Universal Machine)’

**앨런 튜링**

# 튜링 머신?

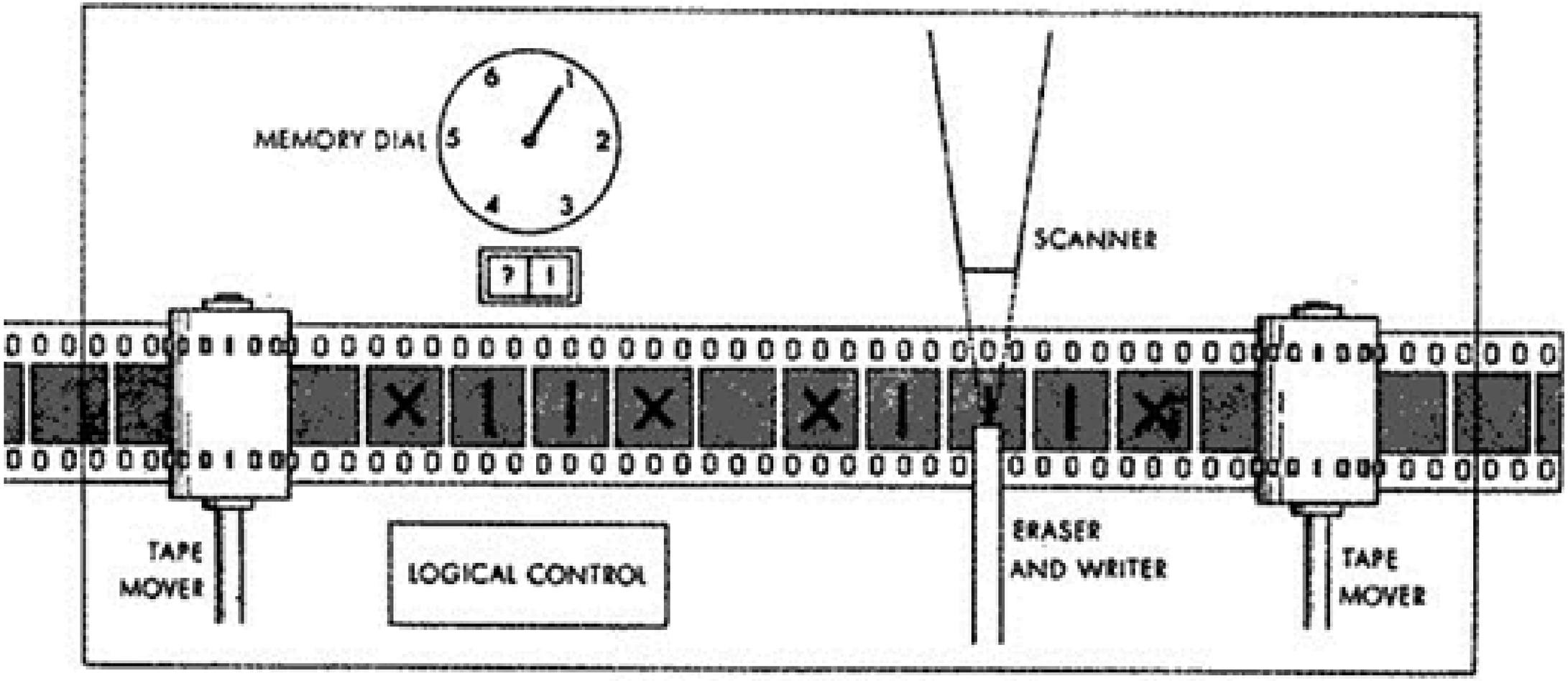
**finite number of states!!!!!!**




A Turing Machine

https://norman3.github.io/papers/docs/neural_turing_machine.html

# 튜링 머신?



# 계산 기계
## 튜링머신

무한한 테이프
Infinite Tape
1 0 0 0 1 1 1 0
Read / Write Head
Control Unit
State: Y
헤드: 움직이며 조작
Action: 좌우로 움직인다
State를 가지고 있다
현재의 State를 기준으로 Symbol에 따라 action이 정해짐

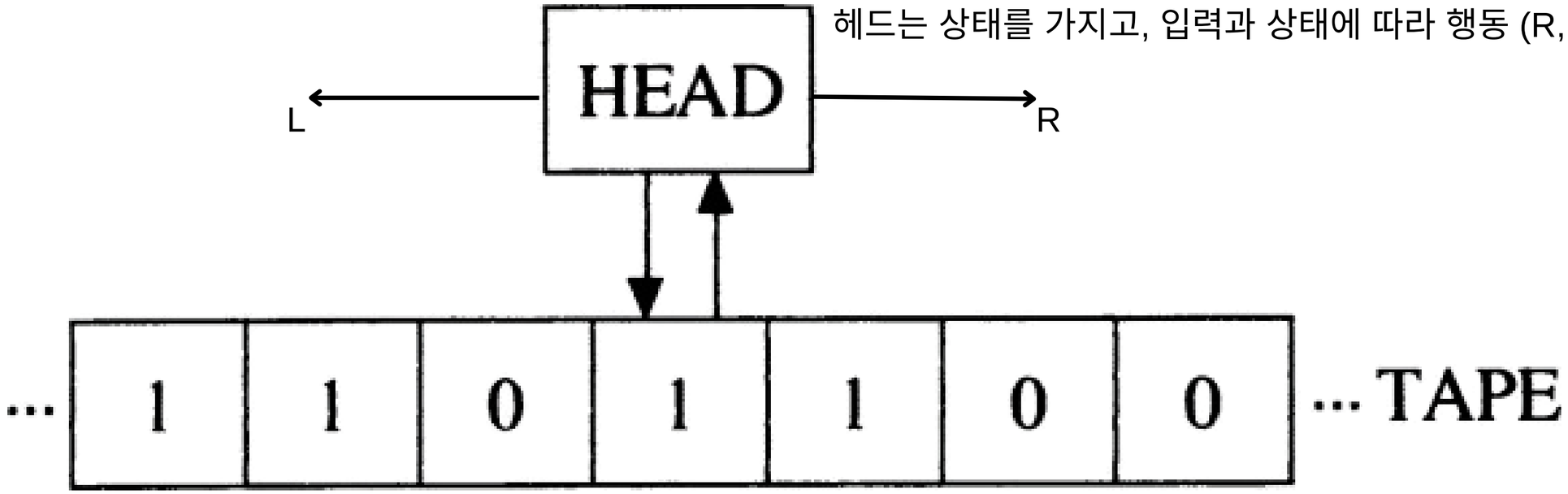
$_{A}0 \rightarrow {}_{A}0_{R}$ 상태 A 에서 0을 만나면, 상태A를 유지하고, 입력을 0으로 그대로 두고, R로 움직임

$_{A}1 \rightarrow {}_{B}1_{R}$ 상태 A 에서 1을 만나면, 상태 B로 바꾸고, 입력을 1로 그대로 두고, R로 움직임

$_{B}0 \rightarrow {}_{A}1_{Stop}$ 상태 B 에서 0을 만나면, 상태 A로 바꾸고, 입력을 1로 바꾸고, Stop임

$_{B}1 \rightarrow {}_{B}1_{R}$ 상태 B 에서 1을 만나면, 상태 B로 유지, 입력을 1로 유지, R로 움직임

헤드는 상태를 가지고, 입력과 상태에 따라 행동 (R, L, Stop)




무한한 테이프에는 값이 적혀 있음

주어진 (State,Symbol) → (New state, New Symbol) + Move

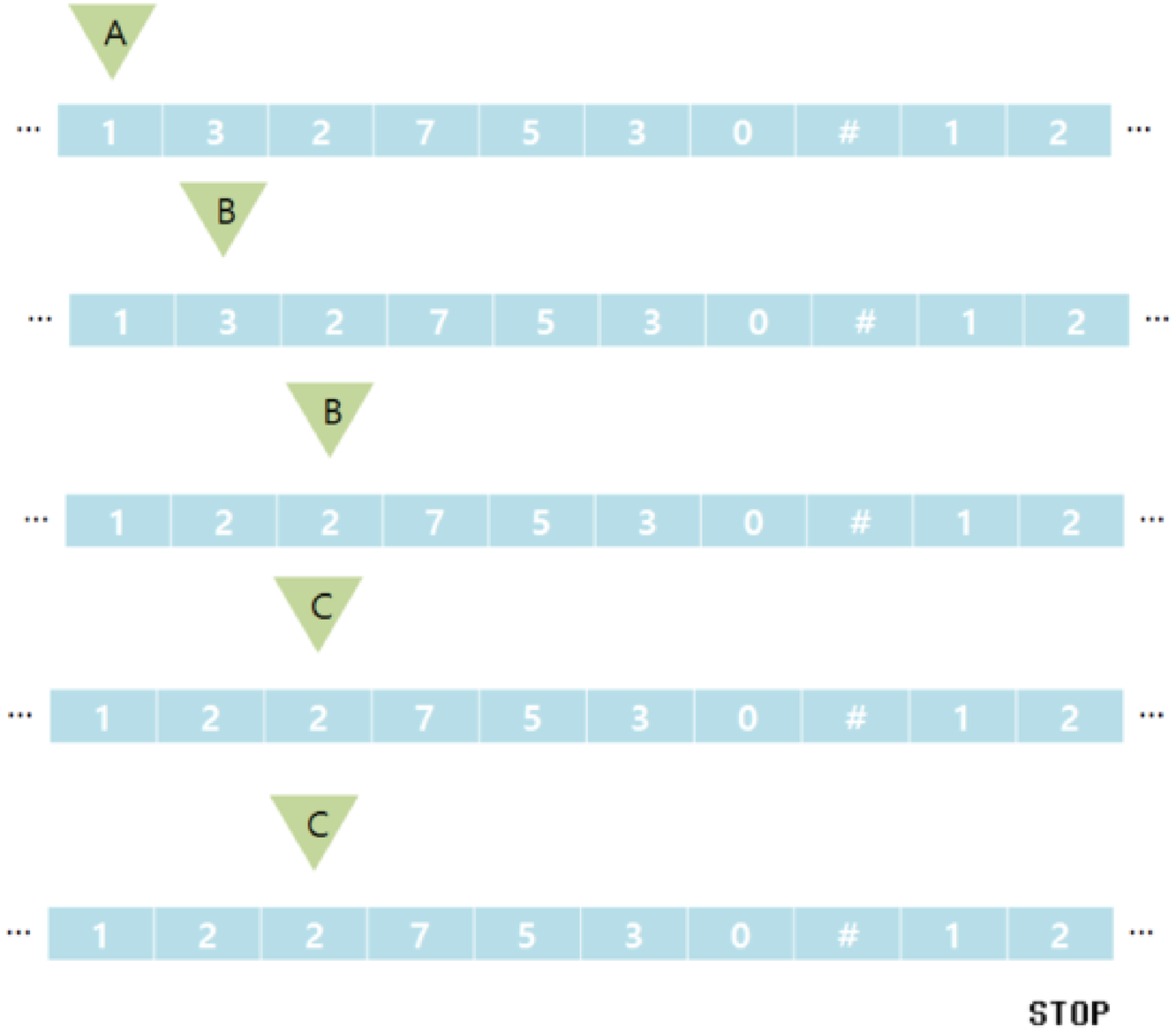
| State | Symbol | New State | New Symbol | Move |
|---|---|---|---|---|
| 1 | 0 | 1 | 1 | R |
| 1 | 1 | 1 | 0 | R |
| 1 | b | 2 | b | R |

# 입력 = 1100 (초기 State=1)

State 1 Symbol 1 → State 1 Symbol 0 Move R : 0**1**00
State 1 Symbol 1 → State 1 Symbol 0 Move R: 0**0**0
State 1 Symbol 0 → State 1 Symbol 1 Move R: 00**1**
State 1 Symbol 0 → State 1 Symbol 1 Move R: 001**?**
**종료!**

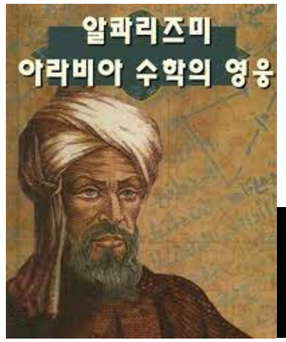
그러므로 1100 → 0011b (HALT)

헤드의 상태가 'A'고 적혀있는 숫자가 '1'이면 오른쪽으로 움직이고 'B'상태가 되어라
헤드의 상태가 'B'고 적혀있는 숫자가 '3'이면 숫자를 2으로 바꾸고 오른쪽으로 움직이고 'B'상태가 되어라
헤드의 상태가 'B'고 적혀있는 숫자가 '2이면 가만히 있고 'C'상태가 되어라
헤드의 상태가 'C'이면 종료하라



현존하는 모든 컴퓨터와 동등한 계산 능력을 지닌.

다만 조금 느린..




특수한 목적을 가진 명령의 집합
= 알고리즘

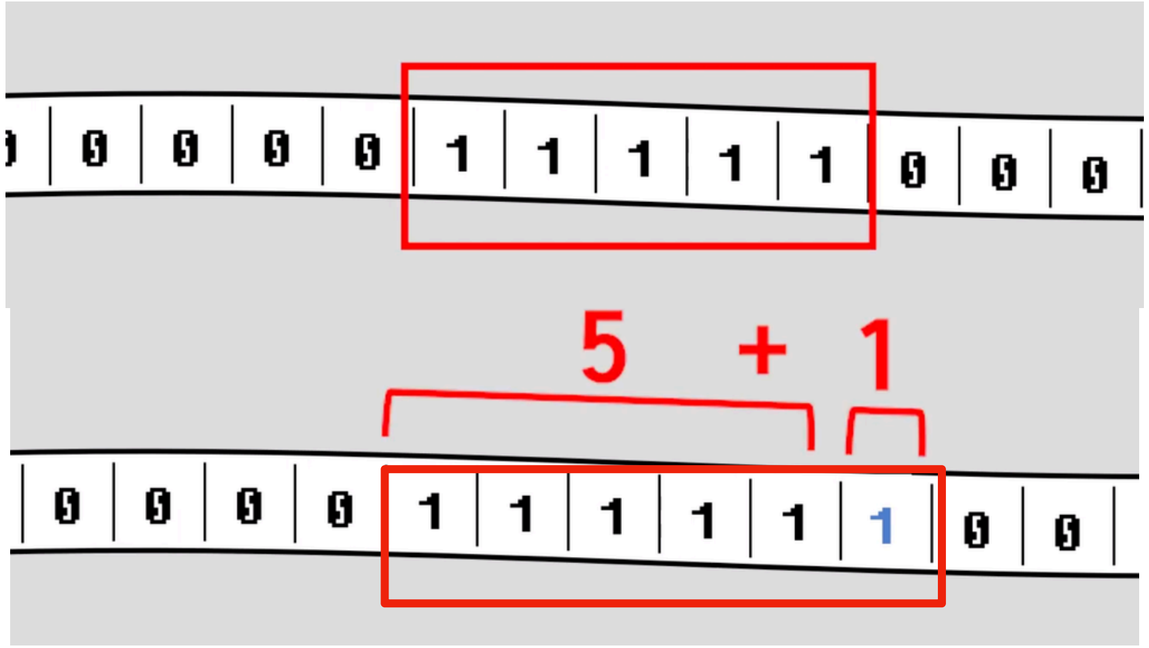
$_A0 \rightarrow {}_A0_R$

$_A1 \rightarrow {}_B1_R$

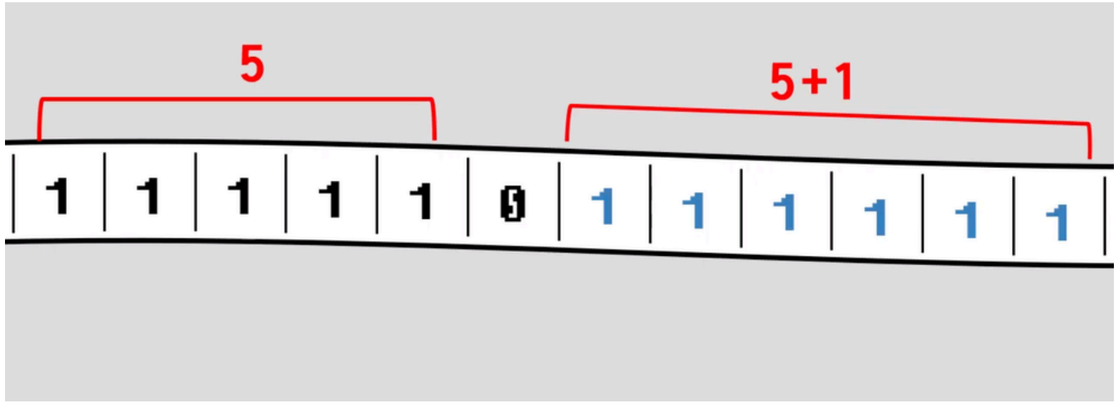
$_B0 \rightarrow {}_A1_{Stop}$

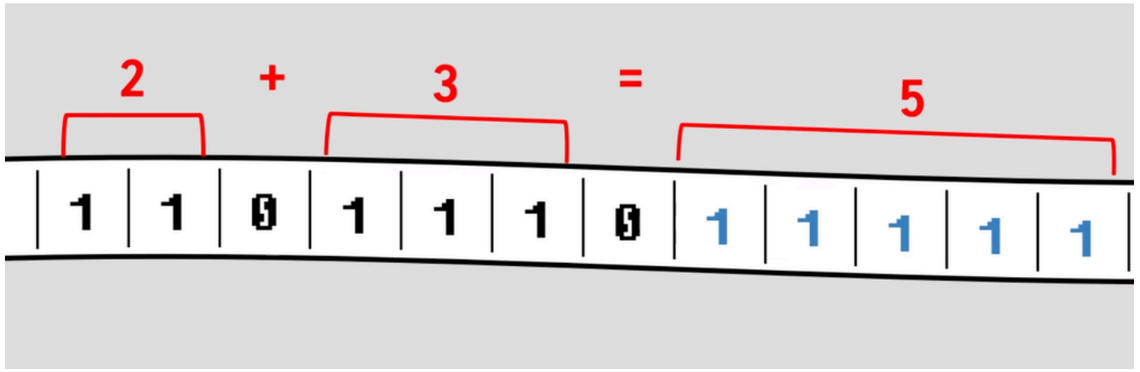
$_B1 \rightarrow {}_B1_R$

무한한 테잎에 1이 연속으로 5개!

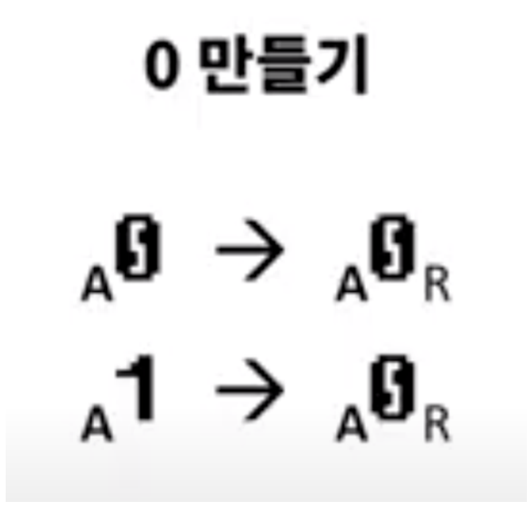


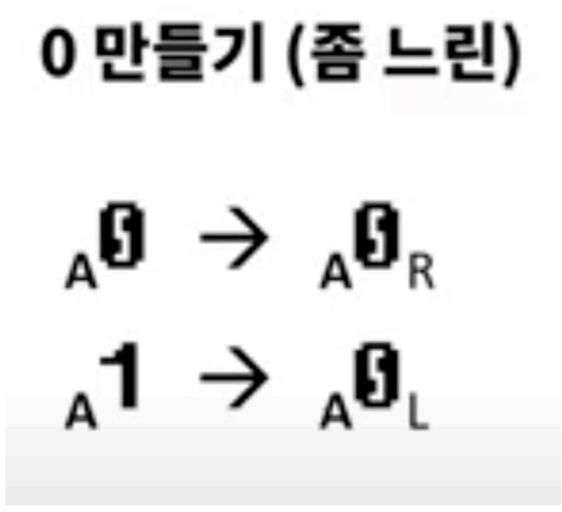

제일 뒤에 1을 하나 더넣어주는 기계



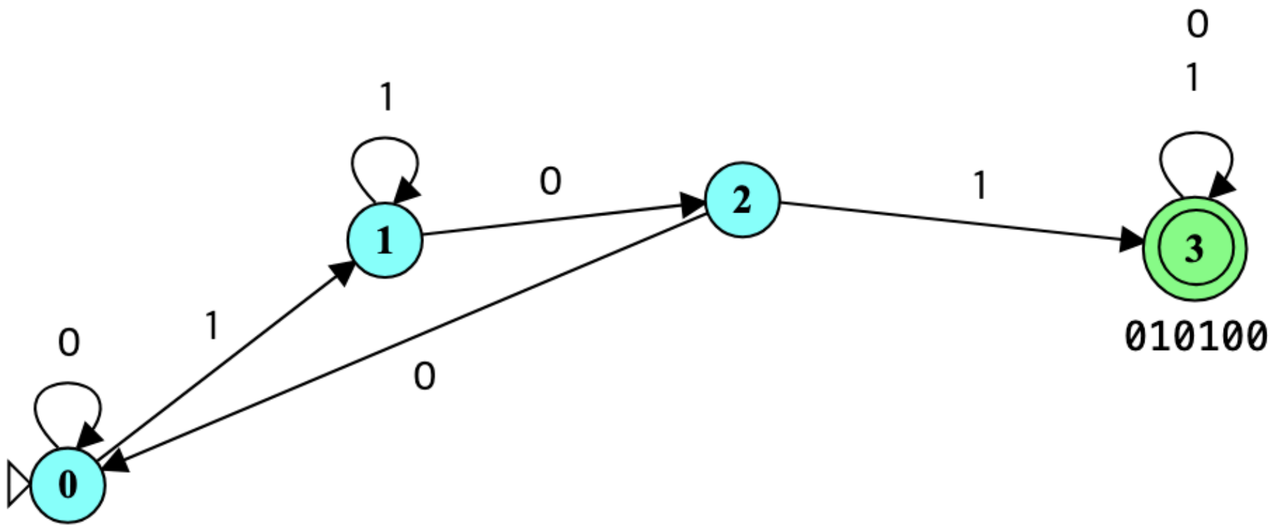

조금씩 확장하면...?


0 만들기
A 0 → A 0 R
A 1 → A 0 R


0 만들기 (좀 느린)
A 0 → A 0 R
A 1 → A 0 L

아무 것도 안함
A 0 → A 0 R
A 1 → A 1 R

https://www.cs.odu.edu/~zeil/cs390/latest/Public/turing-jflap/index.html




New Save Help

**Finite Automaton**

+State +Transition

**Testing**

Input text: 010100 Accepted

*010100|*

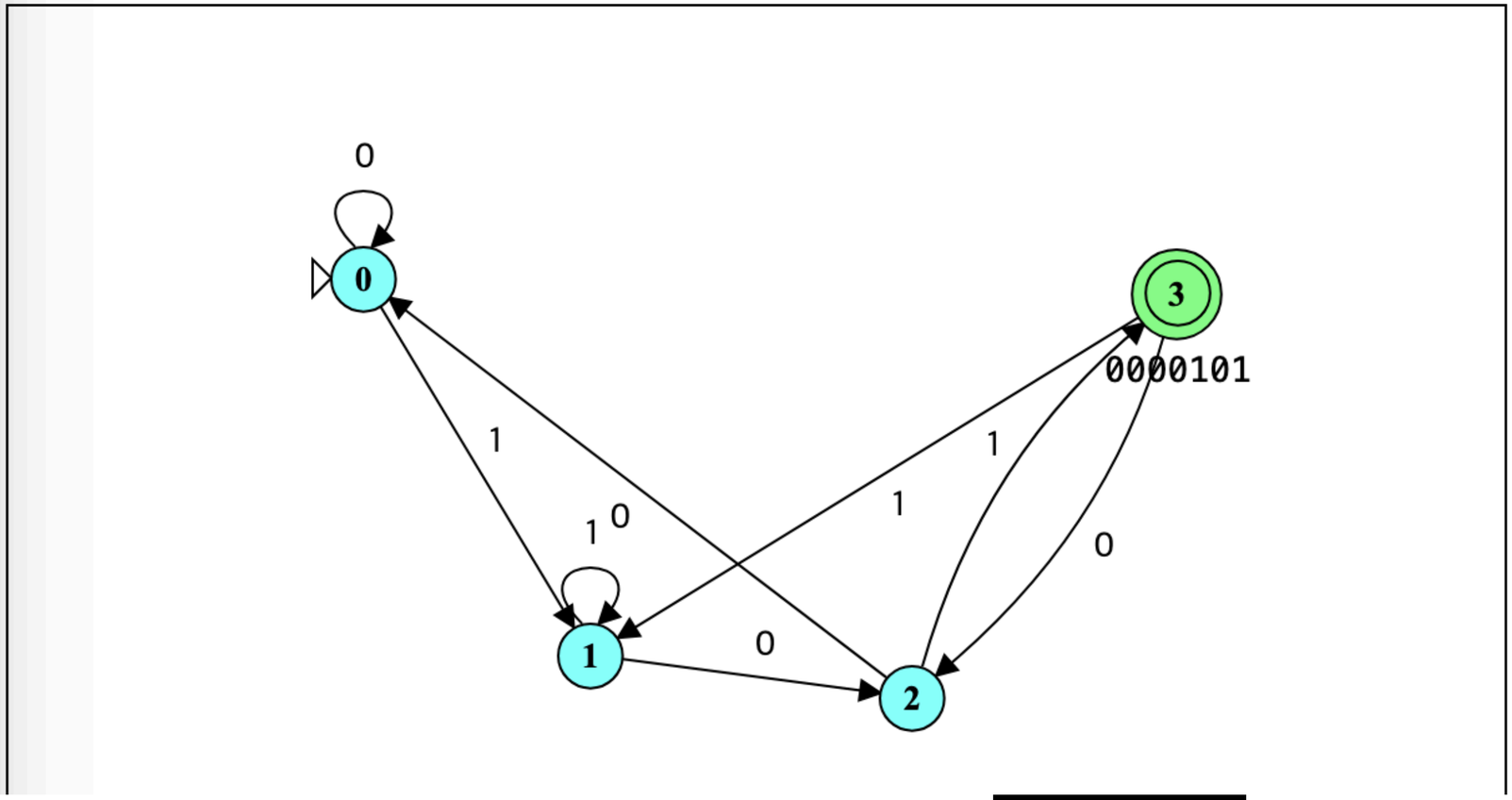
Start Step Finish

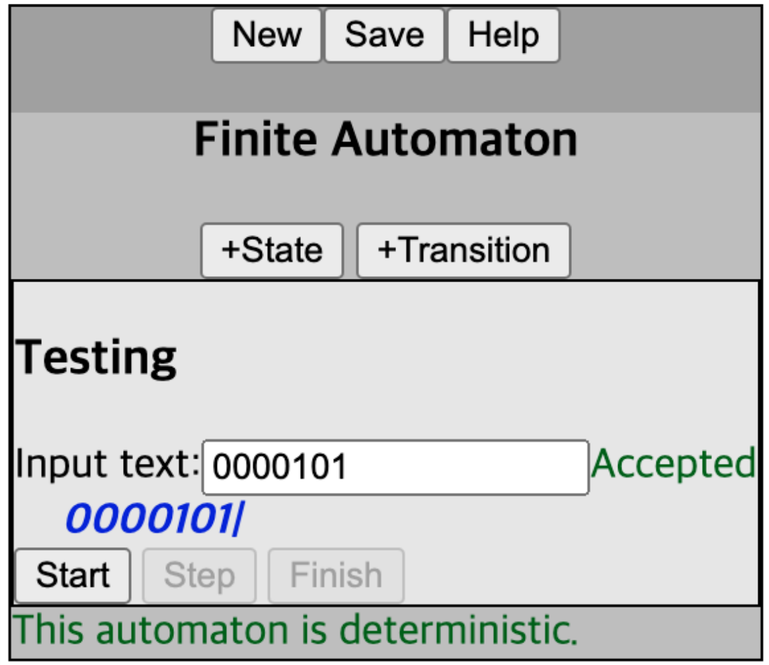
This automaton is deterministic.

Contains 101

https://www.cs.odu.edu/~zeil/automat/automat.cgi?lang=eyJzcGVjaWZpY2F0aW9uIjoiYXV0b21hdG9uRkEiLCJjcmVhdGVkQnkiOiJBbm9ueW1vdXMiLCJwcm9ibGVtSUQiOiIiLCJ1bmxvY2vGDHN0YXRlcyI6W3sibGFiZWwiOiIwIiwiaW5pdGlhbCI6dHJ1ZSwiZmluxA1mYWxzZSwieCI6ODYsInkiOjI2NH0syjoxzDrHLdE7MTnGPDE3Ocw8Mt88xTwzMjDGPDY1zDwz2jzmAMB4Ijo0NjXHdzd9XSwidHJhbnNp5AFG5gD9ZnJvbecA%2FHRv5wDLyF8xIsRtxyLEGcUi5ACxyCIwzCLEGcUi5ACX1UTJIsRv1UTJZvkAiOkAqtlExH%2FxAIgwXG4xIn1dfQ%3D%3D

https://www.cs.odu.edu/~zeil/cs390/latest/Public/turing-jflap/index.html



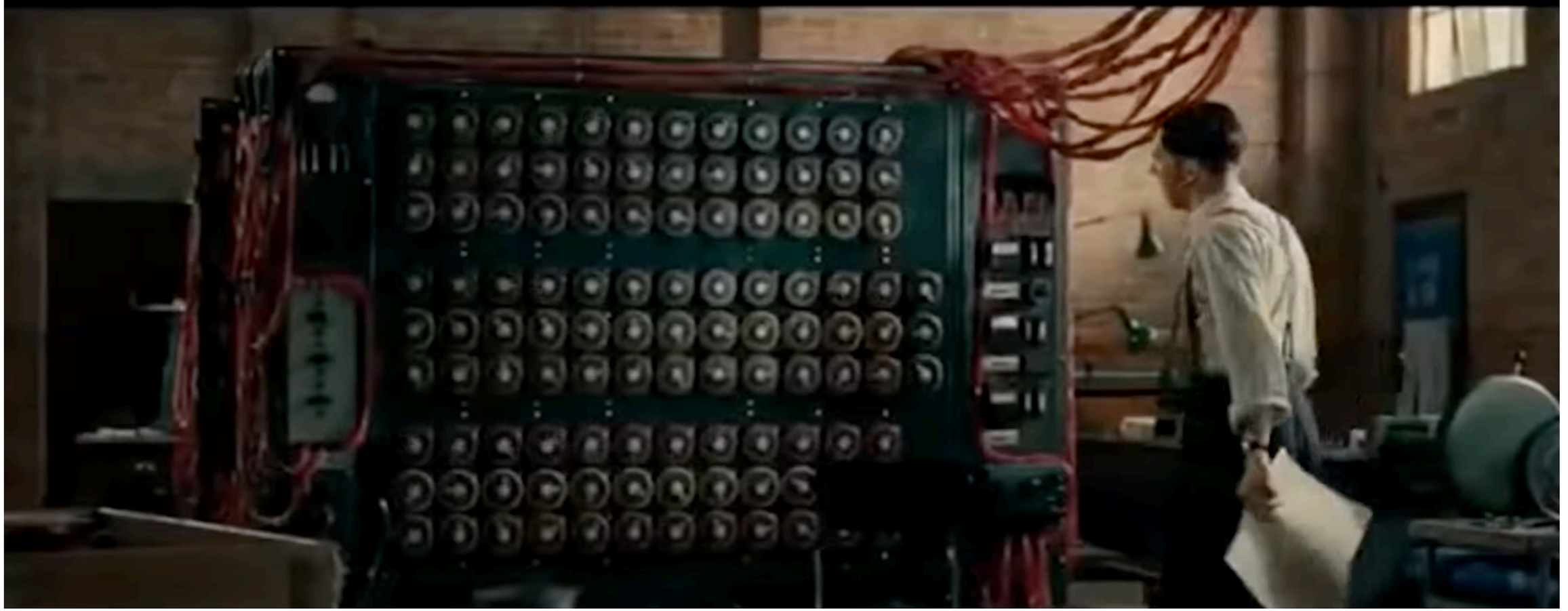




End with 101

https://www.cs.odu.edu/~zeil/automat/automat.cgi?
ang=eyJzcGVjaWZpY2F0aW9uIjoiYXV0b21hdG9uRkEiLCJjcmVhdGVkQnkiOiJBbm9ueW1vdXMiLCJwcm9ibGVtSUQiOiIiLCJ1bmxvY2vGDHN0YXRlcyI6W3sibGFiZWwiOiIwIiwia
W5pdGlhbCI6dHJ1ZSwiZmluxA1mYWxzZSwieCI6MTI5LCJ5Ijo5Nn0syjoxzDrHLdE7MjIwxTsyNDfMPDLfPMU8MzTGdzI2NMw8M9o85gDAeCI6NDPGOzk3fV0sInRyYW5zaeQBRe
YA%2FGZyb23nAPt0b%2BcAysheMSLEbMcixBnFIuQAsMgiMMwixBnFIuQAltVExBneRMki%2BQCIyWbkALPVROkAqtlE6QDuz0RdfQ%3D%3D

문제점이 있다...
덧셈기계는 덧셈만할줄알고
곱셈은 못해..

영화 ‘이미테이션 게임’

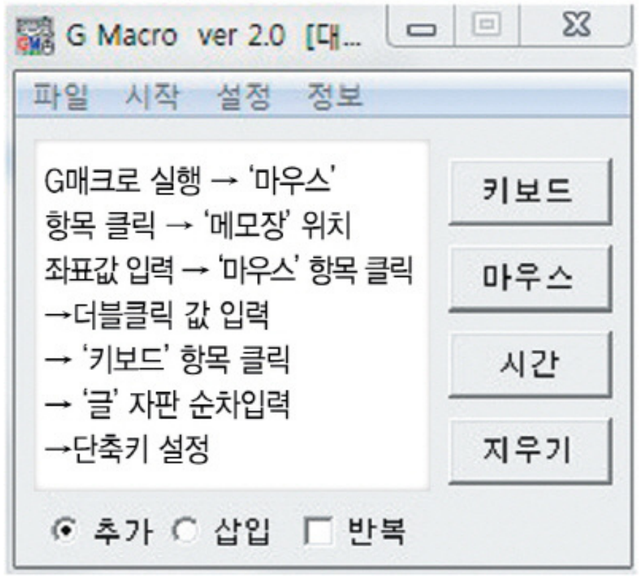


- 암호를 풀어라 = 암호푸는 기계를 만들자 = 만들었다 = 콜로서스(1943)

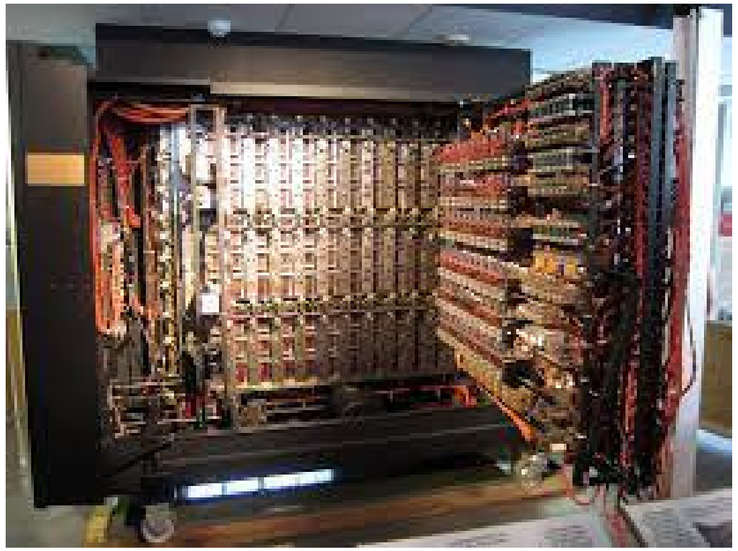
# 콜로서스는 완전한 튜링 머신은 아님

## 암호 풀기만 할 줄 안다. = 범용성 X

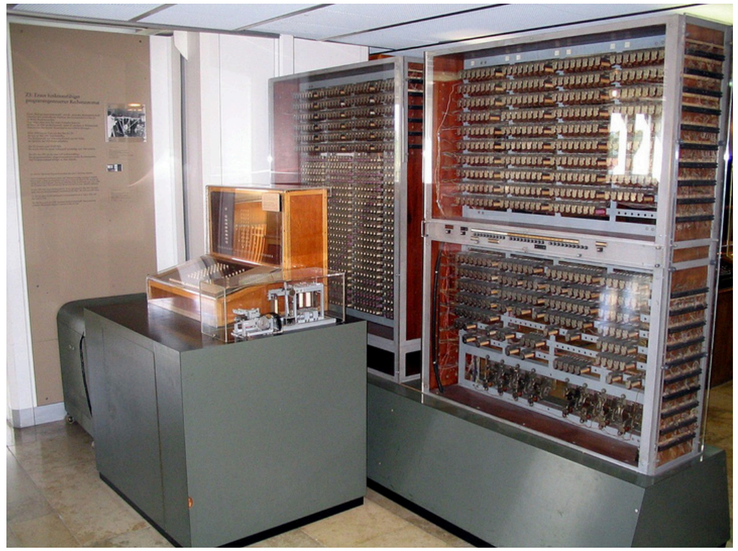
G매크로 이용순서



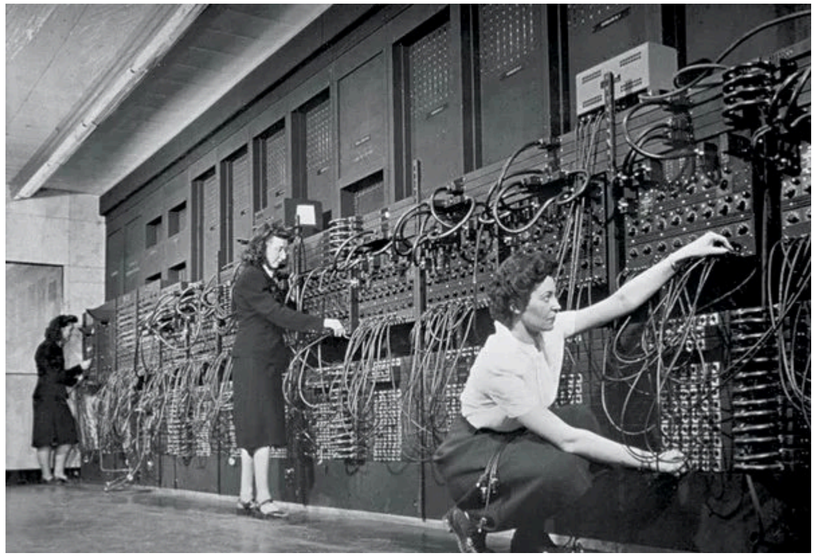
“계산 가능한 수와 결정문제의 응용”
"On computable numbers, with an application to the Entscheidungsproblem"
앨런 튜링: 튜링머신 (1936)



앨런 튜링: 콜로서스 (1943)



콘라트 추세: Z3 (1942)



모글리 & 애커트 : 애니악 (1945)

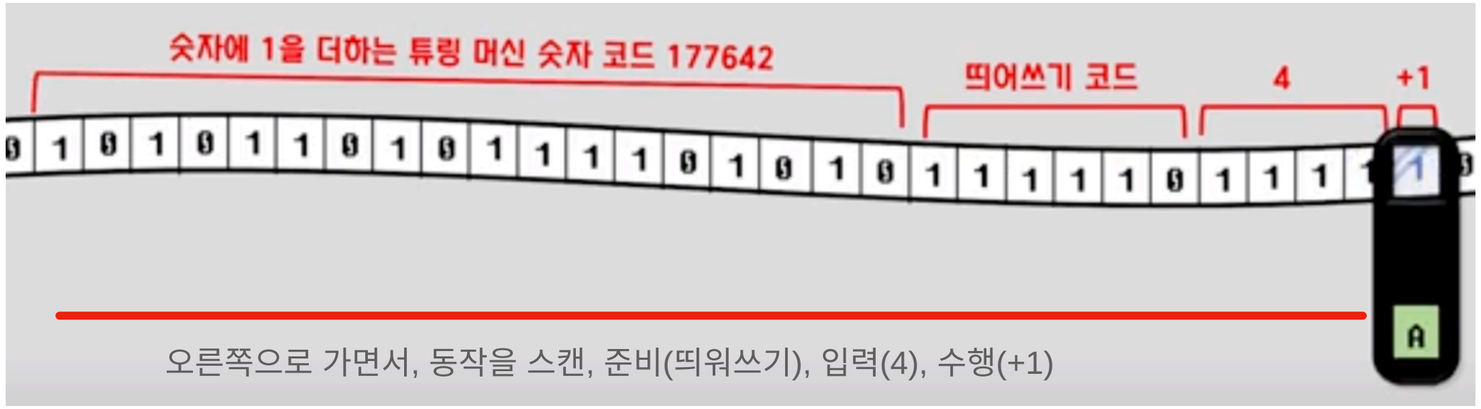
존 폰 노이만 : 이것저것.. (1956)

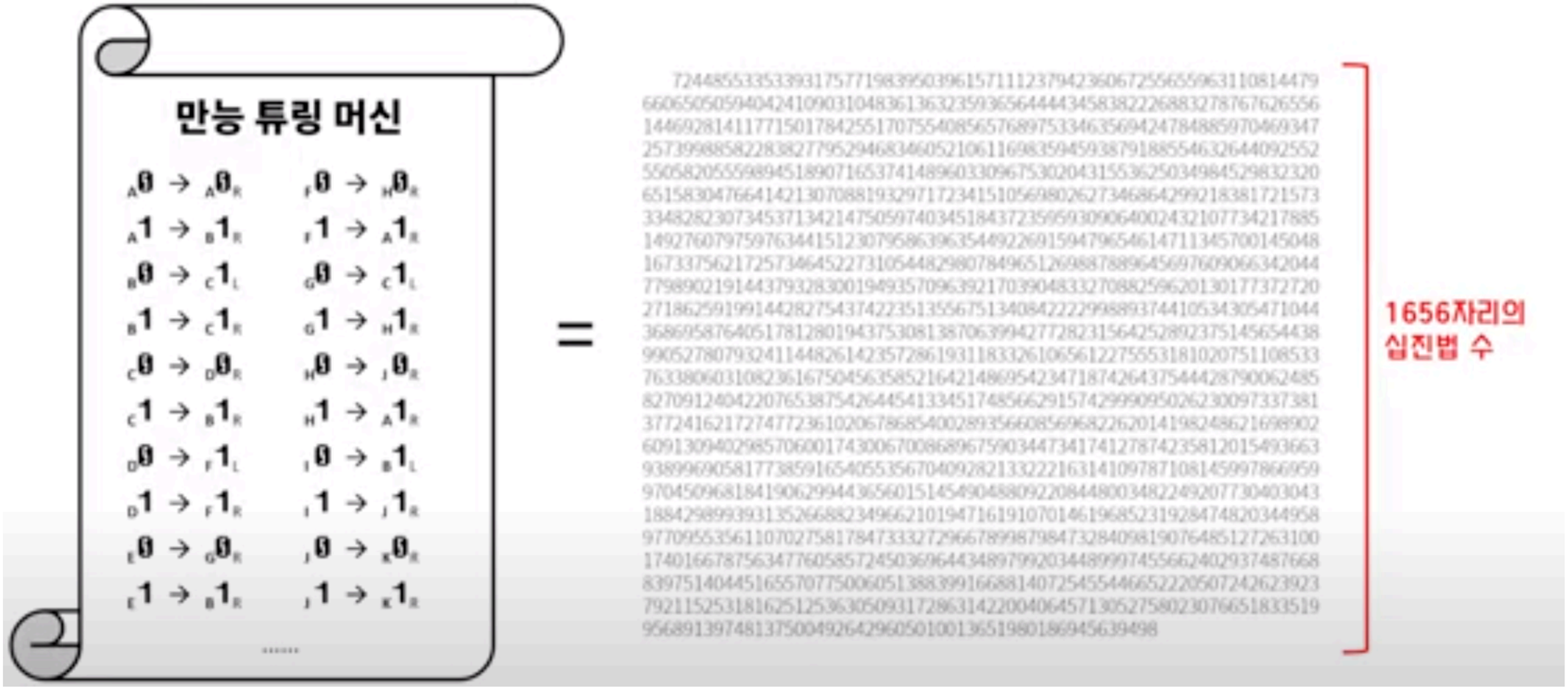
ㅋㅋ

컴퓨터

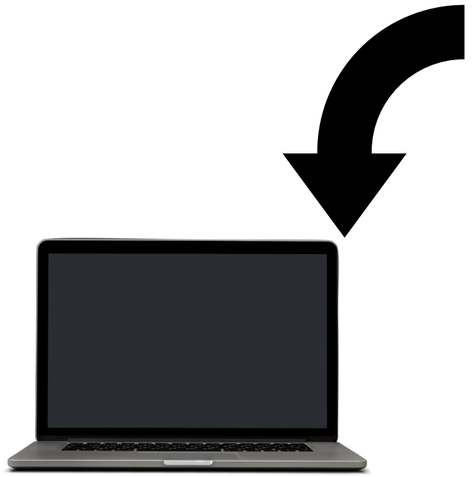
# Universal Turing machine
## 동작을 읽고 수행하는?

- 만능 = 여러 기능을 수행
- 프로그래밍 = “수행해야될 기능을 주고” + “동작을 수행”




만능 튜링 머신
$_A0 \rightarrow {}_A0_R$
$_A1 \rightarrow {}_B1_R$
$_B0 \rightarrow {}_C1_L$
$_B1 \rightarrow {}_C1_R$
$_C0 \rightarrow {}_D0_R$
$_C1 \rightarrow {}_B1_R$
$_D0 \rightarrow {}_F1_L$
$_D1 \rightarrow {}_F1_R$
$_E0 \rightarrow {}_G0_R$
$_E1 \rightarrow {}_B1_R$
$_F0 \rightarrow {}_H0_R$
$_F1 \rightarrow {}_A1_R$
$_G0 \rightarrow {}_C1_L$
$_G1 \rightarrow {}_H1_R$
$_H0 \rightarrow {}_J0_R$
$_H1 \rightarrow {}_A1_R$
$_I0 \rightarrow {}_B1_L$
$_I1 \rightarrow {}_J1_R$
$_J0 \rightarrow {}_K0_R$
$_J1 \rightarrow {}_K1_R$
.....
=
724485533533931757719839603961571112379423606725565596311081447966065050594042410903104836136323593656444434583822268832787676265561446928141177150178425517075540856576897533463569424784885970469347257399885822838277952946834605210611698359459387918855463264409255250558205559894518907165374148960330967530204315536250349845298323206515830476641421307088193297172341510569802627346864299218381721573334828230734537134214750597403451843723569938906400243210773421788514927607975976344151230795863963544922691594796546147113457001450481673375621725734645227310544829807849651269887889645697609066342044779890219144379328300194935709639217039048332708825962013017737272027186259199144282754374223513556751340842222988893744105343054710443686958764051781280194375308138706399427728231564252892375145654438990527807932411448261423572861931183326106561227555318102075110853376338060310823616750456358521642148695423471874264375444287900624858270912404220765387542644541334517485662915742999095026230097337381377241621727477236102067868540028935660856968226201419824862169890260913094029857060017430067008689675903447341741278742358120154936639389969058177385916540553567040928213322216314109787108145997866959970450968184190629944365601514549048809220844800348224920773040304318842989939313526688234966210194716191070146196852319284748203449589770955356110702758178473332729667899879847328409819076485127263100174016678756347760585724503696443489799203448999745566240293748766883975140445165570775006051388399166881407254554466522205072426239237921152531816251253630509317286314220040645713052758023076651833519956891397481375004926429605010013651980186945639498
1656자리의
십진법 수

만능튜링 머신 또한 튜링 머신이다



LEAGUE of LEGENDS

VALORANT

Nintendo
THE LEGEND OF
ZELDA
TEARS OF THE KINGDOM

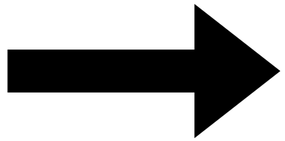
PUBG
배틀그라운드

# 설치하고 실행?.. 튜링머신?

## 힐베르트의 결정 문제
(Hilbert's Decision Problem)

1928

모든 수학적 문제에 대해서
그 문제가 참인지 거짓인지를
확실하게 결정해 줄
명확한 방법이
존재할 지를 증명하시오



## 튜링의 정지 문제
(Turing's Halting Problem)

1936

어떤 튜링 기계가
입력 데이터를 받았을 때,
궁극적으로 정지할지
안할지를 알 수 있을까?

무한 로딩의 지옥속에서...
이게 오류가 나고 죽을것인지?
언젠가 끝날것인지?

# 가정 HN-TM = "모든 프로그램 P와 입력 I에 대해 멈추는지 여부를 판별하는 알고리즘이 존재!

즉, 어떤 프로그램 P와 입력 I에 대해 H(P, I)라는 판별기가 존재:
H(P, I) = YES → 프로그램 P가 입력 I에서 멈춘다.
H(P, I) = NO → 프로그램 P가 입력 I에서 멈추지 않는다.(무한루프)

HN-TM이 존재한다고 가정하고, 이를 이용하여 다음과 같은 프로그램 D(x)를 구성하자

```
def D(x):
    if H(x, x) == YES:   # x(x)가 멈춘다면
        while True:      # 무한 루프 실행 (멈추지 않음)
            pass
    else:
        return 0         # 멈춘다고 예측되면 즉시 종료
```

## D(D)를 실행하면 어떻게 될까?

- **Case 1: H(D, D) = YES (D(D)가 정지함)**
  → `D(D)`는 무한 루프에 빠져야 함 (STOP 안 함) → 모순 발생!
- **Case 2: H(D, D) = NO (D(D)가 정지하지 않음)**
  → `D(D)`는 즉시 종료해야 함 → 또다시 모순 발생!

"튜링의 자기 참조 패러독스" **모순** 즉, HN-TM이 존재한다고 가정하면 자기모순(self-contradiction)이 발생
HN-TM은 존재하지 않는다.

**이 문장은 거짓이다.**

참이면 거짓이고, 거짓이면 참인 역설 발생.

**H(D, D) → "이 프로그램은 멈출 거야."**

멈춘다고 하면 멈추지 않고, 멈추지 않는다고 하면 멈추는 모순 발생.

# 튜링 머신으로 불가능한 것

**처치-튜링 논제**

- 모든 "직관적으로 계산 가능"한 함수는 튜링 기계로 계산 가능하다.
- 즉, 튜링 기계가 계산할 수 없는 함수는 "계산 가능"하다고 볼 수 없다.
- 람다 계산법(Church)과 튜링 기계(Turing)의 계산 능력은 동등하다.

튜링 기계가 할 수 있는 것 = **계산 가능한 모든 것 = 계산이 아닌건 튜링기계가 못함**

물론, 물리적인 한계가 존재하지!!

# 컴퓨터는 추상화된 만능튜링머신(UTM)!

**(1) 무한 테이프의 문제**

튜링 머신은 무한한 테이프(메모리)를 가.

> 하지만 현실에서는 유한한 메모리(RAM, 저장 공간)밖에 없기 때문에 이론적인 UTM을 완벽하게 구현할 수 없음.

**(2) 실행 속도의 문제**

UTM은 다른 튜링 머신을 해석(interpret)하면서 실행

> 느려질 수밖에 없음. (에뮬레이터로 게임하면 느림!)

**(3) 튜링 기계로 해결할 수 없는 문제**

정지 문제(Halting Problem)처럼 어떤 프로그램이 멈출지 여부를 판별하는 문제는 결정 불가능해.

> 즉, UTM이 모든 문제를 해결할 수 없음. UTM이 실행할 수 있는 프로그램 중에는 절대로 멈추지 않는 프로그램도 존재할 수 있어.

# 추상화!

# 생각은 계산인가?

생각이란 무엇인가

사람 = 생각함

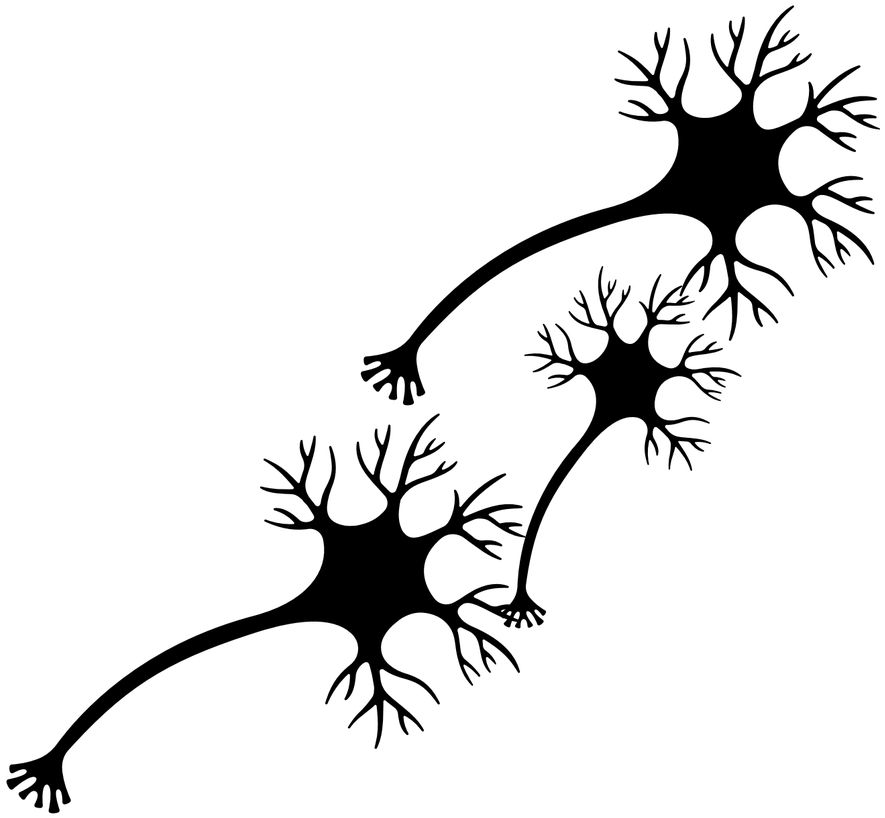
컴퓨터 = 계산하는 기계

계산할 수 있는 사람?

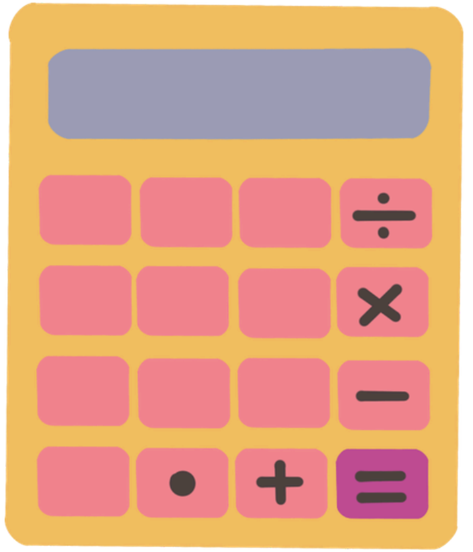
생각할 수 있는 사람

계산할 수 있는 기계

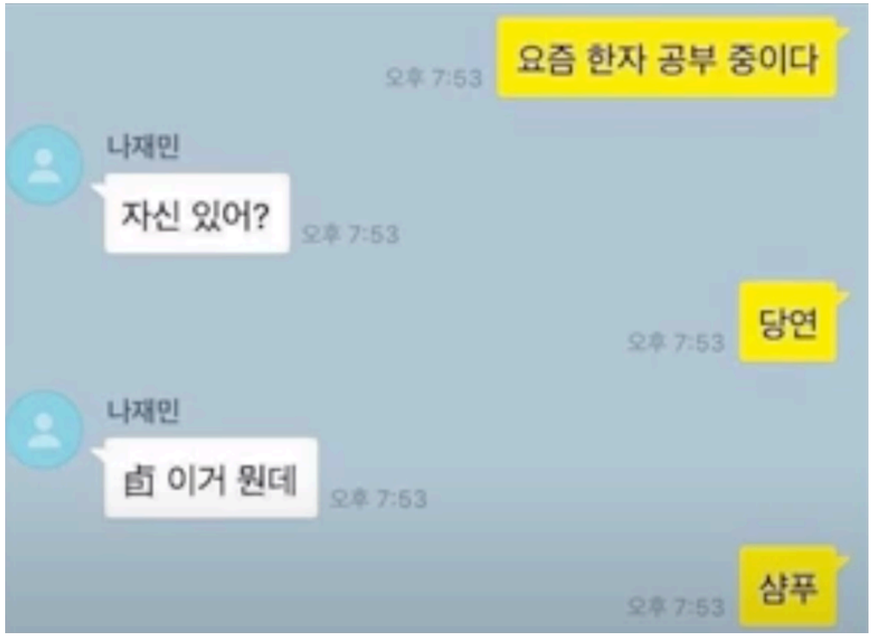
생각할 수 있는 기계?



?

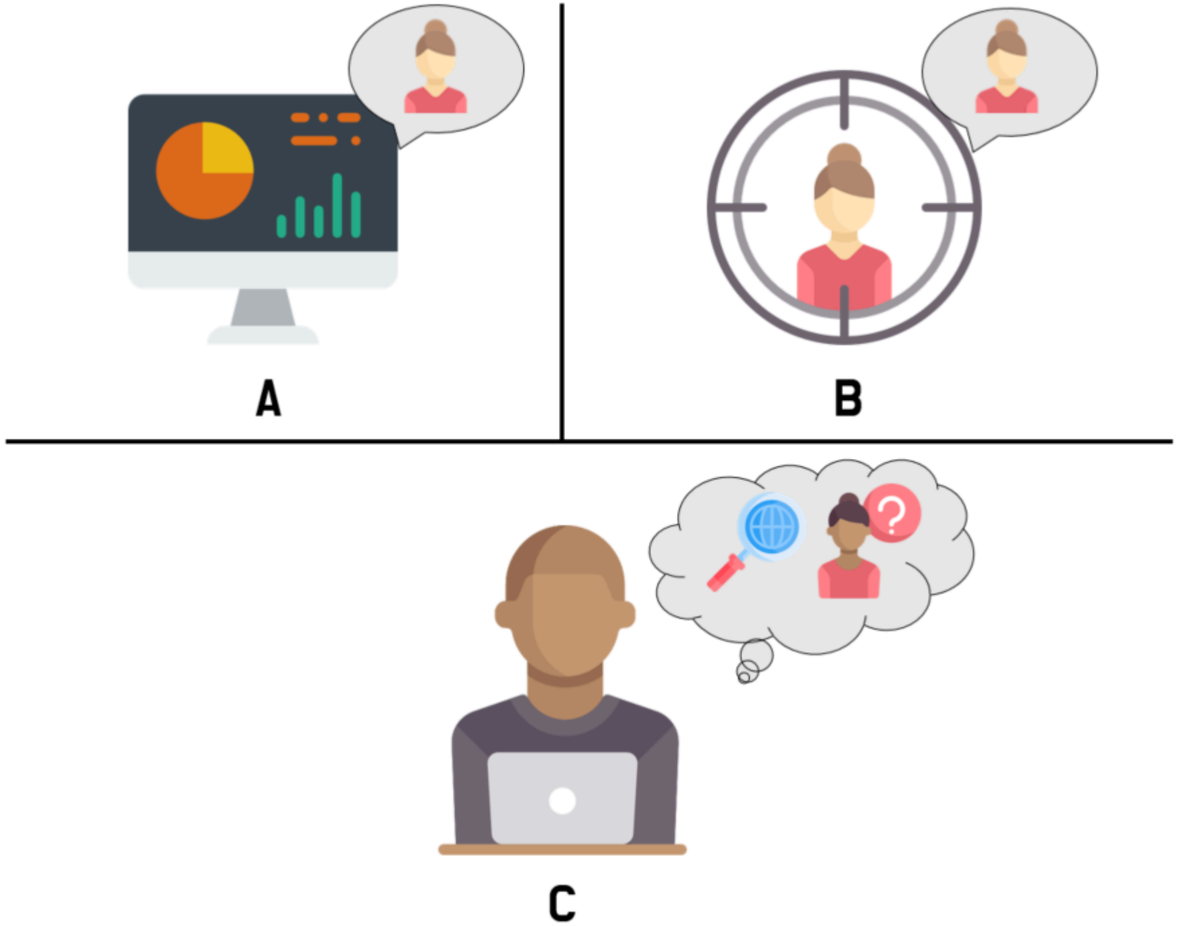


구분되지 않으면 인정해줘야되는거 아닐까?
by앨런튜링




맥락에 대한 파악이 있어야한다.

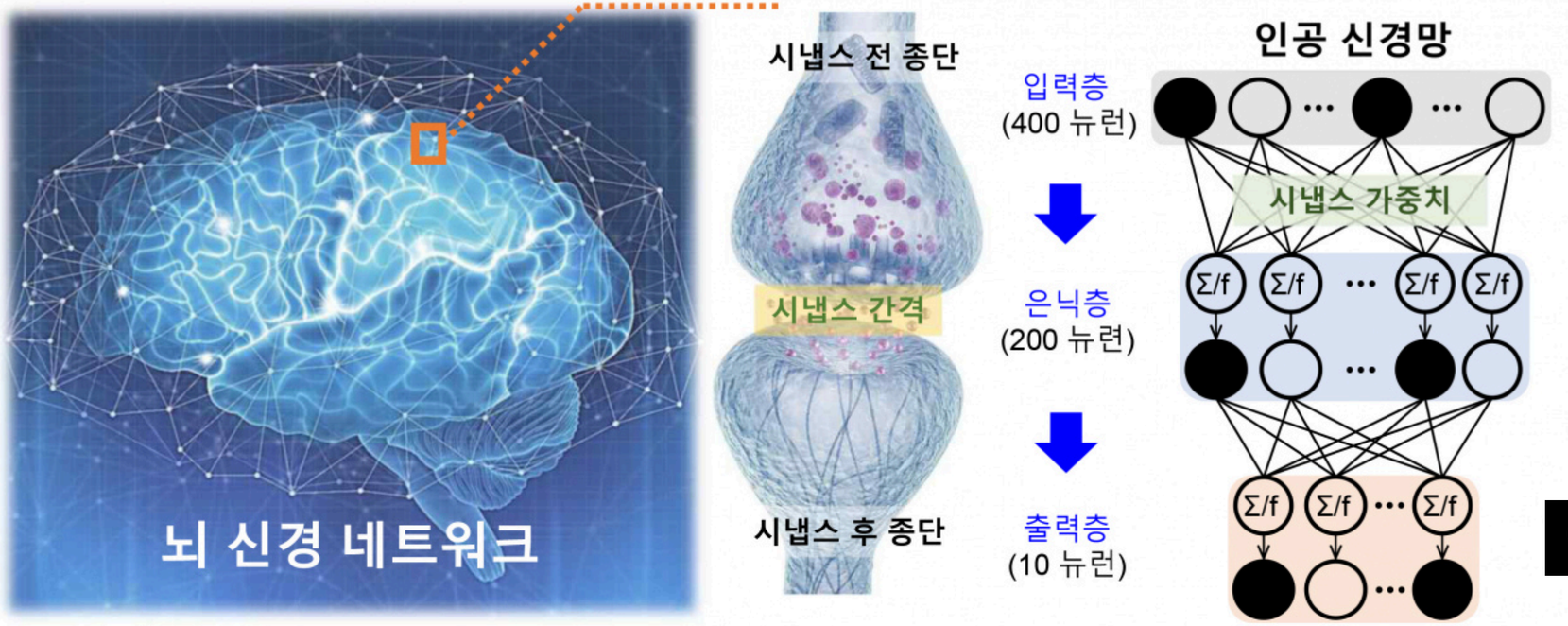
생각 = 연속적인 대화



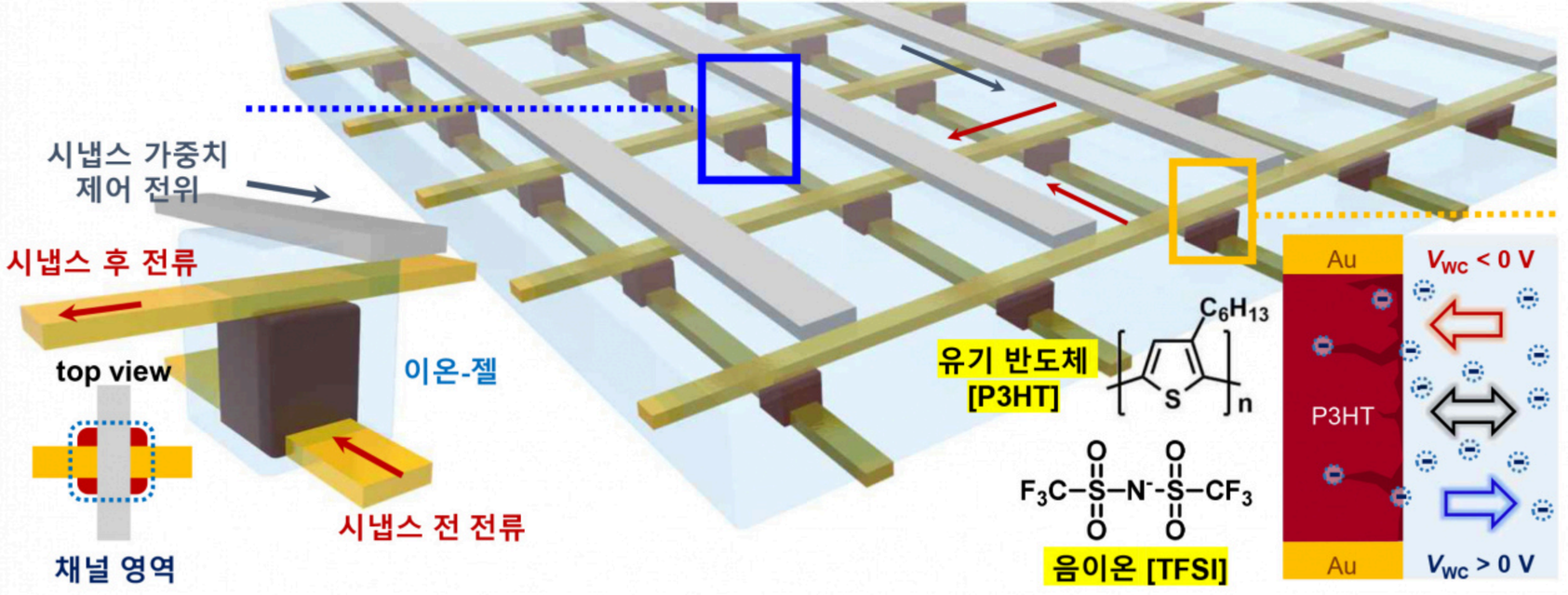

A. M. Turing (1950)
Computing Machinery and Intelligence.
Mind 49: 433-460.

중국어방 논변

만약 성공적으로 중국어 OUT을 받았다면.. 그것은 중국어를 구사할 수 있다고 봐도 되는가?




아예 신경망 동작을 모방하면 어때?



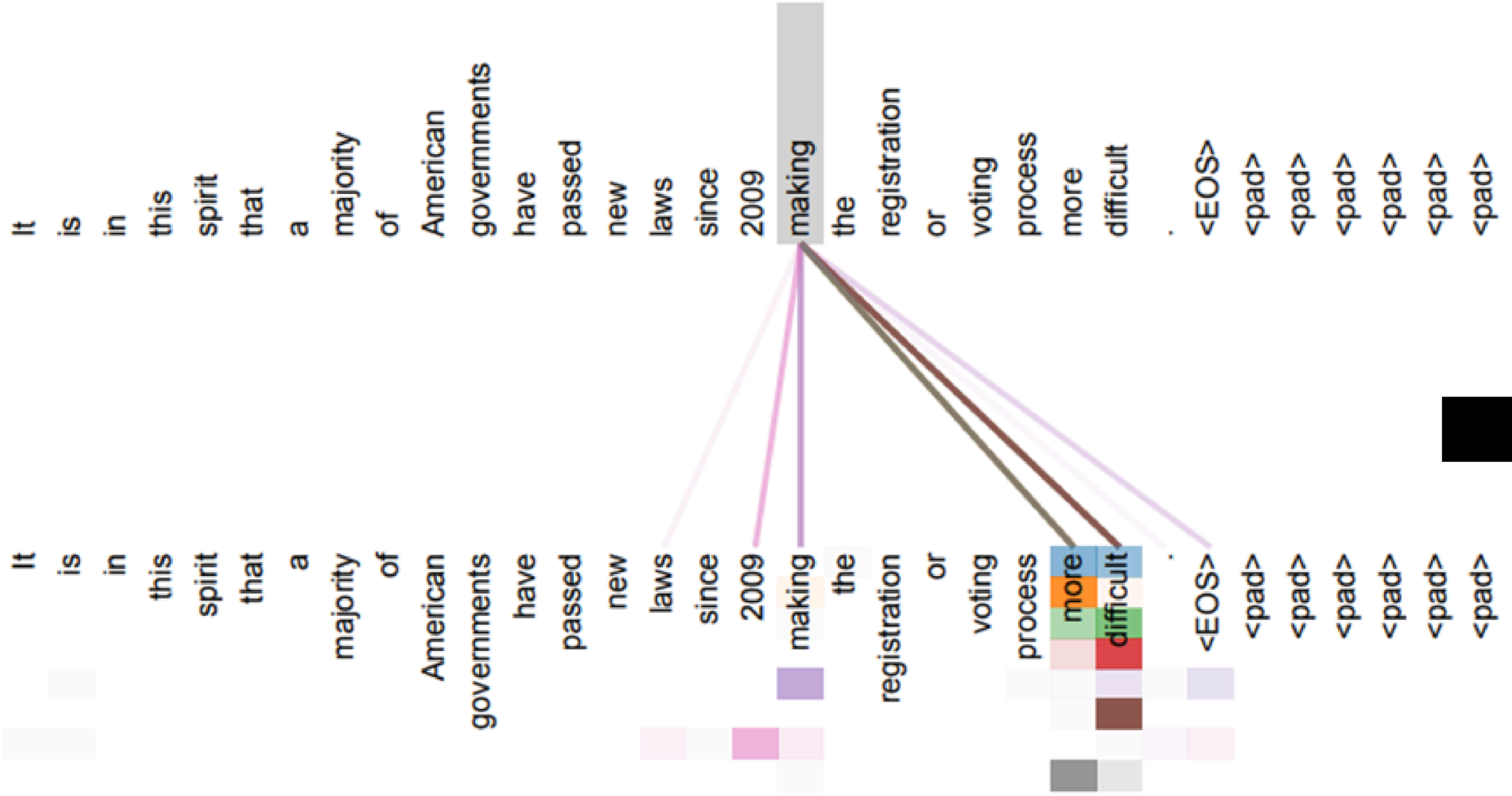



Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb ‘making’, completing the phrase ‘making...more difficult’. Attentions here shown only for the word ‘making’. Different colors represent different heads. Best viewed in color.

뇌세포로 만들지뭐!

**Cortical Labs**

We've combined lab-grown neurons with silicon chips and made it available to anyone, for first time ever.

corticallabs /

# 계산하는 기계는 생각하는 기계가 될 수 있을까?

## 인공지능을 만든 생각들의 역사와 철학

잭 코플랜드 지음
박영대 옮김
김재인 감수

Artificial Intelligence:
A Philosophical Introduction

에디토리얼

YES24

잭 코플랜드

강한 기호체계 가설
Strong Symbol Systems Hypothesis

생각, 의식 모두 계산이고 튜링머신임

VS

로저 펜로드

아님 뇌는 더 뛰어난 무언가임

물리적으로 중력을 아직 통합하지 못함
신경계에서도 양자 비슷한 일이 일어남
새로운게 있음

# 생각이란 무엇입니까?